جزوهٔ آموزشی · کنکور دکتری ۱۴۰۶ · مجموعهٔ ۲۳۵۴

درس ۷ از ۱۰ سرفصل

داده‌ساختارها و الگوریتم‌ها

درس‌نامهٔ کوتاه و فصل‌به‌فصل، مثال و تلهٔ تستی، تمرین با پاسخ، و آزمون آخر هر فصل از سؤال‌های واقعی کنکور

O(n log n)

۱۰	۴۰	۸۴
فصل	درس	سؤال واقعی کنکور با کلید رسمی

فصل‌های این جزوه

۱ تحلیل پیچیدگی، نمادهای مجانبی و روابط بازگشتی

۲ آرایه، لیست پیوندی، پشته و صف

۳ درهم‌سازی (Hashing)

۴ درخت‌ها (BST و AVL) و هیپ

۵ مرتب‌سازی

۶ جستجو و انتخاب (kامین عضو)

۷ تقسیم و حل، و الگوریتم‌های حریصانه

۸ الگوریتم‌های گراف

۹ برنامه‌نویسی پویا

۱۰ عقبگرد، شاخه و حد، و NPکامل (+ سؤال‌های متفرقه)

راهنمای استفاده از جزوه

هر فصل با یک برگهٔ رنگی شروع می‌شود و همیشه همین ترتیب را دارد: درس‌ها ← آزمون فصل ← کارت خلاصه ← پاسخ‌نامه.

✏️ مثال
یک مسئلهٔ حل‌شده برای اینکه قاعده را در عمل ببینی.

⚠️ تلهٔ تستی
جایی که طراح کنکور بیشتر از همه داوطلب‌ها را غافلگیر می‌کند.

💡 نکته و قاعده
میان‌بُرها و قاعده‌هایی که سر جلسه وقت می‌خرند.

✍️ تمرین
آخر هر درس؛ پاسخ‌ها در پاسخ‌نامهٔ آخر همان فصل است (با شمارهٔ صفحه).

📝 آزمون فصل
سؤال‌ها در کارت‌های جدا، با منبع و مربعی برای علامت غلط یا حدسی.

🗂️ کارت خلاصه
چکیدهٔ یک‌صفحه‌ای فصل برای مرور شب قبل از آزمون.

چرخهٔ مطالعهٔ هر فصل (یک بلوک ۹۰ دقیقه‌ای)

۱ — پیش‌آزمون
۵ دقیقه
سؤال اول آزمون فصل را قبل از خواندن امتحان کن

۲ — درس‌نامه
۳۵ دقیقه
مثال‌ها را قبل از دیدن حل، خودت امتحان کن

۳ — تمرین
۳۰ دقیقه
بدون نگاه به پاسخ‌نامه حل کن

۴ — آزمون فصل
۱۵+ دقیقه
با زمان، مثل جلسهٔ کنکور

۵ — دفتر خطا
۵ دقیقه
هر غلط یا حدسی = یک سطر در دفتر خطا

قانون طلایی: خواندن یعنی بلد بودن نیست. جزوه‌ای که روان خوانده شد ولی آزمونش را نزدی، یاد گرفته نشده است. معیار فقط **درصد تست** است، با فرمول نمرهٔ منفی: **(۳ × درست − غلط) ÷ (۳ × کل)**.

منبع سؤال‌ها: سؤال‌های آزمون فصل از کنکور **ارشد مهندسی کامپیوتر ۱۳۹۸ تا ۱۴۰۴** هستند و کلید آن‌ها **کلید رسمی سازمان سنجش** است. جاهایی که کلید رسمی مبهم یا مشکوک است، در راه‌حل توضیح داده شده. فهرست قابل کلیک است: روی هر عنوان بزن تا به همان صفحه بروی. در نمایشگر PDF، نشانک‌ها (bookmark) هم فصل‌ها و درس‌ها را نشان می‌دهند.

فهرست

روی هر سطر بزن تا مستقیم به همان صفحه بروی.

۱ تحلیل پیچیدگی، نمادهای مجانبی و روابط بازگشتی ۱۱ سؤال کنکور ۵

درس ۱ نمادهای مجانبی و مقایسهٔ رشد ۵
درس ۲ شمردن حلقه‌ها ۹
درس ۳ قضیهٔ اصلی (Master Theorem) ۱۶
درس ۴ سایر روش‌های حل رابطهٔ بازگشتی ۱۸
درس ۵ تحلیل سرشکن (Amortized) ۲۱
📝 آزمون فصل ۲۳ | 🗂️ کارت خلاصه ۲۵ | 🔑 پاسخ‌نامه ۲۶

۲ آرایه، لیست پیوندی، پشته و صف ۳ سؤال کنکور ۲۹

درس ۱ آرایه و لیست پیوندی ۲۹
درس ۲ پشته (LIFO) ۳۳
درس ۳ صف (FIFO) و صف دوطرفه ۳۷
📝 آزمون فصل ۴۱ | 🗂️ کارت خلاصه ۴۲ | 🔑 پاسخ‌نامه ۴۳

۳ درهم‌سازی (Hashing) ۶ سؤال کنکور ۴۵

درس ۱ جدول درهم‌سازی و زنجیره‌سازی ۴۵
درس ۲ آدرس‌دهی باز ۴۷
درس ۳ درهم‌سازی سراسری و کاربردها ۴۹
📝 آزمون فصل ۵۱ | 🗂️ کارت خلاصه ۵۲ | 🔑 پاسخ‌نامه ۵۳

۴ درخت‌ها (BST و AVL) و هیپ ۷ سؤال کنکور ۵۵

درس ۱ درخت دودویی و پیمایش‌ها ۵۵
درس ۲ درخت جستجوی دودویی (BST) ۵۸
درس ۳ AVL و چرخش ۶۰
درس ۴ هیپ (Heap) ۶۲
درس ۵ انتخاب ساختار ترکیبی ۶۵
📝 آزمون فصل ۶۷ | 🗂️ کارت خلاصه ۶۹ | 🔑 پاسخ‌نامه ۷۰

۵ مرتب‌سازی ۱۲ سؤال کنکور ۷۲

درس ۱ جدول اصلی ۷۲
درس ۲ مرتب‌سازی سریع (QuickSort) ۷۳
درس ۳ کران پایین و مرتب‌سازی غیرمقایسه‌ای ۷۴
درس ۴ ورودی‌های خاص ۷۴
📝 آزمون فصل ۷۶ | 🗂️ کارت خلاصه ۷۹ | 🔑 پاسخ‌نامه ۸۰

۶ جستجو و انتخاب (kامین عضو) ۹ سؤال کنکور ۸۲

درس ۱ جستجوی دودویی و تعمیم‌هایش ۸۲
درس ۲ حداقل مقایسه برای بیشینه، کمینه و دومین بیشینه ۸۳
درس ۳ انتخاب kامین عضو ۸۴
درس ۴ امید تعداد مقایسه ۸۴
📝 آزمون فصل ۸۶ | 🗂️ کارت خلاصه ۸۸ | 🔑 پاسخ‌نامه ۸۹

۷ تقسیم و حل، و الگوریتم‌های حریصانه ۱۱ سؤال کنکور ۹۱

درس ۱ الگوهای تقسیم و حل ۹۱
درس ۲ جریان بیشینه (در حد تست‌ها) ۹۲
درس ۳ حریصانه، کی درست است؟ ۹۳
درس ۴ کدگذاری هافمن ۹۴
درس ۵ مثال نقض ساختن برای حریصانه ۹۵
📝 آزمون فصل ۹۶ | 🗂️ کارت خلاصه ۹۹ | 🔑 پاسخ‌نامه ۱۰۰

۸ الگوریتم‌های گراف ★ ۱۳ سؤال کنکور ۱۰۳

درس ۱ BFS و DFS ۱۰۳
درس ۲ درخت پوشای کمینه (MST) ۱۰۴
درس ۳ کوتاه‌ترین مسیر ۱۰۵
درس ۴ جریان بیشینه ۱۰۷
درس ۵ گراف و ضرب ماتریس ۱۰۷
📝 آزمون فصل ۱۰۸ | 🗂️ کارت خلاصه ۱۱۲ | 🔑 پاسخ‌نامه ۱۱۳

۹ برنامه‌نویسی پویا ۵ سؤال کنکور ۱۱۶

درس ۱ کی برنامه‌نویسی پویا؟ ۱۱۶
درس ۲ مسائل کلاسیک ۱۱۷
📝 آزمون فصل ۱۲۰ | 🗂️ کارت خلاصه ۱۲۱ | 🔑 پاسخ‌نامه ۱۲۲

۱۰ عقبگرد، شاخه و حد، و NPکامل (+ سؤال‌های متفرقه) ۷ سؤال کنکور ۱۲۳

درس ۱ عقبگرد (Backtracking) ۱۲۳
درس ۲ شاخه و حد (Branch and Bound) ۱۲۴
درس ۳ P، NP، NPسخت، NPکامل ۱۲۴
درس ۴ متفرقه ۱۲۶
📝 آزمون فصل ۱۲۸ | 🗂️ کارت خلاصه ۱۳۰ | 🔑 پاسخ‌نامه ۱۳۱

فصل ۱

۱ تحلیل پیچیدگی، نمادهای مجانبی و روابط بازگشتی

۵ درس | ۱۱ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۱۱ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴، که **۸ تای آن‌ها در ۱۴۰۲ تا ۱۴۰۴** بوده؛ یعنی این روزها هر سال ۲ تا ۳ سؤال. پایهٔ همهٔ فصل‌های بعدی هم همین است.

⏱️ **زمان مطالعه** دو بلوک ۹۰ دقیقه‌ای (درس ۱ تا ۳، بعد درس ۴ و ۵) + ۲۵ دقیقه آزمون فصل.

نقشهٔ فصل:
۱ نمادهای مجانبی و مقایسهٔ رشد | ۲ شمردن حلقه‌ها | ۳ قضیهٔ اصلی (Master Theorem) | ۴ سایر روش‌های حل رابطهٔ بازگشتی | ۵ تحلیل سرشکن (Amortized) | 📝 آزمون فصل

درس ۱ نمادهای مجانبی و مقایسهٔ رشد

این درس به چه سؤالی جواب می‌دهد؟

«این الگوریتم با بزرگ شدن ورودی چقدر کند می‌شود؟» ما زمان را با ثانیه اندازه نمی‌گیریم، چون به سرعت کامپیوتر بستگی دارد. به‌جایش **تعداد قدم‌ها** را بر حسب اندازهٔ ورودی (n) می‌شماریم و فقط **شکل رشد** را نگه می‌داریم.

✏️ **مثال (چرا فقط شکل رشد؟)**

الگوریتم A دقیقاً $100n$ قدم و الگوریتم B دقیقاً n^2 قدم برمی‌دارد.

- با $n = 10$: ۱۰۰۰ ⇐ A قدم، ۱۰۰ ⇐ B قدم. B سریع‌تر است.
- با $n = 1000$: ۱۰۰ ⇐ A هزار، B ⇐ یک میلیون. A ده برابر سریع‌تر است.
- از $n = 100$ به بعد A همیشه برنده است. برای ورودی‌های بزرگ (که مسئلهٔ واقعی ماست)، ضریب ۱۰۰ مهم نیست؛ مهم این است که یکی «خطی» و دیگری «مربعی» رشد می‌کند.

نمادها به زبان ساده

نمادها مثل علامت‌های مقایسهٔ عددها هستند، ولی برای **سرعت رشد** توابع: O مثل ≤، Ω مثل ≥، Θ مثل =، o مثل <، ω مثل >. «از یک جایی به بعد» ($n \geq n_0$) و «با یک ضریب ثابت» (c) یعنی رفتار اول کار و ضریب‌ها را نادیده بگیر.

💡 شهود

برای n خیلی بزرگ فقط سریع‌ترین جمله مهم است و ضریب‌ها مهم نیستند. $3n^2 + 100n$ مثل n^2 رفتار می‌کند.

نماد	معنی ساده	تعریف
$f = O(g)$	f حداکثر هم‌اندازهٔ g رشد می‌کند (≤)	ثابت‌های c و n_0 هستند که برای $n \geq n_0$ ، $f(n) \leq c \cdot g(n)$
$f = \Omega(g)$	حداقل (≥)	$f(n) \geq c \cdot g(n)$
$f = \Theta(g)$	هم‌اندازه (=)	هم O هم Ω
$f = o(g)$	اکیداً کندتر (<)	$f/g \to 0$
$f = \omega(g)$	اکیداً سریع‌تر (>)	$f/g \to \infty$

✏️ مثال برای هر سطر جدول

$f(n) = 3n^2 + 100n$.

- **O:** $f = O(n^2)$ با $c = 4$ و $n_0 = 100$ ، چون $3n^2 + 100n \leq 4n^2$ یعنی $100n \leq n^2$ یعنی $n \geq 100$.
- **Ω:** $f = \Omega(n^2)$ با $c = 3$ ، چون برای هر n داریم $3n^2 + 100n \geq 3n^2$.
- **Θ:** هر دو برقرار است، پس $f = \Theta(n^2)$.
- **o:** $n = o(n^2)$ چون $n/n^2 = 1/n \to 0$. ولی $f \neq o(n^2)$ چون $f/n^2 \to 3$ ، نه صفر.
- **ω:** $n^2 = \omega(n \log n)$ چون $n^2/(n \log n) = n/\log n \to \infty$.

چطور نمادها را در تست به کار ببرم؟

۱. در هر تابع، **سریع‌ترین جمله** را پیدا کن و ضریبش را بینداز: $5n^3 + 2^n \Leftarrow 2^n$.

۲. دو تابع را با جای‌شان روی نردبان زیر مقایسه کن.

۳. اگر روی نردبان نبودند، از هر دو لگاریتم بگیر (بخش بعد).

نردبان رشد

$1 < \log\log n < \log n < (\log n)^k < n^{\varepsilon} < n < n \log n < n^2 < n^k < 2^n < 3^n < n! < n^n$

✏️ مثال (n = ۱۶، با log در مبنای ۲)

$\log\log n = 2$ ، $\log n = 4$ ، $n = 16$ ، $n \log n = 64$ ، $n^2 = 256$ ، $2^n = 65536$ ، $n! \approx 2\times10^{13}$ ، $n^n \approx 1.8\times10^{19}$.

⚠️ تلهٔ تستی

نردبان برای n **خیلی بزرگ** است: در $n = 16$ ، $(\log n)^{10} = 4^{10} \approx 10^6$ از $n^2 = 256$ بزرگ‌تر است، ولی بالاخره n از هر $(\log n)^k$ جلو می‌زند.

ابزار مقایسه: لگاریتم بگیر

مثال ✏️

$n^{\log n}$ یا $(\log n)^n$ ؟ لگاریتم اولی $\log^2 n$ است و لگاریتم دومی $n \cdot \log\log n$. دومی خیلی بزرگ‌تر است، پس $(\log n)^n$ بزرگ‌تر است. ⚠️ اگر لگاریتم‌ها فقط **در ضریب** فرق کنند، نتیجه نگیر: $\log 4^n = 2\log 2^n$ ولی $4^n \neq \Theta(2^n)$.

یادآوری لگاریتم (پیش‌نیاز تله‌ها)

$\log_2 n$ یعنی «چند بار باید ۲ را در خودش ضرب کنم تا به n برسم؟». مثلاً $\log_2 8 = 3$ چون $2 \times 2 \times 2 = 8$ ، و $\log_2 1024 = 10$. در این کتاب هر جا مبنا ننوشته باشیم، مبنا ۲ است. سه قانون برای همهٔ این بخش کافی است:

- **ضرب ⇐ جمع:** $\log(a \cdot b) = \log a + \log b$. مثال: $\log(8 \times 4) = 3 + 2 = 5 = \log 32$.
- **توانِ داخل ⇐ ضریب بیرون:** $\log(a^k) = k \cdot \log a$. مثال: $\log(8^2) = 2 \times 3 = 6 = \log 64$.
- **عوض کردن مبنا فقط یک ضریب ثابت است:** $\log_{10} n = \log_2 n / \log_2 10 \approx 0.3 \cdot \log_2 n$.

تله‌ها

۱. مبنای log مهم نیست ($\log_2 n = \Theta(\log_{10} n)$)، ولی **مبنای توان مهم است** ($2^n \neq \Theta(3^n)$).

۲. $\log n^2 = 2\log n = \Theta(\log n)$ ولی $\log^2 n = (\log n)^2$ بزرگ‌تر است.

۳. $\log(n^n) = n \log n$ و $\log(n!) = \Theta(n \log n)$ ؛ این دو Θ هم‌اند.

۴. $a^{(\log_b n)} = n^{(\log_b a)}$ ؛ مثلاً $2^{(\log_2 n)} = n$.

۵. $\sum 1/i = \Theta(\log n)$ (هارمونیک)، $\sum i^k = \Theta(n^{k+1})$ ، و در سری هندسی جملهٔ بزرگ غالب است.

۶. بعضی توابع **مقایسه‌ناپذیرند**؛ مثلاً اگر توان نوسان کند (مثل $n^{(1+\sin n)}$)، نه O است نه Ω.

توضیح هر تله با عدد

مثال (تلهٔ ۱: مبنای log در برابر مبنای توان) ✏️

$\log_2 n = \log_{10} n / \log_{10} 2 \approx 3.32 \cdot \log_{10} n$ ؛ نسبت این دو همیشه همان ۳٫۳۲ است (ضریب ثابت) ⇐ Θ برابر. ولی $3^n/2^n = 1.5^n$ با بزرگ شدن n به بی‌نهایت می‌رود (برای $n = 10$ حدود ۵۸، برای $n = 50$ بیش از ۶۰۰ میلیون) ⇐ ضریب ثابت نیست ⇐ Θ متفاوت.

✏️ مثال (تلهٔ ۲: جای توان)

فرق فقط در **جای** توان است:

- $\log n^2$ یعنی $\log(n^2)$: توان **داخل** log است ⇐ طبق قانون دوم می‌شود $2\cdot\log n$ ؛ فقط یک ضریب ۲ ⇐ $\Theta(\log n)$.
- $\log^2 n$ یعنی $(\log n)^2$: خودِ جواب log به توان ۲ رسیده ⇐ واقعاً بزرگ‌تر.
- با عدد: $n = 1024$ ($\log n = 10$): $\log n^2 = 20$ ولی $\log^2 n = 100$. با $n \approx 10^6$ ($\log n = 20$): 40 در برابر 400 . نسبت مدام بزرگ‌تر می‌شود، پس $\log^2 n$ مرتبهٔ بالاتری دارد.

✏️ مثال (تلهٔ ۳: log(n!))

$n! = 1 \times 2 \times \ldots \times n$. طبق قانون اول، $\log(n!) = \log 1 + \log 2 + \ldots + \log n$:

- **سقف:** هر جمله حداکثر $\log n$ است و n جمله داریم ⇐ $\log(n!) \le n \log n$.
- **کف:** نیمهٔ بزرگ‌تر جمله‌ها (از $n/2$ تا n) هر کدام حداقل $\log(n/2)$ است ⇐ $\log(n!) \ge (n/2)\cdot\log(n/2)$ ، که آن هم از مرتبهٔ $n \log n$ است.
- پس $\log(n!) = \Theta(n \log n)$. با عدد: $n = 1024$: $\log_2(1024!) \approx 8769$ ، $n \log n = 10240$ و کف $= 512 \times 9 = 4608$؛ جواب بین این دو است.
- $\log(n^n) = n\cdot\log n$ دقیقاً (قانون دوم). پس **لگاریتمِ** $n!$ و n^n هم‌مرتبه‌اند.

⚠️ تلهٔ تستی

این یعنی لگاریتم‌هایشان هم‌مرتبه‌اند، **نه خودشان**. $n!$ خیلی کوچک‌تر از n^n است ($n!/n^n \to 0$). همان قاعدهٔ «ابزار مقایسه» بالا: وقتی لگاریتم‌ها فقط در ضریب فرق دارند، دربارهٔ خود توابع نتیجه نگیر.

✏️ مثال (تلهٔ ۴: جابه‌جایی پایه و توان)

$a^{(\log_b n)} = n^{(\log_b a)}$. دلیل: از دو طرف $\log_b$ بگیر؛ هر دو طرف می‌شوند $(\log_b n)\cdot(\log_b a)$.

- $2^{(\log_2 n)} = n^{(\log_2 2)} = n^1 = n$.
- $4^{(\log_2 n)} = n^{(\log_2 4)} = n^2$. چک با $n = 8$: $4^3 = 64 = 8^2$.✓
- $3^{(\log_2 n)} = n^{(\log_2 3)} \approx n^{1.585}$. این همان زمان الگوریتم کاراتسوبا است.
- **کاربرد:** در قضیهٔ اصلی (درس ۳) تعداد برگ‌های درخت بازگشت $a^{(\log_b n)}$ است و همین تبدیل آن را به $n^{(\log_b a)}$ می‌رساند.

✏️ مثال (تلهٔ ۵: سه جمع پرکاربرد)

- **هارمونیک:** $1 + 1/2 + 1/3 + \dots + 1/n \approx \ln n$. با $n = 1024$: جمع ≈ 7.5 و $\ln 1024 \approx 6.9$ ؛ هم‌مرتبه‌اند.
  شهود: جمله‌ها را دسته کن: $1/2$ ، $1/3 + 1/4$ ، $1/5 + \dots + 1/8$ ، ...؛ هر دسته حدود ½ تا ۱ است و تعداد دسته‌ها $\log_2 n$ است.
- **توان‌ها:** $1 + 2 + \dots + n = n(n+1)/2 = \Theta(n^2)$ و $1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6 = \Theta(n^3)$. شهود: n جمله، هر کدام حداکثر n^k ⇐ سقف n^{k+1} ؛ و نصف جمله‌ها حداقل $(n/2)^k$ ⇐ کف هم از همین مرتبه.
- **هندسی:** اگر هر جمله دو برابر قبلی است، **آخرین** جمله غالب است: $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k$.
  اگر هر جمله نصف قبلی است، **اولین** جمله غالب است: $1 + 1/2 + 1/4 + \dots < 2$ (یعنی $\Theta(1)$).

✏️ مثال (تلهٔ ۶: مقایسه‌ناپذیر)

$f = O(g)$ یعنی «از یک جایی به بعد، **همیشه** $f \le c \cdot g$ ». اگر f بالا و پایین برود، ممکن است نه همیشه زیر g بماند نه همیشه بالای آن.

- **مثال ساده:** $f(n) = n$ برای n زوج و $f(n) = 1$ برای n فرد؛ و $g(n) = \sqrt{n}$. برای n های زوج f از g خیلی بزرگ‌تر است (پس $f \neq O(g)$) و برای n های فرد خیلی کوچک‌تر (پس $f \neq \Omega(g)$).
- **همان در کتاب:** توان $1 + \sin n$ بین ۰ و ۲ نوسان می‌کند، پس $n^{(1+\sin n)}$ گاهی نزدیک n^2 و گاهی نزدیک ۱ است ⇐ نسبت به n نه O است نه Ω.
- **در تست:** اگر گزینه‌ای مثل «هیچ‌کدام» یا «نه O نه Ω» دیدی و یکی از توابع نوسانی بود (sin، cos، یا تعریف جدا برای زوج و فرد)، جدی بگیرش.

✍️ تمرین درس ۱ — اول خودت حل کن، بعد پاسخ‌نامه

۱. $\log(n!)$ از چه مرتبه‌ای است؟

۲. درست یا نادرست: $2^{n+1} = \Theta(2^n)$ ؛ $2^{2n} = O(2^n)$.

۳. به‌ترتیب صعودی مرتب کن: $n \log n$ ، $n^{1.5}$ ، $2^{(\log_2 n)}$ ، $(\log n)^{10}$

۴. $n^{(1/\log_2 n)}$ از چه مرتبه‌ای است؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۲۶

درس ۲ شمردن حلقه‌ها

این درس به چه سؤالی جواب می‌دهد؟

فقط یک سؤال: **«دستور درونی (مثلاً x++) روی هم چند بار اجرا می‌شود؟»** این عدد را بر حسب n می‌نویسیم و طبق درس ۱ فقط جملهٔ بزرگش را نگه می‌داریم. تقریباً هر تست «مرتبهٔ این کد چیست؟» با شش ابزار زیر حل می‌شود.

💡 **روش اصلی کل درس**

برای n کوچک (مثلاً ۸ یا ۱۶) یک **جدول ردگیری** بکش: یک ستون برای مقدارهای متغیر بیرونی و یک ستون برای «حلقهٔ درونی در این دور چند بار اجرا شد». بعد ستون دوم را جمع بزن. همهٔ مثال‌های این درس همین کار را می‌کنند؛ وقتی چند بار با دست انجامش بدهی، الگو را بدون جدول هم می‌بینی.

✏️ **مثال (یک حلقهٔ ساده)**

```
for (i = 1; i ⩽ n; i++)
    x++;
```

با $n = 5$:

i	۱	۲	۳	۴	۵
تعداد اجرای x++	۱	۱	۱	۱	۱

جمع ۵. در حالت کلی n بار ⇐ $\Theta(n)$. اگر گام $i \mathrel{+}= 3$ بود، i مقدارهای $1, 4, 7, \dots$ را می‌گرفت و حدود $n/3$ دور می‌شد، که هنوز $\Theta(n)$ است (ضریب ثابت مهم نیست).

ابزار ۱: دو حلقهٔ مستقل ⇐ ضرب کن

وقتی حد حلقهٔ درونی **به متغیر بیرونی بستگی ندارد**، حلقهٔ درونی در **هر** دور بیرونی همان تعداد دور را می‌زند. پس تعداد دورها در هم ضرب می‌شوند.

✏️ **مثال**

```
for (i = 1; i ⩽ n; i++)
    for (j = 1; j ⩽ n; j++)
        x++;
```

با $n = 3$:

i	j ها	تعداد اجرای x++ در این دور
۱	۱، ۲، ۳	۳
۲	۱، ۲، ۳	۳
۳	۱، ۲، ۳	۳
جمع		**۹**

۹ = 3×3 . مثل کلاسی با ۳ ردیف و هر ردیف ۳ صندلی. در حالت کلی $n \times n$ ⇐ $\Theta(n^2)$. اگر بیرونی تا n و درونی تا m برود، جواب $n \times m$ است.

ابزار ۲: حلقهٔ درونی به i وابسته است ⇐ جمع بزن

💡 قاعده

اگر حد حلقهٔ درونی به i بستگی دارد، **ضرب نکن، جمع بزن.** چون طول حلقهٔ درونی در هر دور فرق می‌کند.

✏️ مثال (درونی تا i)

```
for (i = 1; i ⩽ n; i++)
    for (j = 1; j ⩽ i; j++)
        x++;
```

با $n = 5$:

i	j ها	تعداد
۱	۱	۱
۲	۱، ۲	۲
۳	۱، ۲، ۳	۳
۴	۱ تا ۴	۴
۵	۱ تا ۵	۵
جمع		**۱۵**

$1 + 2 + 3 + 4 + 5 = 15 = 5 \cdot 6/2$. شکل کار یک پلکان است، یعنی تقریباً نصف یک مربع ۵×۵. در حالت کلی $n(n+1)/2 \approx n^2/2$ ⇐ $\Theta(n^2)$.

✏️ مثال (سه شکل رایج همین الگو، با n = ۵)

حلقهٔ درونی	تعداد در دور i	جمع برای n = ۵	مرتبه
for (j = 1; j <= i; j++)	i	1+2+3+4+5 = 15	$\Theta(n^2)$
for (j = i; j <= n; j++)	$n - i + 1$	5+4+3+2+1 = 15	$\Theta(n^2)$
for (j = 1; j < i; j++)	$i - 1$	0+1+2+3+4 = 10	$\Theta(n^2)$

هر سه، پلکانی‌اند و جمعشان حدود $n^2/2$ است. ⚠️ اینجا ضرب کردن ($n \times n$) اتفاقاً همان مرتبه را می‌دهد، ولی در ابزار ۴ جواب غلط می‌دهد؛ پس عادت کن جمع را بنویسی.

ابزار ۳: متغیر ضرب یا تقسیم می‌شود ⇐ لگاریتم

وقتی i هر دور **در عددی ثابت ضرب** (یا بر آن تقسیم) می‌شود، خیلی سریع به n می‌رسد. سؤال این است: «چند بار باید دو برابر کنم تا از n رد شوم؟» جواب حدود $\log_2 n$ است.

✏️ مثال (دو برابر شدن)

```
for (i = 1; i ⩽ n; i *= 2)
    x++;
```

با $n = 20$: i مقدارهای $1, 2, 4, 8, 16$ را می‌گیرد؛ بعد ۳۲ می‌شود که از ۲۰ بیشتر است ⇐ **۵** دور. چک با فرمول: $\lfloor \log_2 20 \rfloor + 1 = 4 + 1 = 5$. با $n = 1{,}000{,}000$ فقط ۲۰ دور. مثل کاغذی که هر بار نصف تا می‌شود.

✏️ مثال (نصف شدن و سه برابر شدن)

حلقه	مقدارهای i	تعداد دور
for (i = 20; i > 0; i /= 2)	20, 10, 5, 2, 1	۵ (حدود $\log_2 20$)
for (i = 1; i <= 100; i *= 3)	1, 3, 9, 27, 81	۵ (حدود $\log_3 100$)

مبنای log (۲ یا ۳) فقط یک ضریب ثابت است، پس هر دو $\Theta(\log n)$ اند.

✏️ مثال (به توان ۲ رساندن)

```
for (i = 2; i ⩽ n; i = i * i)
```

با $n = 300$: i مقدارهای $2, 4, 16, 256$ را می‌گیرد؛ بعد 65536 می‌شود ⇐ **۴** دور. هر دور **توانِ** ۲ دو برابر می‌شود ($2^1, 2^2, 2^4, 2^8$)، پس تعداد دورها «لگاریتمِ لگاریتم» است ⇐ $\Theta(\log \log n)$. حتی با $n = 65536$ فقط ۵ دور.

ابزار ۴: بیرونی دو برابر می‌شود و درونی تا i می‌رود ⇐ $\Theta(n)$

⚠️ تلهٔ معروف

در این کد استدلال «بیرونی $\log n$ دور × درونی حداکثر n دور = $n \log n$ » غلط است.

```
for (i = 1; i ⩽ n; i *= 2)
    for (j = 1; j ⩽ i; j++)
        x++;
```

✏️ **مثال (ردگیری با n = ۱۶)**

i	تعداد اجرای درونی (تا i)
۱	۱
۲	۲
۴	۴
۸	۸
۱۶	۱۶
جمع	**۳۱**

اگر ضرب می‌کردی: $5 \times 16 = 80$. ولی جواب واقعی ۳۱ است، کمتر از $2n = 32$. درونی فقط در **دور آخر** n بار اجرا شد؛ دورهای قبلی خیلی کوچک‌تر بودند.

چرا همیشه کمتر از ۲n؟ در دنبالهٔ دو برابر شونده، هر عدد از جمع همهٔ قبلی‌ها بزرگ‌تر است: $16 > 1 + 2 + 4 + 8 = 15$. پس کل جمع کمتر از دو برابر جملهٔ آخر است. مثل پس‌اندازی که هر ماه دو برابر ماه قبل است: پول ماه آخر از همهٔ ماه‌های قبل روی هم بیشتر است. جواب $\Theta(n)$.

ابزار ۵: گام حلقهٔ درونی i است ⇐ هارمونیک ⇐ $\Theta(n \log n)$

```
for (i = 1; i ⩽ n; i++)
    for (j = 1; j ⩽ n; j += i)    // ← تا جلو می‌پرد i هر بار
        x++;
```

حلقهٔ درونی از ۱ تا n با پرش‌های i تایی می‌رود، پس حدود n/i بار اجرا می‌شود: اول با پرش ۱ (n بار)، بعد با پرش ۲ (نصف)، بعد با پرش ۳ (یک‌سوم)، ...

✏️ **مثال (ردگیری با n = ۸)**

i	j ها	تعداد
۱	۱، ۲، ۳، ...، ۸	۸
۲	۱، ۳، ۵، ۷	۴
۳	۱، ۴، ۷	۳
۴	۱، ۵	۲
۵	۱، ۶	۲
۶	۱، ۷	۲
۷	۱، ۸	۲
۸	۱	۱
جمع		**۲۴**

در حالت کلی $n/1 + n/2 + n/3 + \ldots + n/n = n \times (1 + 1/2 + \ldots + 1/n) \approx n \times \ln n$. چک: $8 \times \ln 8 \approx 16.6$ و $8 \times (1 + ½ + \ldots + ⅛) \approx 21.7$؛ جواب واقعی ۲۴ از همین مرتبه است (اختلاف به خاطر گرد شدن رو به بالای هر سطر است). جواب $\Theta(n \log n)$.

💡 **نکتهٔ حفظی:** سری هارمونیک $1 + 1/2 + \ldots + 1/n \approx \ln n$.

ابزار ۶: شرط با i*i ⇐ جذر

```
i = 1;
while (i * i ⩽ n) i++;
```

✏️ **مثال (ردگیری با n = ۲۰)**

i=۱ ⇐ $1 \le 20$ ✓؛ i=۲ ⇐ $4 \le 20$ ✓؛ i=۳ ⇐ $9 \le 20$ ✓؛ i=۴ ⇐ $16 \le 20$ ✓؛ i=۵ ⇐ $25 > 20$ ✗. پس **۴** دور، یعنی $\lfloor \sqrt{20} \rfloor = 4$. با $n = 100$ ده دور. جواب $\Theta(\sqrt{n})$.

ترکیب ابزارها

تست‌ها معمولاً دو ابزار را با هم ترکیب می‌کنند. همان جدول ردگیری را بکش.

✏️ مثال (ابزار ۲ + ابزار ۶)

```
for (i = 1; i ⩽ n; i++)
    for (j = 1; j * j ⩽ i; j++)
        x++;
```

حلقهٔ درونی برای هر i حدود $\sqrt{i}$ بار اجرا می‌شود (ابزار ۶) و چون به i وابسته است، جمع می‌زنیم (ابزار ۲). با $n = 16$:

i	۱ تا ۳	۴ تا ۸	۹ تا ۱۵	۱۶
تعداد دور درونی ($\lfloor\sqrt{i}\rfloor$)	۱	۲	۳	۴
سهم در جمع	$3 \times 1 = 3$	$5 \times 2 = 10$	$7 \times 3 = 21$	$1 \times 4 = 4$

جمع $3 + 10 + 21 + 4 = 38$. در حالت کلی $\sqrt{1} + \sqrt{2} + \ldots + \sqrt{n} \approx (2/3)\cdot n^{1.5}$ (برای ۱۶: حدود ۴۳) ⇐ $\Theta(n\sqrt{n})$.

جدول خلاصهٔ شش الگو

کد	مرتبه	چرا
for(i=1;i<=n;i++) for(j=1;j<=i;j++)	$\Theta(n^2)$	$1+2+\ldots+n$
for(i=1;i<=n;i*=2)	$\Theta(\log n)$	i دو برابر می‌شود
for(i=2;i<=n;i=i*i)	$\Theta(\log \log n)$	$i = 2^{(2^k)}$
for(i=1;i<=n;i*=2) for(j=1;j<=i;j++)	$\Theta(n)$	$1+2+4+\ldots+n < 2n$
for(i=1;i<=n;i++) for(j=1;j<=n;j+=i)	$\Theta(n \log n)$	$\Sigma\, n/i$ هارمونیک
i=1; while(i*i<=n) i++;	$\Theta(\sqrt{n})$	

روش ۴ قدمی برای هر تست

۱. **برای هر حلقه:** جمع با عدد ثابت ⇐ حدود n دور؛ ضرب یا تقسیم در عدد ثابت ⇐ $\log n$ ؛ به توان رساندن ⇐ $\log \log n$؛ شرط $i*i <= n$ ⇐ $\sqrt{n}$.

۲. **درونی به متغیر بیرونی وابسته است؟** نه ⇐ ضرب کن. بله ⇐ جمع را بنویس.

۳. **جمع را با این چهار فرمول ساده کن:** $1+2+\ldots+n \approx n^2/2$ ؛ $1+2+4+\ldots+n < 2n$ ؛ $n+n/2+n/4+\ldots < 2n$ ؛ $n+n/2+n/3+\ldots+1 \approx n \ln n$.

۴. **فقط جملهٔ بزرگ را نگه دار.**

💡 **ترفند چک سر جلسه:** تعداد دورها را برای $n = 16$ و $n = 32$ بشمار و نسبت را ببین. اگر حدود ۲ برابر شد ⇐ n ؛ حدود ۴ برابر ⇐ n^2 ؛ حدود ۲٫۵ برابر ⇐ $n \log n$ ؛ تقریباً ثابت ماند (فقط ۱ واحد بیشتر) ⇐ $\log n$. مثلاً ابزار ۴: 31 و 63 (حدود ۲ برابر ⇐ $\Theta(n)$)؛ ترکیب بالا: 38 و 110 (حدود ۲٫۹ برابر، نزدیک $2^{1.5} \approx 2.8$ ⇐ $n\sqrt{n}$).

تمرین درس ۲ — اول خودت حل کن، بعد پاسخ‌نامه

۱. `for(i=n; i>1; i=i/2) for(j=0; j<i; j++) x++;`

۲. `for(i=1;i<=n;i++) for(j=1;j<=i;j++) for(k=1;k<=j;k++) x++;`

۳. `for(i=1;i<=n;i++) for(j=1;j<=n;j*=2) x++;`

پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۲۶

درس ۳ قضیهٔ اصلی (Master Theorem)

این درس به چه سؤالی جواب می‌دهد؟

الگوریتم‌های **تقسیم و حل** (مثل مرتب‌سازی ادغامی) مسئله را به چند تکهٔ کوچک‌تر می‌شکنند، هر تکه را با **خودشان** حل می‌کنند و بعد جواب‌ها را ترکیب می‌کنند. زمانشان یک رابطهٔ بازگشتی است:

$T(n) = a\cdot T(n/b) + f(n)$ یعنی: a زیرمسئله، هر کدام به اندازهٔ n/b، به‌علاوهٔ $f(n)$ کار برای شکستن و ترکیب.

مثال (خواندن رابطه)

مرتب‌سازی ادغامی آرایه را نصف می‌کند ($b = 2$)، هر نصفه را جدا مرتب می‌کند ($a = 2$) و ادغام دو نیمه n قدم طول می‌کشد ($f(n) = n$) ⇐ $T(n) = 2T(n/2) + n$.

شهود: دعوای ریشه و برگ‌ها

اگر بازگشت را به شکل درخت بکشی، ریشه $f(n)$ کار می‌کند و پایین‌تر گره‌ها زیاد می‌شوند ولی هر کدام کار کمتری دارند. تعداد برگ‌ها $n^{(\log_b a)}$ است. پس فقط یک سؤال مانده: **$f(n)$ (کار ریشه) بزرگ‌تر است یا n^c (کار برگ‌ها)، با $c = \log_b a$؟**

- برگ‌ها برنده ⇐ جواب n^c (حالت ۱).
- مساوی ⇐ همهٔ $\log n$ سطح به یک اندازه کار می‌کنند ⇐ $n^c \cdot \log n$ (حالت ۲).
- ریشه برنده ⇐ جواب $f(n)$ (حالت ۳).

مثال (قدم‌به‌قدم)

$T(n) = 4T(n/2) + n$.

۱. $a = 4$ ، $b = 2$ ⇐ $c = \log_2 4 = 2$ ⇐ کار برگ‌ها n^2 .

۲. کار ریشه $f(n) = n$ ، که از n^2 چندجمله‌ای کوچک‌تر است (با فاصلهٔ n^1).

۳. برگ‌ها برنده ⇐ حالت ۱ ⇐ $\Theta(n^2)$.

برای $T(n) = a·T(n/b) + f(n)$ با $a \geq 1$ و $b > 1$: **قدم ۱:** $c = \log_b a$ را حساب کن. **قدم ۲:** $f(n)$ را با n^c مقایسه کن.

نسخهٔ کلاسیک (سه حالت)

حالت	شرط	جواب
۱	f **چندجمله‌ای کوچک‌تر** از n^c است: $f = O(n^{c-\varepsilon})$	$\Theta(n^c)$
۲	$f = \Theta(n^c)$	$\Theta(n^c \cdot \log n)$
۳	f **چندجمله‌ای بزرگ‌تر** است: $f = \Omega(n^{c+\varepsilon})$ ، و $a·f(n/b) \leq \delta·f(n)$ با $\delta<1$	$\Theta(f(n))$

✏️ **مثال (چرا سه حالت؟ هزینهٔ سطح‌های درخت بازگشت)**

در سطح k، تعداد a^k گره داریم و هر کدام $f(n/b^k)$ کار می‌کند.

- $8T(n/2) + n^2$: سطح k هزینهٔ $8^k·(n/2^k)^2 = 2^k·n^2$ دارد ⇐ **رو به افزایش**؛ برگ‌ها غالب‌اند: $\Theta(n^3)$ (حالت ۱).
- $2T(n/2) + n$: هر سطح $2^k·(n/2^k) = n$ ، و $\log n$ سطح داریم: $\Theta(n \log n)$ (حالت ۲).
- $2T(n/2) + n^2$: سطح k هزینهٔ $n^2/2^k$ دارد ⇐ **رو به کاهش**؛ ریشه غالب است: $\Theta(n^2)$ (حالت ۳). شرط نظم: $2·(n/2)^2 = n^2/2 \leq ½·n^2$ ✓.

⚠️ **مهم برای تست**

وقتی f فقط به اندازهٔ یک ضریب $\log$ با n^c فرق دارد (مثل $n \log n$ در برابر n)، در هیچ‌کدام از سه حالت کلاسیک نمی‌افتد. اگر تست بپرسد «کدام را **نمی‌توان مستقیماً** با قضیهٔ اصلی حل کرد»، منظورش همین شکاف است (سؤال ۸ آزمون فصل).

حالت ۲ گسترده (برای پیدا کردن جواب)

اگر $f = \Theta(n^c \cdot \log^k n)$:

k	جواب
$k > -1$	$\Theta(n^c \cdot \log^{k+1} n)$
$k = -1$	$\Theta(n^c \cdot \log \log n)$
$k < -1$	$\Theta(n^c)$

✏️ **مثال (با $f + 2T(n/2)$، یعنی $c = 1$)**

- $f = n \log n$ ($k = 1$): جمع سطح‌ها $n \cdot (\log n + (\log n - 1) + \dots + 1) \approx n \log^2 n / 2$ ⇐ $\Theta(n \log^2 n)$.
- $f = n/\log n$ ($k = -1$): جمع سطح‌ها $n \cdot (1/\log n + \dots + 1/2 + 1)$ یک سری هارمونیک با $\log n$ جمله است ⇐ $\Theta(n \log \log n)$.
- $f = n/\log^2 n$ ($k = -2$): سری $\sum 1/j^2$ همگراست ⇐ $\Theta(n)$.

مثال‌ها

رابطه	c	جواب
$2T(n/2) + n$	۱	حالت ۲: $\Theta(n \log n)$ (مرتب‌سازی ادغامی)
$T(n/2) + 1$	۰	حالت ۲: $\Theta(\log n)$ (جستجوی دودویی)
$8T(n/2) + n^2$	۳	حالت ۱: $\Theta(n^3)$
$2T(n/2) + n^2$	۱	حالت ۳: $\Theta(n^2)$
$2T(n/2) + n \log n$	۱	گسترده با k=۱: $\Theta(n \log^2 n)$

✍️ **تمرین درس ۳** — اول خودت حل کن، بعد پاسخ‌نامه

۱. $9T(n/3) + n^2$

۲. $3T(n/4) + n \log n$

۳. $7T(n/2) + n^2$

۴. $2T(n/4) + \sqrt{n}$

۵. $2T(n/2) + n/\log n$

۶. $4T(n/2) + n^2 \log n$

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۲۶

درس ۴ سایر روش‌های حل رابطهٔ بازگشتی

این درس به چه سؤالی جواب می‌دهد؟

قضیهٔ اصلی فقط شکل $a \cdot T(n/b) + f(n)$ را حل می‌کند؛ یعنی وقتی مسئله به تکه‌های **هم‌اندازه** شکسته می‌شود. ولی در تست‌ها رابطه‌هایی می‌آید مثل $T(n/3) + T(2n/3) + n$ (تکه‌های نامساوی)، $T(\sqrt{n})$ (جذر) یا $T(n-1)$ (یکی‌یکی کم شدن). این درس برای هر کدام یک ابزار می‌دهد.

ایدهٔ مشترک همه: رابطهٔ بازگشتی می‌گوید «کار کل = کار همین مرحله + کار زیرمسئله‌ها». اگر همهٔ مرحله‌ها را باز کنی و کارشان را **جمع بزنی**، جواب به دست می‌آید. ابزارهای زیر فقط راه‌های سریع برای این جمع زدن‌اند.

✏️ مثال (باز کردن ساده)

$T(n) = T(n-1) + 1$ با $T(0) = 0$: $T(n) = 1 + T(n-1) = 1 + 1 + T(n-2) = \dots = n$ ⇐ $\Theta(n)$. هر قدم ۱ واحد کار دارد و n قدم داریم.

نقشهٔ انتخاب ابزار

شکل رابطه	ابزار
$aT(n/b) + f$	قضیهٔ اصلی (درس ۳)
تقسیم نامساوی: $T(\alpha n) + T(\beta n) + \dots$	درخت بازگشت یا آکرا-بازی
$T(\sqrt{n})$	تغییر متغیر
$T(n-1)$ یا $T(n-c)$	باز کردن و جمع زدن

الف) درخت بازگشت

برای $T(n) = T(\alpha n) + T(\beta n) + \Theta(n)$:

- اگر $\alpha+\beta < 1$ جواب $\Theta(n)$ است.
- اگر $\alpha+\beta = 1$ جواب $\Theta(n \log n)$ است (هزینهٔ هر سطح n و عمق $\log n$).
- اگر $\alpha+\beta > 1$ از آکرا-بازی استفاده کن.

✏️ مثال

- $T(n) = T(n/3) + T(2n/3) + n$: هر سطح کامل درخت هزینهٔ n دارد و عمق $\log_{3/2} n$ است ⇐ $\Theta(n \log n)$.
- $T(n) = T(n/2) + T(n/4) + n$: هزینهٔ سطح‌ها $n, 3n/4, 9n/16, \dots$ است (سری هندسی نزولی) ⇐ $\Theta(n)$.

ب) آکرا-بازی (ابزار نهایی تقسیم‌های نامساوی)

برای $T(x) = \Sigma\, a_i \cdot T(b_i \cdot x) + g(x)$:

۱. عدد p را طوری پیدا کن که $\Sigma\, a_i \cdot b_i^p = 1$. (ترفند: اول توان g را امتحان کن.)

۲. جواب $\Theta(x^p \cdot (1 + \int_1^x g(u)/u^{(p+1)}\, du))$ است.

انتگرال‌های لازم: $\int du/u = \ln x$ ، $\int du/(u \ln u) = \ln \ln x$. **نتیجهٔ عملی:** اگر $g = x^p \cdot \log^k x$ باشد، همان جدول «حالت ۲ گسترده» با $c = p$ جواب را می‌دهد.

✏️ مثال

- $T(n) = 2T(n/4) + T(n/2) + n$: توان g را امتحان کن، $p = 1$: $2\cdot(1/4) + 1/2 = 1$ ✓. پس $g = n^1\cdot\log^0 n$ با $k = 0$ ⇐ $\Theta(n \log n)$.
- $T(n) = 2T(n/2) + 8T(n/4) + n$ (اینجا $2\cdot½ + 8\cdot¼ = 3 > 1$): $p = 2$ چون $2\cdot(1/4) + 8\cdot(1/16) = 1$ ✓. انتگرال $\int_1^n u/u^3\,du = 1 - 1/n$ کراندار است ⇐ $\Theta(n^2)$.

ج) تغییر متغیر (وقتی $\sqrt{n}$ دیدی)

قرار بده $n = 2^m$ و $S(m) = T(2^m)$.

- $T(n) = 2T(\sqrt{n}) + \log n$ تبدیل می‌شود به $S(m) = 2S(m/2) + m$، پس $S = m \log m$ و $T = \Theta(\log n \cdot \log\log n)$.
- $T(n) = T(\sqrt{n}) + 1$ تبدیل می‌شود به $S(m) = S(m/2) + 1$، پس $T = \Theta(\log\log n)$.
- **ترفند تقسیم:** برای $T(n) = \sqrt{n}\cdot T(\sqrt{n}) + n$ اول دو طرف را بر n تقسیم کن و $R(n) = T(n)/n$ بگذار.

د) کم‌شونده‌ها

رابطه	جواب
$T(n-1) + 1$	$\Theta(n)$
$T(n-1) + n$	$\Theta(n^2)$
$T(n-1) + \log n$	$\Theta(n \log n)$
$T(n-1) + 1/n$	$\Theta(\log n)$
$2T(n-1) + 1$	$\Theta(2^n)$ (هانوی)
$T(n-1) + T(n-2)$	$\Theta(\varphi^n)$ با $\varphi \approx 1.618$ (فیبوناچی)
$T(n-1) + 2T(n-2) + T(n-3)$	ریشهٔ معادلهٔ $x^3 = x^2 + 2x + 1$ (≈ ۲.۱۵)؛ پس **کران ساده** 3^n است

✏️ مثال (باز کردن)

$T(n) = T(n-1) + n$ با $T(1) = 1$: $T(n) = n + (n-1) + T(n-2) = \ldots = n + (n-1) + \ldots + 1 = n(n+1)/2 = \Theta(n^2)$. به همین ترتیب $T(n-1) + \log n$ می‌شود $\log 1 + \ldots + \log n = \log(n!) = \Theta(n \log n)$، و $2T(n-1) + 1$ با $T(1) = 1$ می‌شود $1, 3, 7, 15, \ldots$ یعنی $2^n - 1$.

تمرین درس ۴ — اول خودت حل کن، بعد پاسخ‌نامه

۱. $T(n) = T(n/2) + T(n/4) + T(n/8) + n$

۲. $T(n) = T(n/4) + T(3n/4) + n$

۳. $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$

۴. $T(n) = 2T(\sqrt{n}) + 1$

۵. $T(n) = T(n-2) + n$

پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۲۶

درس ۵ تحلیل سرشکن (Amortized)

از صفر: قلک پس‌انداز

آرایهٔ پویا (مثل vector یا ArrayList) وقتی پر شود، آرایهٔ دو برابر بزرگ‌تری می‌گیرد و همه را کپی می‌کند. پس **بیشتر** درج‌ها ارزان‌اند (۱ قدم) ولی **گاهی** یک درج خیلی گران است (کپی همه). سؤال: «به‌طور میانگین هر درج چقدر خرج دارد؟»

تشبیه: هر بار که درج ارزان می‌کنی، ۲ سکهٔ اضافه در قلک می‌گذاری. وقتی نوبت کپی گران رسید، هزینه‌اش را از قلک می‌دهی. اگر ثابت کنی قلک **هیچ‌وقت منفی نمی‌شود**، یعنی هر عمل به‌طور میانگین فقط ۳ سکه خرج دارد، یعنی $O(1)$، حتی با اینکه بعضی عمل‌ها تکی گران‌اند.

مثال

۸ درج با ظرفیت اولیهٔ ۱: کپی‌ها در اندازه‌های ۱، ۲ و ۴ ⇐ $1 + 2 + 4 = 7$ کپی. کل کار $8 + 7 = 15$، یعنی کمتر از ۲ واحد برای هر درج.

شهود

میانگین هزینه روی **بدترین دنبالهٔ n عمل**. این تحلیل احتمالاتی نیست و تحلیل حالت متوسط هم نیست؛ تضمین بدترین حالت است.

مثال	تحلیل	هزینهٔ سرشکن
آرایهٔ پویا با **دو برابر** شدن	کپی‌ها $1+2+4+\ldots+n < 2n$	$O(1)$
آرایهٔ پویا با **اضافهٔ ثابت** (مثلاً +۱۰۰)	کپی‌ها $\Theta(n^2)$	$\Theta(n)$
شمارندهٔ دودویی (فقط افزایش)	بیت i هر 2^i بار عوض می‌شود؛ جمع $< 2n$	$O(1)$
پشته با MULTIPOP	هر عنصر حداکثر یک push و یک pop دارد	$O(1)$

✏️ مثال برای هر سطر جدول

- **دو برابر شدن:** ۹ درج با ظرفیت اولیهٔ ۱: کپی در اندازه‌های ۱، ۲، ۴، ۸ ⇐ 15 کپی؛ کل کار $9 + 15 = 24 < 3\times9$.
- **اضافهٔ ۱۰۰:** با ۱۰۰۰ درج، کپی در اندازه‌های ۱۰۰، ۲۰۰، ...، ۹۰۰ ⇐ $100 + 200 + \ldots + 900 = 4500$ کپی. این عدد با $n^2/200$ رشد می‌کند.
- **شمارنده:** ۸ افزایش از ۰: تعداد بیت‌های عوض‌شده $1, 2, 1, 3, 1, 2, 1, 4$ است، جمعاً $15 < 2\times8$.
- **MULTIPOP:** ۵ push، MULTIPOP(3) ، ۲ push، MULTIPOP(10) : کل pop ها حداکثر به تعداد push ها، یعنی ۷ است؛ پس ۹ عمل روی هم حداکثر ۱۴ واحد کار دارند.

💡 سه روش

- **تجمعی:** کل هزینه تقسیم بر n.
- **حسابداری:** عمل‌های ارزان پیش‌پرداخت می‌دهند تا عمل‌های گران پوشش داده شوند؛ اعتبار هیچ‌وقت منفی نمی‌شود.
- **پتانسیل:** هزینهٔ سرشکن = هزینهٔ واقعی + $\Delta\Phi$. مثلاً در شمارنده با Φ = تعداد بیت‌های ۱، هر increment هزینهٔ سرشکن ۲ دارد.

✏️ مثال

- **حسابداری (پشته):** برای هر push دو سکه بگیر: یکی خرج خود push، یکی ذخیره روی عنصر برای pop آینده. MULTIPOP از سکه‌های ذخیره پرداخت می‌شود.
- **پتانسیل (شمارنده):** $0111 \to 1000$: هزینهٔ واقعی ۴ (چهار بیت عوض شد) و Φ از ۳ به ۱ رسید ($\Delta\Phi = -2$)، پس هزینهٔ سرشکن $2 = 2 - 4$.

✍️ تمرین درس ۵

اول خودت حل کن، بعد پاسخ‌نامه

۱. آرایهٔ پویا هر بار که پر شد **سه برابر** می‌شود. هزینهٔ سرشکن هر درج؟

۲. شمارندهٔ k بیتی که هم increment و هم decrement دارد. هزینهٔ سرشکن هر عمل در بدترین دنباله؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۲۶**

📝 آزمون فصل ۱

سؤال‌های واقعی کنکور ارشد کامپیوتر

۱۱ سؤال | ⏱️ زمان: ۲۵ دقیقه برای ۱۱ سؤال | 🔑 کلید و راه‌حل: **صفحهٔ ۲۷**

روی هیچ سؤالی بیشتر از ۴ دقیقه نمان.

✔️ مثل جلسهٔ کنکور: زمان بگیر، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۴ · سؤال ۵۶ — غلط یا حدسی؟ ☐

فرض کنید $T(x) = 2T(x/2) + (8/9)\cdot T(3x/4) + \Theta(x^2/\log x)$. کدام مورد درست است؟

۱) $\Theta(x^2 \log x)$
۲) $\Theta(x^2/\log x)$
۳) $\Theta(x^2 \log(\log x))$
۴) $\Theta(2 \log x - \log(\log x))$

۲ ارشد ۱۴۰۳ · سؤال ۶۱ — غلط یا حدسی؟ ☐

تابع بازگشتی زیر را در نظر بگیرید:

```
int f(int n) {
    if (n == 0 || n == 1) return (n + 1);
    else if (n == 2) return 4;
    return (f(n-1) + 2*f(n-2) + f(n-3));
}
```

برای $n \geq 3$ ، کدام گزینه بهترین کاندید برای $f(n)$ است؟

۱) $f(n) \leq 2^n$
۲) $f(n) \leq 3^n$
۳) $f(n) \leq 2^{n+1}$
۴) $f(n) \leq 4^n$

۳ ارشد ۱۴۰۳ · سؤال ۶۳ — غلط یا حدسی؟ ☐

بهترین پیچیدگی زمانی مورد نیاز برای محاسبهٔ مجموع دو جملهٔ nام و zام از دنبالهٔ فیبوناچی چیست؟

۱) $O(i \cdot j)$
۲) $O(\max\{i, j\})$
۳) $O(2^i + 2^j)$
۴) $O(\max\{2^i, 2^j\})$

۴ ارشد ۱۴۰۳ · سؤال ۶۶ — غلط یا حدسی؟ ☐

فرض کنید $f(n) = n$ و $g(n) = n^{(1+\sin n)}$ که n یک عدد صحیح مثبت است. کدام‌یک از گزاره‌های زیر درست است؟ II $f(n) = O(g(n))$ .I. $f(n) = \Omega(g(n))$

۱) فقط I
۲) فقط II
۳) نه I و نه II
۴) هم I و هم II

۵ ارشد ۱۴۰۲ · سؤال ۵۷ — غلط یا حدسی؟

مرتبهٔ زمانی الگوریتم زیر کدام است؟

```
for (i = 1; i ⩽ n; i++)
    for (j = 1; j ⩽ n / i; j++)   // ⌊n/i⌋
        S.O;
```

۱) $O(n)$ ۲) $O(n \log n)$ ۳) $O(n^2)$ ۴) $\theta(n)$

۶ ارشد ۱۴۰۲ · سؤال ۶۳ — غلط یا حدسی؟

یک الگوریتم یک آرایهٔ دوبعدی $n \times n$ را به‌عنوان ورودی دریافت می‌کند و زمان اجرای آن برحسب n عبارت است از $T(n) = O(n \log n)$. زمان الگوریتم برحسب اندازهٔ ورودی $m = n^2$ کدام است؟

۱) $O(m^2 \log m)$ ۲) $O(m^2 \log^2 m)$ ۳) $O(\sqrt{m} \log m)$ ۴) $O(m \log m)$

۷ ارشد ۱۴۰۲ · سؤال ۶۴ — غلط یا حدسی؟

برای حل یک مسئله باید از بین چهار الگوریتم انتخاب کنید. کدام‌یک ارجحیت دارد؟

۱) الگوریتم A نمونه‌ای به اندازهٔ n را با **بیست** نمونه با اندازهٔ $n/3$ حل می‌کند و سپس راه‌حل‌ها را در زمان $O(n^2)$ ترکیب می‌کند.

۲) الگوریتم B نمونه‌ای به اندازهٔ n را با **هشت** نمونه با اندازهٔ $n/2$ حل می‌کند و سپس راه‌حل‌ها را در زمان $O(n^3)$ ترکیب می‌کند.

۳) الگوریتم C نمونه‌ای به اندازهٔ n را با حل بازگشتی دو نمونه با اندازهٔ $2 \times n$ حل می‌کند و سپس راه‌حل‌ها را در زمان $O(n)$ ترکیب می‌کند.

۴) الگوریتم D نمونه‌ای به اندازهٔ n را با حل بازگشتی دو نمونه با اندازهٔ $n-1$ حل می‌کند و سپس راه‌حل‌ها را در زمان ثابت ترکیب می‌کند.

۸ ارشد ۱۴۰۲ · سؤال ۶۷ — غلط یا حدسی؟

کدام‌یک از روابط بازگشتی زیر را **نمی‌توان** مستقیماً با قضیهٔ اصلی حل کرد؟

۱) $T(n) = 16T(n/4) + n$ ۲) $T(n) = T(n/5) + 20$

۳) $T(n) = 3T(n/4) + n \log n$ ۴) $T(n) = 2T(n/2) + n \log n$

۹ ارشد ۱۴۰۱ · سؤال ۶۶ — غلط یا حدسی؟

در کدام مورد، توابع به‌ترتیب صعودی (و نه اکیداً صعودی) بر مبنای رشد تابع از سمت چپ به راست مرتب شده‌اند؟

۱) $\log n, \log^2 n, \log n^2, n \log^2 n, \log n!, \log n^n$

۲) $\log n, \log n^2, \log^2 n, n \log^2 n, \log n^n, \log n!$

۳) $\log n, \log^2 n, \log n^2, \log n!, \log n^n, n \log^2 n$

۴) $\log n, \log n^2, \log^2 n, \log n^n, \log n!, n \log^2 n$

۱۰ ارشد ۱۳۹۹ · سؤال ۸۵ — غلط یا حدسی؟

به ازای چند زوج (a, b) از اعداد طبیعی **کوچک‌تر از ۵**، جواب رابطهٔ بازگشتی $T(n) = aT(n/b) + n^2$ برابر $\theta(n^2)$ می‌شود؟

۱) ۷ ۲) ۸ ۳) ۱۱ ۴) ۱۲

۱۱ ارشد ۱۳۹۸ · سؤال ۵۱ — غلط یا حدسی؟

پاسخ رابطهٔ بازگشتی $T(n) = T(n / \log n) + O(1)$ کدام است؟

۱) $O(\log n)$ ۲) $O(\log \log n)$

۳) $O(\log n / \log \log n)$ ۴) $O(\log n / (\log \log n)^2)$

کارت خلاصهٔ فصل ۱

برای مرور سریع، شب قبل از آزمون

- **نردبان:** $1 < \log \log n < \log n < n^{\varepsilon} < n < n \log n < n^k < 2^n < n! < n^n$
- $\log(n!) = \Theta(n \log n) = \Theta(\log n^n)$ ؛ $\log n^2 = \Theta(\log n)$ ولی $\log^2 n$ بزرگ‌تر است. توان نوسانی یعنی مقایسه‌ناپذیر.
- **حلقه:** حلقهٔ وابسته را جمع بزن. $i*=2$ یعنی $\log n$ ؛ $i=i*i$ یعنی $\log \log n$ ؛ $j+=i$ یعنی هارمونیک.
- **قضیهٔ اصلی:** $c = \log_b a$. سه حالت کلاسیک «چندجمله‌ای» فاصله می‌خواهند. فاصلهٔ $\log$ در نسخهٔ کلاسیک «مستقیم حل نمی‌شود»؛ جوابش را با حالت گسترده بگیر ($k>-1$: $\log^{k+1}$ ؛ $k=-1$: $\log \log$ ؛ $k<-1$: n^c).
- **تقسیم نامساوی:** اگر $\Sigma \alpha<1$ جواب n است؛ اگر $=1$ جواب $n \log n$ ؛ در غیر این صورت آکرا-بازی ($\Sigma a_i b_i^p = 1$).
- **$\sqrt{n}$:** قرار بده $n = 2^m$. **$T(n/\log n)$:** جواب $\log n / \log \log n$.
- **سرشکن:** دو برابر کردن $O(1)$ ، اضافهٔ ثابت $\Theta(n)$ ، شمارندهٔ دوطرفه $\Theta(k)$.

پاسخ‌نامهٔ فصل ۱

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۱

برگشت ← صفحهٔ ۹

۱. $\Theta(n \log n)$

۲. درست؛ نادرست ($2^{2n} = 4^n$).

۳. $(\log n)^{10} < 2^{(\log_2 n)} = n < n \log n < n^{1.5}$

۴. $= 2^{(\log_2 n / \log_2 n)} = 2$ ، یعنی $\Theta(1)$.

تمرین درس ۲

برگشت ← صفحهٔ ۱۶

۱. $n + n/2 + n/4 + \ldots < 2n$ ، پس $\Theta(n)$. (ضرب کردن جواب غلط $n \log n$ را می‌دهد.)

۲. $\Sigma_i \Sigma_j j = \Sigma\, i^2/2 \approx n^3/6$ ، پس $\Theta(n^3)$.

۳. حلقهٔ درونی به i وابسته **نیست**، پس اینجا ضرب درست است: $\Theta(n \log n)$.

تمرین درس ۳

برگشت ← صفحهٔ ۱۸

۱. $c=2$ ، حالت ۲: $\Theta(n^2 \log n)$

۲. $c \approx 0.79$ ، حالت ۳ و شرط نظم برقرار است: $\Theta(n \log n)$

۳. $c = \log_2 7 \approx 2.81$ ، حالت ۱: $\Theta(n^{2.81})$ (استراسن)

۴. $c = 1/2$ ، حالت ۲: $\Theta(\sqrt{n} \log n)$

۵. گسترده با $k=-1$: $\Theta(n \log \log n)$ (در نسخهٔ کلاسیک مستقیم حل نمی‌شود)

۶. گسترده با $k=1$: $\Theta(n^2 \log^2 n)$ (در نسخهٔ کلاسیک مستقیم حل نمی‌شود)

تمرین درس ۴

برگشت ← صفحهٔ ۲۱

۱. $1/2 + 1/4 + 1/8 = 7/8 < 1$ ، پس $\Theta(n)$.

۲. $1/4 + 3/4 = 1$ ، پس $\Theta(n \log n)$.

۳. $R(n) = R(\sqrt{n}) + 1$ ، پس $R = \log \log n$ و $T = \Theta(n \log \log n)$.

۴. $S(m) = 2S(m/2) + 1$ ، پس $S = \Theta(m)$ و $T = \Theta(\log n)$.

۵. $n + (n-2) + (n-4) + \ldots \approx n^2/4$ ، پس $\Theta(n^2)$.

تمرین درس ۵

برگشت ← صفحهٔ ۲۲

۱. $O(1)$ (هر ضریب ثابت بزرگ‌تر از ۱ کافی است).

۲. $\Theta(k)$. دنبالهٔ متناوب increment و decrement روی مرز $0111\ldots1 \leftrightarrow 1000\ldots0$ هر بار k بیت را عوض می‌کند. تحلیل $O(1)$ فقط برای افزایش تنها درست است.

آزمون فصل ۱ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۲۳

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰	سؤال ۱۱
۳	۲	۲	۳	۲	۳	۱	۴	۴	۳	۳

۱ **گزینهٔ ۳**

آکرا-بازی. p=۲ را امتحان کن: $2\cdot(1/4) + (8/9)\cdot(9/16) = 1/2 + 1/2 = 1$. انتگرال $\int (u^2/\log u)/u^3\,du = \int du/(u \log u)$ است، پس جواب $u) = \log\log x$. $\Theta(x^2 \log\log x)$ (این همان حالت گسترده با k=-۱ است.)

۲ **گزینهٔ ۲**

با استقرا: $f(n) \le 3^{n-1} + 2\cdot3^{n-2} + 3^{n-3} = 16\cdot3^{n-3} \le 27\cdot3^{n-3} = 3^n$. پایه‌ها هم درست‌اند ($1\le1$ ، $2\le3$ ، $4\le9$).

- گزینهٔ ۱ غلط است: $f(3) = 9 > 8$.
- گزینهٔ ۳ هم غلط است: $f(12) = 8641 > 2^{13} = 8192$ ، چون f تقریباً مثل 2.2^n رشد می‌کند.
- گزینهٔ ۴ درست است ولی از 3^n شل‌تر است، پس «بهترین» نیست.

۳ **گزینهٔ ۲**

با یک حلقه تا $\max(i, j)$ هر دو جمله در مسیر به دست می‌آیند. (با توان ماتریس حتی $O(\log)$ هم ممکن است، ولی بین گزینه‌ها بهترین همین است.)

۴ **گزینهٔ ۳**

$\sin n$ برای n صحیح هم به نزدیکی ۱ می‌رسد و هم به نزدیکی -۱، پس g گاهی تقریباً n^2 و گاهی تقریباً n^0 است. f نه همیشه زیر g می‌ماند نه همیشه بالای آن. پس مقایسه‌ناپذیرند.

۵ **گزینهٔ ۲**

$\sum \lfloor n/i \rfloor = \Theta(n \log n)$ (سری هارمونیک).

۶ **گزینهٔ ۳**

$n = \sqrt{m}$ ، پس $n \log n = \sqrt{m} \cdot \log \sqrt{m} = \Theta(\sqrt{m} \log m)$.

۷ **گزینهٔ ۱**

- A: $T = 20T(n/3) + n^2$. چون $\log_3 20 \approx 2.73 > 2$ ، حالت ۱ است و جواب $\Theta(n^{2.73})$.
- B: $8T(n/2) + n^3$. حالت ۲ و جواب $\Theta(n^3 \log n)$.
- C: اندازهٔ زیرمسئله **بزرگ‌تر** می‌شود و الگوریتم هیچ‌وقت تمام نمی‌شود.
- D: $2T(n-1) + 1$ و جواب $\Theta(2^n)$.

A از همه سریع‌تر است.

---

۸ **گزینهٔ ۴**

$c = 1$ و $f = n \log n$ فقط یک ضریب log بزرگ‌تر از n است، یعنی شکاف بین حالت ۲ و ۳ نسخهٔ کلاسیک.

- گزینهٔ ۱: حالت ۱ (n^2).
- گزینهٔ ۲: حالت ۲ ($\log n$).
- گزینهٔ ۳: $c \approx 0.79$ و $n \log n$ چندجمله‌ای بزرگ‌تر است، پس حالت ۳.

---

۹ **گزینهٔ ۴**

$\log n = \Theta(\log n^2) < \log^2 n < \log n^n = \Theta(\log n!) = \Theta(n \log n) < n \log^2 n$. چون مرتب‌سازی «نه اکیداً» است، ترتیب دو تابع Θ برابر مهم نیست. گزینهٔ ۳ غلط است چون $\log^2 n$ را قبل از $\log n^2$ گذاشته.

---

۱۰ **گزینهٔ ۳**

$a, b \in \{1,2,3,4\}$ و $b = 1$ معنی ندارد (مسئله کوچک نمی‌شود). برای اینکه جواب n^2 باشد باید $\log_b a < 2$ ، یعنی $a < b^2$:

- $b = 2$: $a < 4$ ، یعنی $a = ۱، ۲، ۳$ (**۳ زوج**؛ $a = 4$ جواب $n^2 \log n$ می‌دهد).
- $b = 3$: $a < 9$ ، یعنی $a = ۱$ تا ۴ (**۴ زوج**).
- $b = 4$: $a < 16$ ، یعنی $a = ۱$ تا ۴ (**۴ زوج**).

جمع: **۱۱**.

---

۱۱ **گزینهٔ ۳**

در هر مرحله log مقدار به اندازهٔ حدود $\log \log n$ کم می‌شود ($\log(n/\log n) = \log n - \log \log n$). از $\log n$ شروع می‌کنیم، پس تعداد مراحل حدود $\log n / \log \log n$ است.

فصل ۲

آرایه، لیست پیوندی، پشته و صف

۲

۳ درس | ۳ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۳ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴. سؤال‌ها مفهومی‌اند: «کدام روش کار نمی‌کند؟» یا «کدام ساختار ساختنی نیست؟». این فصل پایهٔ هیپ، گراف (BFS و DFS) و تحلیل سرشکن هم هست.

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ آرایه و لیست پیوندی | ۲ پشته (LIFO) | ۳ صف (FIFO) و صف دوطرفه | 📝 آزمون فصل

درس ۱ آرایه و لیست پیوندی

از صفر: داده‌ساختار یعنی «طرز چیدن»

داده‌ساختار یعنی **طرز چیدن داده‌ها در حافظه** تا کارهای خاصی سریع شوند. هیچ ساختاری در همه‌چیز بهترین نیست؛ هر کدام چیزی را سریع می‌کند و چیز دیگری را کند. تست‌های این فصل تقریباً همیشه همین را می‌پرسند: «کدام عمل در کدام ساختار چقدر طول می‌کشد؟» و «فلان روش روی فلان ساختار اجراشدنی است؟». این فصل دو چیدمان پایه را مقایسه می‌کند.

- **آرایه = کمدهای شماره‌دار کنار هم.** چون شماره داری، مستقیم سراغ کمد ۳۷ می‌روی ($O(1)$). ولی برای اضافه کردن یک کمد در **وسط**، همهٔ کمدهای بعدی باید یکی جابه‌جا شوند ($O(n)$).
- **لیست پیوندی = گنج‌یابی.** هر برگه (گره) یک مقدار دارد و **آدرس برگهٔ بعدی** را. برای رسیدن به برگهٔ ۳۷ باید از اول ۳۶ برگه را دنبال کنی ($O(n)$). ولی اضافه کردن یک برگه در وسط فقط عوض کردن دو آدرس است ($O(1)$)، به شرطی که آن نقطه را در دست داشته باشی.
  - **یک‌طرفه:** هر گره فقط آدرس **بعدی** را دارد ($next$).
  - **دوطرفه:** هر گره آدرس **قبلی و بعدی** را دارد ($prev$ و $next$).

✏️ مثال (درج ۱۵ بین ۱۰ و ۲۰)

- **آرایه** [10, 20, 30, 40] : ۴۰، ۳۰ و ۲۰ هر کدام یک خانه عقب می‌روند (از آخر شروع کن تا چیزی پاک نشود)، بعد ۱۵ در خانهٔ خالی می‌نشیند ⇐ [10, 15, 20, 30, 40] . سه جابه‌جایی؛ در آرایهٔ n تایی تا n جابه‌جایی.
- **لیست** $10 \to 20 \to 30 \to 40$ ، با p = گرهٔ ۱۰: گرهٔ تازهٔ n با مقدار ۱۵ بساز، بعد:
  1. `n->next = p->next;` (۱۵ به ۲۰ اشاره کند)
  2. `p->next = n;` (۱۰ به ۱۵ اشاره کند) ⇐ $10 \to 15 \to 20 \to 30 \to 40$. دو تغییر، هر قدر هم لیست بلند باشد.

⚠️ تلهٔ تستی

ترتیب این دو خط مهم است. اگر اول `p->next = n` را بنویسی، آدرس ۲۰ (و کل بقیهٔ لیست) گم می‌شود.

✏️ مثال (حذف گرهٔ بعد از p)

در $10 \to 15 \to 20$ با p = ۱۰: `p->next = p->next->next;` ⇐ $10 \to 20$. گرهٔ ۱۵ از زنجیر بیرون افتاد.

عمل	آرایه	لیست یک‌طرفه	لیست دوطرفه
دسترسی به عضو iام	O(1)	O(n)	O(n)
درج یا حذف در ابتدا	O(n)	O(1)	O(1)
درج **بعد از** گره‌ای که اشاره‌گرش را داریم	O(n)	O(1)	O(1)
حذف گره‌ای که اشاره‌گرش را داریم	O(n)	O(n) (به گره قبلی نیاز داریم)	O(1)
جستجو	O(n) (اگر مرتب باشد O(log n))	O(n)	O(n)

✏️ مثال برای هر سطر جدول

آرایهٔ [10, 20, 30, 40] و لیست 10 → 20 → 30 → 40 .

- **دسترسی:** در آرایه A[2] مستقیم از روی آدرس شروع + 2 × اندازهٔ خانه خوانده می‌شود. در لیست باید از سر راه بیفتی: 10 → 20 → 30 .
- **درج در ابتدا (۵):** در آرایه هر ۴ عنصر یک خانه جابه‌جا می‌شوند. در لیست یک گرهٔ تازه می‌سازی که به ۱۰ اشاره کند و head را عوض می‌کنی.
- **حذف گرهٔ p = ۳۰:** برای حذف، گرهٔ **قبلی** (۲۰) باید به ۴۰ اشاره کند. در لیست یک‌طرفه، از ۳۰ نمی‌شود به ۲۰ رسید؛ باید از اول بگردی (O(n)). در لیست دوطرفه، ۲۰ همان p->prev است: p->prev->next = p->next و p->next->prev = p->prev (O(1)).
- **جستجو:** در آرایهٔ مرتب با جستجوی دودویی، ۳۰ با ۲ مقایسه پیدا می‌شود؛ در لیست حتی اگر مرتب باشد، نمی‌شود به وسط پرید.

⚠️ تلهٔ تستی

در لیست **یک‌طرفه** نمی‌شود به عقب رفت. هر روشی که «از انتها به سمت ابتدا حرکت کن» می‌گوید، در لیست یک‌طرفه **اجراشدنی نیست**.

ترفند دو اشاره‌گر (کند و تند)

در لیست یک‌طرفه طول را نمی‌دانیم و برگشت هم نداریم. ترفند این است که **دو اشاره‌گر با سرعت‌های متفاوت** هم‌زمان حرکت کنند.

✏️ مثال (وسط لیست در یک پیمایش)

کند یک گام و تند دو گام جلو می‌رود؛ وقتی تند به انتها رسید، کند وسط است (چون نصف راه را رفته). لیست 1 → 2 → 3 → 4 → 5 :

گام	کند	تند
شروع	۱	۱
۱	۲	۳
۲	۳	۵ (آخر)

تند به آخر رسید ⇐ کند روی **۳** است، یعنی وسط.

✏️ مثال (kامین گره از آخر، k = ۲)

اشاره‌گر دوم را k گام جلوتر شروع کن و هر دو را با هم جلو ببر تا دومی از انتها بیرون برود (NULL شود). فاصلهٔ آن دو همیشه k است، پس وقتی دومی «یکی بعد از آخر» است، اولی k تا قبل از آن است.

گام	اولی	دومی
شروع	۱	۳
۱	۲	۴
۲	۳	۵
۳	**۴**	NULL

جواب ۴، یعنی دومین گره از آخر.

✏️ مثال (تشخیص حلقه، فلوید)

اگر ۵ به‌جای NULL به ۳ اشاره کند، لیست حلقه دارد و تند هیچ‌وقت به NULL نمی‌رسد. ولی تند هر گام یکی به کند نزدیک‌تر می‌شود (مثل دو دونده در پیست دایره‌ای) و بالاخره به هم می‌رسند:

گام	کند	تند
شروع	۱	۱
۱	۲	۳
۲	۳	۵
۳	۴	۴ (۵ ⇐ ۳ ⇐ ۴)

بعد از ۳ گام هر دو روی ۴ هستند ⇐ **حلقه دارد**. اگر حلقه نبود، تند به NULL می‌رسید.

✏️ مثال (وارون کردن لیست)

با سه اشاره‌گر: prev (قبلی)، cur (فعلی) و nx (بعدی، تا گم نشود). در هر گام: nx = cur->next; cur->next = prev; prev = cur; cur = nx;. روی $1 \to 2 \to 3$:

گام	prev	cur	تغییر	لیست تا اینجا
شروع	NULL	۱		
۱	۱	۲	۱ به NULL اشاره کند	1
۲	۲	۳	۲ به ۱ اشاره کند	$2 \to 1$
۳	۳	NULL	۳ به ۲ اشاره کند	$3 \to 2 \to 1$

head تازه همان prev است (۳). زمان $O(n)$ و حافظهٔ اضافه فقط سه اشاره‌گر، یعنی $O(1)$.

تمرین درس ۱ — اول خودت حل کن، بعد پاسخ‌نامه

۱. در لیست $1 \to 7 \to 4 \to 9 \to 2 \to 8 \to 5$ با روش کند و تند، وقتی تند به آخرین گره می‌رسد، کند روی کدام گره است؟

۲. در همان لیست، سومین گره از آخر کدام است؟ جدول دو اشاره‌گر را بکش.

۳. در کدام ساختار، حذف گره‌ای که اشاره‌گرش را داریم $O(1)$ است؟ چرا در بقیه نه؟

پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۴۳

درس ۲ پشته (LIFO)

از صفر: پشته = ستون بشقاب‌ها

بشقاب تازه را **روی** ستون می‌گذاری (push) و بشقاب را هم فقط از **رو** برمی‌داری (pop). پس آخرین بشقابی که گذاشتی، اولین بشقابی است که برمی‌داری: **LIFO** (آخر وارد، اول خارج). به بشقاب‌های پایین دسترسی نداری مگر اینکه بالایی‌ها را برداری.

مثال (ردگیری)

push ۱، push ۲، push ۳، pop، push ۴، pop، pop:

عمل	پشته (بالا سمت راست)	خروجی
push ۱	[1]	
push ۲	[1, 2]	
push ۳	[1, 2, 3]	
pop	[1, 2]	۳
push ۴	[1, 2, 4]	
pop	[1, 2]	۴
pop	[1]	۲

خروجی‌ها به ترتیب: 3, 4, 2.

کجا استفاده می‌شود؟ دکمهٔ «برگشت» مرورگر، دکمهٔ Undo، و فراخوانی تابع‌ها (پشتهٔ فراخوانی در فصل بازگشت برنامه‌سازی): هر جا «آخرین کار» باید اول برگردانده شود.

- push و pop در $O(1)$، چه با آرایه چه با لیست.

- **پشته با min یا max در $O(1)$:** یک پشتهٔ کمکی که در هر سطح، کمینه یا بیشینهٔ فعلی را نگه می‌دارد.
- کاربردها: تطابق پرانتزها، فراخوانی تابع و بازگشت، DFS، تبدیل و ارزیابی عبارت.

✏️ مثال (پشتهٔ min)

هر بار که عددی push می‌شود، در پشتهٔ کمکی min(عدد تازه، بالای کمکی) را push کن. push ها 5, 3, 7 :

عمل	پشتهٔ اصلی	پشتهٔ کمکی	کمینهٔ فعلی
push ۵	[5]	[5]	۵
push ۳	[5, 3]	[5, 3]	۳
push ۷	[5, 3, 7]	[5, 3, 3]	۳
pop	[5, 3]	[5, 3]	۳
pop	[5]	[5]	۵

کمینه همیشه بالای پشتهٔ کمکی است و با pop خودبه‌خود به کمینهٔ قبلی برمی‌گردد.

✏️ مثال (تطابق پرانتزها)

با دیدن هر پرانتز **باز** push کن؛ با دیدن هر **بسته**، بالای پشته باید جفت همان باشد، pop کن. آخر کار پشته باید خالی باشد.

- ([ ]) : push (، push [، با] بالای پشته [است ✓ pop، با) بالای پشته (است ✓ pop، پشته خالی ⇐ **معتبر**.
- [ ) : push (، با] بالای پشته (است ✗ ⇐ **نامعتبر**.
-) : آخر کار پشته خالی نیست ⇐ **نامعتبر**.

جایگشت‌های پشته‌ای

ورودی $1, 2, \ldots, n$ به‌ترتیب وارد پشته می‌شود و هر لحظه می‌توان push یا pop کرد. هر pop یک عدد به خروجی می‌فرستد. کدام ترتیب‌ها در خروجی ممکن‌اند؟

✏️ مثال (ساختن ۲۳۱)

عمل	پشته	خروجی تا اینجا
push ۱	[1]	
push ۲	[1, 2]	
pop	[1]	۲
push ۳	[1, 3]	۲
pop	[1]	۳، ۲
pop	[]	۱ ،۳ ،۲ ✓

✏️ مثال (چرا ۳۱۲ ناممکن است)

برای اینکه ۳ اول بیرون بیاید، باید ۱ و ۲ قبلاً push شده باشند و هنوز در پشته باشند: [1, 2, 3] . بعد از pop کردن ۳، بالای پشته **۲** است، نه ۱. پس بعد از ۳ حتماً ۲ می‌آید و 312 ساخته نمی‌شود.

- **الگوی ممنوع:** خروجی نمی‌تواند سه عدد $i < j < k$ را به ترتیب k, i, j داشته باشد (مثل همین 3, 1, 2).
- تعداد خروجی‌های ممکن **عدد کاتالان** است: $C_n = C(2n,n)/(n+1)$. برای n = ۱ تا ۵: ۱، ۲، ۵، ۱۴، ۴۲.

✏️ مثال (n = ۳)

پنج خروجی ممکن: 123 ، 132 ، 213 ، 231 ، 321 . ششمی، یعنی 312 ، همان الگوی ممنوع است. $C_3 = C(6,3)/4 = 20/4 = 5$ ✓.

چرا نمایش پسوندی؟

ما می‌نویسیم a + b (عملگر **وسط**، میانوندی). در نمایش **پسوندی** عملگر **بعد** از دو عملوندش می‌آید: a b + . فایده‌اش این است که **هیچ پرانتز و قاعدهٔ اولویتی لازم ندارد** و ماشین‌حساب با یک پشته آن را حساب می‌کند: (2 + 3) × 4 ⇐ 2 3 + 4 × و 2 + 3 × 4 ⇐ 2 3 4 × + . ترتیب عملوندها عوض نمی‌شود؛ فقط عملگرها جابه‌جا می‌شوند.

ارزیابی پسوندی

💡 روش

از چپ به راست بخوان. عدد را push کن. با دیدن عملگر، **دو** عدد بالای پشته را pop کن (اولی که بیرون می‌آید عملوند **دوم** است)، عمل را انجام بده و نتیجه را push کن. آخر کار، تنها عدد پشته جواب است.

✏️ مثال

$6\ 2\ 3\ +\ -\ 4\ *$

نماد	کار	پشته
۶	push	[6]
۲	push	[6, 2]
۳	push	[6, 2, 3]
+	$2 + 3 = 5$	[6, 5]
-	$6 - 5 = 1$	[1]
۴	push	[1, 4]
*	$1 \times 4 = 4$	[4]

جواب **۴**. ⚠️ در `-` و `/` ترتیب مهم است: عدد زیری منهای عدد بالایی ($5 - 6$، نه $6 - 5$).

تبدیل میانوندی به پسوندی

💡 روش

۱. **عملوند** (حرف یا عدد) را مستقیم در خروجی بنویس.

۲. **عملگر** را در پشته بگذار؛ ولی قبل از آن، عملگرهایی از بالای پشته را که اولویتشان **بیشتر یا مساوی** است pop کن و در خروجی بنویس.

۳. `(` را push کن. با `)`، تا رسیدن به `(` pop کن و بنویس؛ خود پرانتزها نوشته نمی‌شوند.

۴. آخر کار، همهٔ پشته را pop کن.

✏️ مثال (ساده)

- $a + b * c$: a ⇐ خروجی؛ + ⇐ پشته؛ b ⇐ خروجی؛ * اولویتش از + بیشتر است ⇐ روی آن push؛ c ⇐ خروجی؛ پایان ⇐ * و بعد + ⇐ $a\ b\ c\ *\ +$.
- $a - b + c$: وقتی + می‌رسد، − با اولویت **مساوی** بالای پشته است، پس اول pop می‌شود ⇐ $a\ b\ -\ c\ +$ (یعنی $(a - b) + c$ ، از چپ به راست).

✏️ مثال (کامل، با جدول)

a + b * (c – d) / e

نماد	خروجی	پشته (بالا سمت راست)	توضیح
a	a		
+	a	+	
b	a b	+	
*	a b	+ *	* قوی‌تر از +
)	a b	+ * (	
c	a b c	+ * (	
-	a b c	+ * (–	داخل پرانتز چیزی pop نمی‌شود
d	a b c d	+ * (–	
)	a b c d –	+ *	تا (pop
/	a b c d – *	+ /	* هم‌اولویت است ⇐ pop
e	a b c d – * e	+ /	
پایان	a b c d – * e / +		خالی کردن پشته

✍️ تمرین درس ۲

اول خودت حل کن، بعد پاسخ‌نامه

۱. عبارت پسوندی 4 2 – 3 * را با جدول پشته ارزیابی کن.

۲. آیا خروجی 1 3 2 با پشته (ورودی ۱، ۲، ۳) ممکن است؟ اگر بله، دنبالهٔ push و pop را بنویس.

۳. a * b + c را به پسوندی تبدیل کن.

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۴۳

درس ۳ صف (FIFO) و صف دوطرفه

از صفر: صف = صف نانوایی

نفر تازه به **ته** صف می‌رود (enqueue) و نفر **سر** صف نان می‌گیرد و می‌رود (dequeue). پس اولین نفری که آمده، اولین نفری است که می‌رود: **FIFO** (اول وارد، اول خارج).

✏️ مثال (صف در برابر پشته)

دنبالهٔ «درج ۱، درج ۲، حذف، درج ۳، حذف»:

عمل	صف (سر سمت چپ)	خروجی صف	پشته	خروجی پشته
درج ۱	[1]		[1]	
درج ۲	[1, 2]		[1, 2]	
حذف	[2]	۱	[1]	۲
درج ۳	[2, 3]		[1, 3]	
حذف	[3]	۲	[1]	۳

صف `1, 2` بیرون داد (به ترتیب آمدن)، پشته `2, 3` (آخری‌ها اول).

کجا استفاده می‌شود؟ صف چاپگر، صف درخواست‌ها، و الگوریتم BFS در گراف (فصل ۸).

صف حلقوی با آرایه

در آرایه، با هر dequeue جای اول خالی می‌شود. به‌جای اینکه همه را یکی جلو بکشیم (`O(n)`)، دو اندیس `front` (سر) و `rear` (ته) داریم که فقط جلو می‌روند و وقتی به آخر آرایه رسیدند، به اول **دور می‌زنند**؛ مثل عقربهٔ ساعت که بعد از ۱۲ به ۱ برمی‌گردد. این یعنی حساب به پیمانهٔ اندازهٔ آرایه: `rear = (rear + 1) mod m` .

مشکل: «پر» و «خالی» هر دو ممکن است `front == rear` بدهند. راه حل رایج: **یک خانه را همیشه خالی نگه دار**؛ پس «پر» یعنی `(rear + 1) mod m == front` .

✏️ مثال (آرایهٔ ۵ خانه، ظرفیت ۴)

rear خانه‌ای است که درج بعدی در آن می‌نشیند و front خانهٔ عضو اول.

عمل	front	rear	محتوا (خانه ۰ تا ۴)	وضعیت
شروع	۰	۰	_ _ _ _ _	خالی
درج A، B، C، D	۰	۴	A B C D _	پر ⇐ (4+1) mod 5 = 0 = front
حذف (A)، حذف (B)	۲	۴	_ _ C D _	
درج E	۲	۰	_ _ C D E	rear دور زد
درج F	۲	۱	F _ C D E	دوباره پر ⇐ (1+1) mod 5 = 2 = front

صف با دو پشته

روش

همهٔ درج‌ها روی پشتهٔ S۱. برای حذف: اگر S۲ خالی است، **همهٔ** S۱ را یکی‌یکی pop کن و در S۲ push کن (ترتیب برعکس می‌شود و قدیمی‌ترین بالای S۲ می‌آید)؛ بعد از S۲ pop کن.

مثال

عمل	S۱	S۲	خروجی
درج ۱، ۲، ۳	[1, 2, 3]	[]	
حذف (S۲ خالی ⇐ انتقال)	[]	[3, 2, 1]	۱
درج ۴	[4]	[3, 2]	
حذف (S۲ خالی نیست)	[4]	[3]	۲

ترتیب خروجی $1, 2$ درست است (FIFO). یک حذف گران بود (۳ انتقال)، ولی هر عضو **حداکثر یک بار** از S۱ به S۲ منتقل می‌شود؛ پس روی هم هزینهٔ **سرشکن** هر عمل $O(1)$ است (درس ۵ فصل ۱).

صف دوطرفه (deque)

درج و حذف از **هر دو سر** در $O(1)$؛ یعنی هم پشته است و هم صف.

مثال

push_front(1) ، push_back(2) ، push_front(0) ⇐ [0, 1, 2] ؛ pop_back عدد ۲ و pop_front عدد ۰ را برمی‌گرداند.

یک کران پایین مهم

هیچ ساختاری نمی‌تواند هم **درج** و هم **حذف کمینه (DeleteMin)** را در $O(1)$ انجام دهد (در مدل مقایسه‌ای). وگرنه با n درج و n بار DeleteMin در $O(n)$ مرتب می‌کردیم، که با کران $\Omega(n \log n)$ مرتب‌سازی تناقض دارد. ولی **FindMin** (فقط نگاه کردن، بدون حذف) در $O(1)$ با پشتهٔ کمکی ممکن است.

مثال

اگر چنین ساختاری بود، [3, 1, 2] را درج می‌کردیم (۳ عمل) و سه بار DeleteMin می‌زدیم: 1, 2, 3 . یعنی هر آرایه با $2n$ عمل $O(1)$ در $O(n)$ مرتب می‌شد؛ که ناممکن است.

✍️ تمرین فصل ۲

اول خودت حل کن، بعد پاسخ‌نامه

۱. `(a + b) * c – d / e` را به پسوندی تبدیل کن.

۲. عبارت پسوندی `5 3 2 * + 4 –` را ارزیابی کن.

۳. کدام جایگشت از ۱، ۲ و ۳ با یک پشته تولید نمی‌شود؟ با ورودی ۱ تا ۴ چند جایگشت پشته‌ای داریم؟

۴. وسط یک لیست یک‌طرفه را با یک پیمایش و بدون دانستن طول چطور پیدا می‌کنی؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۴۳

📝 آزمون فصل ۲

سؤال‌های واقعی کنکور ارشد کامپیوتر

۳ سؤال · ⏱️ زمان: ۸ دقیقه · 🔑 کلید و راه‌حل: **صفحهٔ ۴۳**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۴ · سؤال ۶۷ — غلط یا حدسی؟ ☐

فرض کنید یک لیست پیوندی یک‌طرفه از n گره داریم و می‌خواهید گره‌ای را که در موقعیت $3n/4$ قرار گرفته پیدا کنید، اما مقدار n را نمی‌دانیم. فرض کنید n مضربی از ۴ است. کدام‌یک از گزینه‌ها ما را به نتیجه **نمی‌رساند**؟

۱) ابتدا طول لیست را به‌طور کامل محاسبه کنید، سپس به اندازهٔ $3n/4$ از ابتدا به جلو حرکت کنید تا گرهٔ موردنظر را پیدا کنید.

۲) با دو اشاره‌گر که یکی از ابتدا با سرعت سه گره و یکی از انتها (پس از پیدا کردن گرهٔ انتهایی) با سرعت یک گره حرکت کنند و در لحظهٔ رسیدن به هم، نتیجه حاصل خواهد شد.

۳) ابتدا طول لیست را به‌طور کامل محاسبه کنید، از دو اشاره‌گر استفاده کنید. یکی را در ابتدای لیست و دیگری را در گرهٔ $n/4$ تنظیم کنید. با سرعت یک گره حرکت کنید تا گرهٔ جلوتر به انتها برسد تا به هدف برسیم.

۴) با دو اشاره‌گر که یکی با سرعت یک گره و دیگری به سرعت دو گره حرکت می‌کند، به نیمه می‌رسیم. یک گره جلو برویم و اشاره‌گر دوم که به انتها رسیده را مجدد برابر اشاره‌گر اول قرار خواهیم داد تا مجدد عمل انجام شود و نتیجه حاصل شود.

۲ ارشد ۱۴۰۰ · سؤال ۶۳ — غلط یا حدسی؟ ☐

چند مورد از داده‌ساختارهای زیر را **نمی‌شود** ساخت که اعمال گفته‌شده را در زمان خواسته‌شده انجام دهد؟

- عمل‌های FindMin، Pop، Push و FindMax را در $\Theta(1)$ انجام دهد.
- عمل‌های Pop، Push و DeleteMin را در $\Theta(1)$ انجام دهد.
- عمل‌های Push و Pop را در $\Theta(1)$ انجام دهد.

۱) صفر ۲) یک ۳) دو ۴) سه

۳ ارشد ۱۳۹۸ · سؤال ۵۴

غلط یا حدسی؟

فرض کنید k پشتهٔ $S_1, \ldots, S_k$ را در اختیار داریم. تنها اعمال مجاز: گرفتن یک عنصر از ورودی و push کردن آن داخل پشتهٔ S_1؛ pop کردن یک عنصر از پشتهٔ S_k و قرار دادن در خروجی؛ و pop کردن یک عنصر از پشتهٔ S_i و push کردن آن داخل S_{i+1} (برای $i < k$). فرض کنید اعداد $1, \ldots, n$ به‌ترتیب از کوچک به بزرگ به‌عنوان ورودی داده می‌شود. کوچک‌ترین k که می‌توان همهٔ جایگشت‌های $1, \ldots, n$ را با اعمال مجاز گفته‌شده تولید کرد، در بین گزینه‌ها کدام است؟

۱) ۱ ۲) ۲ ۳) n ۴) $n - 1$

کارت خلاصهٔ فصل ۲

برای مرور سریع، شب قبل از آزمون

- ✔ لیست یک‌طرفه: حرکت به عقب ممنوع؛ حذف گره به گره قبلی نیاز دارد. دوطرفه حذف $O(1)$ دارد.
- ✔ دو اشاره‌گر: وسط لیست، kام از آخر، تشخیص حلقه.
- ✔ جایگشت‌های پشته‌ای عدد کاتالان‌اند ($1, 2, 5, 14, 42$) و الگوی k, i, j ممنوع است.
- ✔ پسوندی: عملگرهای با اولویت بیشتر یا مساوی را pop کن. ارزیابی: دو تا pop، یکی push.
- ✔ صف با دو پشته: سرشکن $O(1)$. «درج و DeleteMin هر دو $O(1)$» نشدنی است؛ «FindMin در $O(1)$» شدنی است.

پاسخ‌نامهٔ فصل ۲

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۱

برگشت ← صفحهٔ ۳۳

۱. **۹**. گام‌ها (کند، تند): $(5,5)$ ، $(8,2)$ ، $(2,4)$ ، $(9,1)$.

۲. **۴**. دومی از گرهٔ چهارم (۹) شروع می‌کند؛ وقتی دومی NULL شد، اولی روی ۴ است (گره‌های آخر: $4, 7, 1$).

۳. لیست **دوطرفه**: گرهٔ قبلی را از prev داریم. در یک‌طرفه باید قبلی را از اول پیدا کنیم و در آرایه بقیه باید جابه‌جا شوند.

تمرین درس ۲

برگشت ← صفحهٔ ۳۷

۱. پشته: $[4]$ ، $[4, 2]$ ، با $-$ ⇐ $[2]$ ، $[2, 3]$ ، با $*$ ⇐ $[6]$. جواب **۶**.

۲. **بله**: pop (۲)، push ۳، push ۲، pop (۳)، pop (۱)، push ۱.

۳. $a\ b\ *\ c\ +$. وقتی $+$ می‌رسد، $*$ قوی‌تر بالای پشته است و اول pop می‌شود.

تمرین فصل ۲

برگشت ← صفحهٔ ۴۰

۱. $a\ b\ +\ c\ *\ d\ e\ /\ -$

۲. $3 \cdot 2 = 6$ ، بعد $5 + 6 = 11$ ، بعد $11 - 4 = 7$.

۳. $3, 1, 2$. برای $n = 4$، تعداد $C_4 = 14$ است.

۴. اشاره‌گر کند و تند. وقتی تند به انتها رسید، کند در وسط است.

آزمون فصل ۲ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۴۱

سؤال ۱	سؤال ۲	سؤال ۳
۲	۲	۴

۱ گزینهٔ ۲

در لیست **یک‌طرفه** نمی‌شود از انتها به عقب حرکت کرد. از نظر حساب درست است (مکان‌ها $3t$ و $n - t$ در $t = n/4$ به هم می‌رسند)، ولی اجراشدنی نیست. بقیه کار می‌کنند:

- گزینهٔ ۳: اشاره‌گر جلویی از $n/4$ شروع می‌کند و بعد از $3n/4$ گام به انتها می‌رسد؛ اشاره‌گر عقبی در همین زمان در $3n/4$ است.
- گزینهٔ ۴: روش کند و تند وسط را می‌دهد. تکرار همین روش روی نیمهٔ دوم، وسط نیمهٔ دوم یعنی $3n/4$ را می‌دهد.

۲ **گزینهٔ ۲ (یکی)**

- مورد اول ساختنی است (پشته‌های کمکی min و max).
- مورد سوم پشتهٔ معمولی است.
- مورد دوم **نشدنی** است: با Push و DeleteMin هر دو در $O(1)$ می‌شد در $O(n)$ مرتب کرد، که با کران $\Omega(n \log n)$ تناقض دارد.

---

۳ **گزینهٔ ۴**

- با یک پشته، جایگشت‌هایی مثل $3, 1, 2$ تولید نمی‌شوند.
- دو پشتهٔ سری هم برای n بزرگ همهٔ جایگشت‌ها را نمی‌سازند.
- با $n - 1$ پشته (و طبعاً با n پشته) همهٔ جایگشت‌ها ساختنی‌اند؛ مثلاً برای $n = ۳$، دو پشته کافی است. بین گزینه‌های کافی، کوچک‌ترین $n - 1$ است.

نکتهٔ تستی: وقتی گزینه‌ها هم n و هم $n - 1$ را دارند و هر دو کافی‌اند، «کوچک‌ترین» یعنی $n - 1$.

فصل ۳

درهم‌سازی (Hashing)

۳

۳ درس | ۶ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۶ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴. تیپ‌ها: ردگیری درج با وارسی خطی، طول زنجیرهٔ بدترین حالت برای کلیدهای خاص، و تعریف درهم‌سازی سراسری.

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ جدول درهم‌سازی و زنجیره‌سازی | ۲ آدرس‌دهی باز | ۳ درهم‌سازی سراسری و کاربردها | 📝 آزمون فصل

درس ۱ جدول درهم‌سازی و زنجیره‌سازی

از صفر: رختکن با شمارهٔ چوب‌لباسی

در آرایه، اگر بخواهی ببینی عدد ۴۲ هست یا نه، باید همه را بگردی ($O(n)$). ایدهٔ درهم‌سازی: از **خود کلید** حساب کن که کجا باید باشد و مستقیم همان‌جا را نگاه کن.

مثل رختکنی که شمارهٔ چوب‌لباسی را از شمارهٔ بلیتت حساب می‌کند: «رقم آخر بلیت = شمارهٔ چوب‌لباسی». بلیت ۲۳ ⇐ چوب‌لباسی ۳. این قاعده همان **تابع درهم‌ساز** است: $h(k) = k \bmod 10$.

✏️ **مثال**

$m = 10$ خانه و $h(k) = k \bmod 10$: کلید ۲۳ ⇐ خانهٔ ۳؛ کلید ۴۷ ⇐ خانهٔ ۷. برای پیدا کردن ۴۷ فقط خانهٔ ۷ را نگاه می‌کنی.

برخورد: کلید ۱۳ هم به خانهٔ ۳ می‌رود؛ دو کت برای یک چوب‌لباسی. دو راه حل داریم:

- **زنجیره‌سازی (همین درس):** هر خانه یک **لیست** دارد و کلیدهای هم‌خانه پشت هم در آن صف می‌کشند.
- **آدرس‌دهی باز (درس ۲):** کلید دوم دنبال خانهٔ خالی دیگری می‌گردد.

ضریب بار $\alpha = n/m$ یعنی به‌طور میانگین چند کلید در هر خانه است؛ همان طول میانگین هر زنجیره. هر جستجو باید زنجیرهٔ یک خانه را بگردد، پس هزینه‌اش به α بستگی دارد.

- جدول با m خانه و تابع $h(k)$ که هر کلید را به یک خانه می‌برد. **برخورد** یعنی دو کلید به یک خانه بروند.
- **ضریب بار:** $\alpha = n/m$.
- **زنجیره‌سازی:** هر خانه یک لیست پیوندی است.
  - امید جستجوی ناموفق: $\Theta(1 + \alpha)$ ؛ امید جستجوی موفق: حدود $1 + \alpha/2$.

- **بدترین حالت** جستجو برابر طول بلندترین زنجیره است؛ اگر همه به یک خانه بروند $\Theta(n)$.

- **تابع درهم‌ساز ثابت، تصادفی نیست.** $h(4)$ همیشه یک مقدار مشخص است و «احتمال» نگاشت یک کلید مشخص به یک خانه یا ۰ است یا ۱.

✏️ مثال

$m = 5$ و $h(k) = k \bmod 5$ ، کلیدهای $1, 6, 11, 3$.

- $1, 6, 11$ همه به خانهٔ ۱ می‌روند (دو برخورد) و ۳ به خانهٔ ۳.
- با زنجیره‌سازی: خانهٔ ۱ ⇐ $11 \to 6 \to 1$ ، خانهٔ ۳ ⇐ 3 .
- $\alpha = 4/5 = 0.8$ ؛ امید جستجوی ناموفق حدود $1 + 0.8$. ولی جستجوی ۱۱ (یا هر کلید ناموجود در خانهٔ ۱ مثل ۱۶) ۳ مقایسه می‌خواهد: طول بلندترین زنجیره.
- $h(6)$ همیشه ۱ است؛ اگر کلیدها بد انتخاب شوند (مثل $1, 6, 11, 16, \dots$) همه در یک زنجیره می‌روند.

✏️ مثال (ردگیری کامل زنجیره‌سازی)

$m = 5$ ، $h(k) = k \bmod 5$ ، کلیدهای $12, 7, 3, 22, 17, 8$ به همین ترتیب (هر کلید ته زنجیرهٔ خانه‌اش اضافه می‌شود).

کلید	$k \bmod 5$	زنجیرهٔ آن خانه بعد از درج
۱۲	۲	12
۷	۲	$7 \to 12$ (برخورد)
۳	۳	3
۲۲	۲	$22 \to 7 \to 12$
۱۷	۲	$17 \to 22 \to 7 \to 12$
۸	۳	$8 \to 3$

- **ضریب بار:** $\alpha = 6/5 = 1.2$ ؛ یعنی به‌طور میانگین ۱٫۲ کلید در هر خانه.
- **جستجوی موفق ۲۲:** خانهٔ ۲ ⇐ مقایسه با ۱۲، ۷، ۲۲ ⇐ **۳ مقایسه**.
- **جستجوی ناموفق ۲۷:** خانهٔ ۲ ⇐ کل زنجیره (۴ کلید) را می‌بیند و پیدا نمی‌کند ⇐ **۴ مقایسه**.
- **جستجوی ناموفق ۴۰:** خانهٔ ۰ خالی است ⇐ **۰ مقایسه**. خانه‌های ۰، ۱ و ۴ خالی‌اند و خانهٔ ۲ شلوغ؛ این کلیدها خوب پخش نشده‌اند (چهار تا با رقم یکان ۲ یا ۷). امید $1 + \alpha$ فقط وقتی معنی دارد که کلیدها **یکنواخت** پخش شوند.

💡 تیپ «کدام m بهتر است؟»

وقتی کلیدها ساختار خاصی دارند (مثلاً همه مربع کامل‌اند)، باقی‌مانده‌ها فقط چند مقدار خاص را می‌گیرند. m ای بهتر است که کلیدها را در **خانه‌های بیشتری** پخش کند. معمولاً m اول و دور از توان‌های ۲ بهتر است.

✏️ مثال

کلیدها مربع‌های کامل $1, 4, 9, 16, 25, 36, 49, 64$ هستند.

- $m = 8$ (توان ۲): باقی‌مانده‌ها فقط $0, 1, 4$ ⇐ ۸ کلید در ۳ خانه.
- $m = 7$ (اول): باقی‌مانده‌ها $0, 1, 2, 4$ ⇐ ۸ کلید در ۴ خانه. پس $m = 7$ بهتر است.

درس ۲ آدرس‌دهی باز

از صفر: پارکینگ بدون جای رزرو

همهٔ کلیدها داخل خود جدول می‌نشینند و لیستی نداریم. مثل پارکینگی که جای هر ماشین را از پلاکش حساب می‌کنند: اگر جایت پر بود، طبق یک قاعدهٔ ثابت سراغ جای بعدی می‌روی (وارسی). موقع **جستجو** هم باید همان مسیر را بروی تا به ماشین برسی یا به یک **جای خالی** (یعنی «نیست»).

✏️ مثال

$m = 7$ ، $h(k) = k \bmod 7$ ، وارسی خطی. درج ۱۰ ⇐ خانهٔ ۳. درج ۱۷ ⇐ خانهٔ ۳ پر است ⇐ ۴. جستجوی ۲۴ ⇐ از ۳ شروع کن: ۱۰ (نه)، ۱۷ (نه)، خانهٔ ۵ خالی ⇐ «۲۴ نیست».

✏️ مثال (ردگیری کامل وارسی خطی و دیدن خوشه)

$m = 7$ ، $h(k) = k \bmod 7$ ، کلیدهای $10, 17, 3, 24, 5$ به همین ترتیب.

کلید	$h(k)$	خانه‌هایی که وارسی شد	جای نهایی	تعداد وارسی
۱۰	۳	۳	۳	۱
۱۷	۳	۳ (پر)، ۴	۴	۲
۳	۳	۳، ۴ (پر)، ۵	۵	۳
۲۴	۳	۳، ۴، ۵ (پر)، ۶	۶	۴
۵	۵	۵، ۶ (پر)، ۰ (دور زد)	۰	۳

جدول نهایی (خانهٔ ۰ تا ۶): $5, –, –, 10, 17, 3, 24$. خانه‌های ۳ تا ۶ و ۰ یک **خوشهٔ** پشت‌سرهم ساخته‌اند. کلید ۵ با اینکه خانهٔ خانگی‌اش (۵) با بقیه فرق داشت، گرفتار همین خوشه شد و ۳ وارسی لازم داشت. این همان **خوشه‌بندی اولیه** است: هر چه خوشه بزرگ‌تر شود، احتمال برخورد کلیدهای تازه با آن بیشتر و خوشه باز هم بزرگ‌تر می‌شود.

همهٔ کلیدها داخل خود جدول‌اند (پس $\alpha \le 1$). اگر خانه پر بود، خانهٔ بعدی دنبالهٔ وارسی را امتحان کن.

روش	دنبالهٔ وارسی	مشکل
وارسی خطی	$h(k), h(k)+1, h(k)+2, \dots$ (به پیمانهٔ m)	**خوشه‌بندی اولیه**: خانه‌های پر پشت هم جمع می‌شوند
وارسی درجه دوم	$h(k) + c_1 i + c_2 i^2$	خوشه‌بندی ثانویه
درهم‌سازی دوگانه	$h_1(k) + i \cdot h_2(k)$	بهترین؛ $h_2(k)$ باید نسبت به m اول باشد

✏️ مثال برای هر سطر جدول

$m = 7$ ، کلیدهای $14, 21, 28$ (همه با $k \bmod 7 = 0$)، به همین ترتیب.

- **خطی:** ۱۴ در ۰، ۲۱ در ۱، ۲۸ در ۲. حالا کلید ۱ (خانهٔ خانگی‌اش ۱) هم باید تا خانهٔ ۳ برود؛ خوشه بزرگ‌تر می‌شود.
- **درجه دوم ($h + i^2$):** ۱۴ در ۰، ۲۱ در $0 + 1 = 1$ ، ۲۸ بعد از ۰ و ۱ در $0 + 4 = 4$. ولی همهٔ کلیدهای با خانهٔ خانگی ۰ همین دنبالهٔ $0, 1, 4, \dots$ را می‌روند (خوشه‌بندی ثانویه).
- **دوگانه** با $h_2(k) = 1 + (k \bmod 5)$: ۱۴ در ۰؛ ۲۱ با گام $h_2 = 2$ در خانهٔ ۲؛ ۲۸ با گام $h_2 = 4$ در خانهٔ ۴. کلیدهای هم‌خانه، گام‌های متفاوت دارند.
- امید تعداد وارسی در جستجوی ناموفق (با فرض یکنواخت): حداکثر $1/(1-\alpha)$.
- **حذف:** نمی‌شود خانه را خالی کرد، چون زنجیرهٔ وارسی بقیه را می‌شکند. خانه را «حذف‌شده» علامت می‌زنند.

✏️ مثال

- **امید وارسی:** با $\alpha = 0.5$ حداکثر ۲ وارسی؛ با $\alpha = 0.9$ حداکثر ۱۰. نزدیک پر شدن، هزینه منفجر می‌شود.
- **حذف:** در جدول خطی بالا (۱۴ در ۰، ۲۱ در ۱، ۲۸ در ۲)، اگر ۲۱ را واقعاً پاک کنی، جستجوی ۲۸ از خانهٔ ۰ شروع می‌کند، به خانهٔ خالی ۱ می‌رسد و به اشتباه «نیست» می‌گوید. با علامت «حذف‌شده»، جستجو از خانهٔ ۱ رد می‌شود و ۲۸ را در خانهٔ ۲ پیدا می‌کند.

ردگیری وارسی خطی (تیپ «چند ترتیب ورودی؟»)

اگر کلید x در خانه‌ای غیر از $h(x)$ نشسته، **همهٔ خانه‌های بین $h(x)$ و جای فعلی‌اش قبل از ورود x پر بوده‌اند.** این قیدها یک ترتیب جزئی می‌سازند. تعداد ترتیب‌های سازگار را بشمار؛ کلیدهایی که قیدی ندارند هر جایی می‌توانند بیایند.

✏️ مثال

$m = 7$ ، $h(k) = k \bmod 7$ ، و جدول نهایی: خانهٔ ۳ = ۱۰، خانهٔ ۴ = ۱۷، خانهٔ ۵ = ۵.

- $h(10) = 3$ و $h(5) = 5$: هر دو در خانهٔ خودشان‌اند ⇐ بدون قید.
- $h(17) = 3$ ولی ۱۷ در خانهٔ ۴ است ⇐ خانهٔ ۳ قبل از ۱۷ پر بوده، یعنی **۱۰ قبل از ۱۷**.
- ترتیب‌های سازگار: $3!/2 = 3$ ، یعنی $(10, 17, 5)$ ، $(10, 5, 17)$ ، $(5, 10, 17)$.

تمرین درس ۲ — اول خودت حل کن، بعد پاسخ‌نامه

کلیدهای $10, 22, 31, 4, 15, 28$ را به‌ترتیب در جدول ۱۱ خانه‌ای با $h(k) = k \bmod 11$ و وارسی خطی درج کن.

پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۵۳**

درس ۳ درهم‌سازی سراسری و کاربردها

از صفر: چرا تابع را تصادفی انتخاب کنیم؟

هر تابع درهم‌ساز **ثابت** یک نقطه‌ضعف دارد: کسی که آن را بداند، می‌تواند کلیدهایی بدهد که همه در یک خانه بیفتند (مثلاً برای $k \bmod 10$، کلیدهای $3, 13, 23, 33, \dots$) و جستجو $O(n)$ شود. راه حل: یک **خانوادهٔ** بزرگ از تابع‌ها داشته باش و موقع اجرا یکی را **تصادفی** انتخاب کن، طوری که حریف نداند کدام انتخاب شده. **سراسری** بودن یعنی هر دو کلید دلخواه، فقط با احتمال کم (حداکثر $1/m$) برخورد کنند.

- **تعریف:** خانوادهٔ H از توابع، **سراسری** است اگر برای **هر** دو کلید متمایز u و v: $\Pr[h(u) = h(v)] \le 1/m$ (h تصادفی از H انتخاب می‌شود).
- **خانوادهٔ استاندارد:** $h_{ab}(x) = ((a \cdot x + b) \bmod p) \bmod m$ با p اول، $1 \le a \le p-1$ و $0 \le b \le p-1$. پس $|H| = p(p-1)$.
- **ترفند شمارش:** با k تابع، هر کلید یک «امضا» (بردار k مقدار) دارد. دو کلید با امضای یکسان در **همهٔ** توابع برخورد دارند و احتمال برخوردشان ۱ است. پس تعداد کلیدها حداکثر برابر تعداد امضاهای متمایز است.
- **کاربرد: دو عدد با جمع k.**
  - با درهم‌سازی: امید $O(n)$، ولی بدترین حالت $O(n^2)$.
  - بدون فرض تصادفی: مرتب‌سازی + دو اشاره‌گر در $O(n \log n)$. وقتی تست «الگوریتم کارا» را بدون فرض تصادفی بخواهد، این جواب را می‌خواهد.

✏️ مثال برای هر مورد

- **سراسری:** با دو کلید u, v و $m = 2$ ، خانوادهٔ «همهٔ ۴ تابع از $\{u, v\}$ به $\{0, 1\}$ » سراسری است: در ۲ تا از ۴ تابع برخورد دارند و $2/4 = 1/m$. ولی خانواده‌ای که فقط تابع ثابت $h(x) = 0$ را دارد سراسری نیست، چون احتمال برخورد ۱ است.
- **خانوادهٔ استاندارد:** با $p = 5$ و $m = 3$: $|H| = 5 \times 4 = 20$. مثلاً $h_{2,1}(3) = ((2\cdot3 + 1) \bmod 5) \bmod 3 = (7 \bmod 5) \bmod 3 = 2$.
- **ترفند شمارش:** با ۲ تابع و $m = 2$ ، امضاها $2^2 = 4$ تا هستند (00, 01, 10, 11). اگر ۵ کلید باشد، دو تا امضای یکسان دارند و احتمال برخوردشان ۱ است. پس چنین خانواده‌ای حداکثر برای ۴ کلید سراسری است.
- **دو عدد با جمع ۸** در $[8, 3, 5, 1]$:
  - درهم‌سازی: برای ۸ دنبال ۰ بگرد (نیست)، درجش کن؛ برای ۳ دنبال ۵ (نیست)، درج؛ برای ۵ دنبال ۳ ⇐ **پیدا شد.**
  - مرتب‌سازی: $[1, 3, 5, 8]$ ، اشاره‌گر چپ روی ۱ و راست روی ۸: $9 > 8$ ⇐ راست به ۵: $6 < 8$ ⇐ چپ به ۳: $3 + 5 = 8$ ✓.

✏️ مثال (سراسری بودن را با دست بشمار)

خانوادهٔ استاندارد با $p = 5$ و $m = 2$ ، یعنی $h(x) = ((a\cdot x + b) \bmod 5) \bmod 2$ با $a \in \{1, 2, 3, 4\}$ و $b \in \{0, \dots, 4\}$ (۲۰ تابع). برای کلیدهای $u = 1$ و $v = 3$ ، همهٔ ۲۰ تابع را امتحان کن:

- مثلاً $a = 1, b = 0$: $h(1) = 1 \bmod 2 = 1$ و $h(3) = 3 \bmod 2 = 1$ ⇐ **برخورد**.
- $a = 1, b = 2$: $h(1) = 3 \bmod 2 = 1$ و $h(3) = 5 \bmod 5 = 0$ ⇐ $0 \bmod 2 = 0$ ⇐ بدون برخورد.
- با شمردن همه: در **۸** تابع از ۲۰ تابع برخورد هست ⇐ احتمال $8/20 = 0.4 \le 1/m = 0.5$ ✓.

یعنی هر تابعی که تصادفی انتخاب شود، این دو کلید فقط با احتمال ۰٫۴ هم‌خانه می‌شوند؛ و این برای **هر** جفت کلید برقرار است، پس حریف نمی‌تواند کلیدهای بد انتخاب کند.

✍️ تمرین درس ۳

اول خودت حل کن، بعد پاسخ‌نامه

۱. خانوادهٔ $((ax+b) \bmod 13) \bmod 5$ چند تابع دارد؟

۲. جدول زنجیره‌ای با ۱۰ خانه و ۲۵ کلید. امید جستجوی ناموفق؟

۳. آدرس‌دهی باز با $\alpha = 0.75$. کران امید وارسی جستجوی ناموفق؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۵۳**

📝 آزمون فصل ۳

سؤال‌های واقعی کنکور ارشد کامپیوتر

۶ سؤال | ⏱️ زمان: ۱۵ دقیقه | 🔑 کلید و راه‌حل: **صفحهٔ ۵۳**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۳ · سؤال ۵۶ — غلط یا حدسی؟ ☐

گوییم $H = \{h : U \to \{0, 1, \ldots, m-1\}\}$ یک خانواده از درهم‌سازهای سراسری (universal) است، هرگاه برای هر $h \in H$ و $K_1, K_2 \in U$ داشته باشیم $\Pr(h(K_1) = h(K_2)) \le 1/m$. اگر مجموعهٔ H که فقط متشکل از توابعی به شکل $h_ab(x) = ((ax + b) \bmod n) \bmod 20$ است، یک خانوادهٔ درهم‌ساز سراسری باشد، آنگاه در مورد تعداد عناصر H چه می‌توان گفت؟

۱) H می‌تواند ۳۸۰ عضو داشته باشد.
۲) H می‌تواند ۴۲۰ عضو داشته باشد.
۳) H می‌تواند ۵۰۶ عضو داشته باشد.
۴) H می‌تواند ۶۵۰ عضو داشته باشد.

۲ ارشد ۱۴۰۲ · سؤال ۶۵ — غلط یا حدسی؟ ☐

فرض کنید $A = [a_1, a_2, \ldots, a_n]$ آرایه‌ای از اعداد صحیح متمایز باشد و k یک عدد صحیح داده شده باشد. هدف این است که دو عدد متمایز از A را پیدا کنید که مجموع آن‌ها دقیقاً k باشد، یا گزارش دهید که چنین عناصری وجود ندارد. یک الگوریتم کارا برای حل این مسئله چه زمانی خواهد داشت؟

۱) $O(\log n)$ ۲) $O(n)$ ۳) $O(n \log n)$ ۴) $O(n^2)$

۳ ارشد ۱۴۰۰ · سؤال ۶۱ — غلط یا حدسی؟ ☐

خانوادهٔ $H = \{h_1, h_2\}$ از توابع درهم‌ساز را در نظر بگیرید که $h_i : \{1, 2, \ldots, n\} \to \{0, 1\}$. برای آنکه این خانواده یک خانوادهٔ درهم‌ساز سراسری باشد، n حداکثر چند می‌تواند باشد؟ خانوادهٔ توابع H سراسری است، اگر و فقط اگر به ازای هر دو مقدار u و v داشته باشیم $\Pr[h(u) = h(v)] \le 1/m$ که m اندازهٔ جدول درهم‌سازی است.

۱) ۲
۲) ۴
۳) ۸
۴) به ازای هر n این کار امکان‌پذیر است.

۴ ارشد ۱۳۹۹ · سؤال ۸۶ — غلط یا حدسی؟

جدول درهم‌سازی ۱۰ خانه‌ای و تابع درهم‌ساز $h(x) = 3x + 5 \bmod 10$ و روش زنجیره‌ای به‌عنوان روش رفع تصادم را در نظر بگیرید. در این خصوص کدام گزینه درست است؟

۱) احتمال آنکه ورودی $x = 4$ به خانهٔ ۷ نگاشت شود برابر ۱ است.

۲) احتمال آنکه ورودی $x = 4$ به خانهٔ ۷ نگاشت شود برابر ۰ است.

۳) احتمال آنکه ورودی $x = 4$ به خانهٔ ۷ نگاشت شود برابر $1/10$ است.

۴) احتمال آنکه دو ورودی مختلف به یک خانه نگاشت شوند برابر $1/10$ است.

۵ ارشد ۱۳۹۹ · سؤال ۹۷ — غلط یا حدسی؟

فرض کنید برای درهم‌سازی از روش زنجیره‌ای با یک جدول به اندازهٔ m استفاده شده است. تابع درهم‌ساز رکورد با کلید k را به خانهٔ $k \bmod m$ نگاشت می‌کند. اگر بدانیم کلید رکوردها زیرمجموعهٔ $\{i^2 \mid 1 \le i \le 100\}$ است، به ازای کدام m هزینهٔ جستجو در بدترین حالت کمتر است؟

۱) ۷ ۲) ۹ ۳) ۱۱ ۴) ۱۲

۶ ارشد ۱۳۹۸ · سؤال ۵۵ — غلط یا حدسی؟

فرض کنید از آدرس‌دهی باز و وارسی خطی برای درهم‌سازی استفاده شده و تابع درهم‌ساز i^2 به پیمانهٔ ۷ است. بعد از دریافت همهٔ اعداد $0, \ldots, 6$ می‌دانیم نحوهٔ قرارگیری اعداد در جدول درهم‌ساز به‌صورت زیر است: $A[0..6] = 0, 6, 4, 3, 1, 5, 2$ به ازای چند جایگشت ورودی، وضعیت جدول درهم‌ساز به شکل بالا خواهد بود؟

۱) ۱۸ ۲) ۲۱ ۳) ۳۶ ۴) ۱۲۰

کارت خلاصهٔ فصل ۳

برای مرور سریع، شب قبل از آزمون

- ✔ $\alpha = n/m$. زنجیره‌ای: امید $1 + \alpha$ و بدترین حالت = بلندترین زنجیره. آدرس‌دهی باز: امید $\le 1/(1-\alpha)$.
- ✔ وارسی خطی خوشهٔ اولیه دارد، درجه دوم خوشهٔ ثانویه، دوگانه بهترین است. حذف با علامت «حذف‌شده».
- ✔ ردگیری وارسی خطی: کلیدی که جابه‌جا نشسته، بعد از پر شدن خانه‌های بین $h(x)$ و جایش آمده است.
- ✔ سراسری: $\Pr[\text{برخورد}] \ge 1/m$. خانوادهٔ $((ax+b) \bmod p) \bmod m$ با p اول، $p(p-1)$ عضو دارد. امضاهای متمایز ⇐ کران n.
- ✔ کلیدهای خاص (مثل مربع‌ها): m ای که باقی‌ماندهٔ متمایز بیشتری می‌دهد بهتر است.
- ✔ «دو عدد با جمع k» به‌صورت تضمین‌شده $O(n \log n)$ است.

پاسخ‌نامهٔ فصل ۳

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین درس ۲

برگشت ← صفحهٔ ۴۹

- $10 \to 10$ ، $22 \to 0$ ، $31 \to 9$ ، $4 \to 4$
- $15 \to 4$ پر است، پس ۵
- $28 \to 6$

جدول (خانه ۰ تا ۱۰): $22, -, -, -, 4, 15, 28, -, -, 31, 10$

تمرین درس ۳

برگشت ← صفحهٔ ۵۰

۱. $13 \times 12 = 156$

۲. $\alpha = 2.5$ ، پس حدود $1 + 2.5 = 3.5$

۳. $1/(1 - 0.75) = 4$

آزمون فصل ۳ – کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۵۱

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶
۳	۳	۲	۱	۳	۲

۱ گزینهٔ ۳

در خانوادهٔ استاندارد، n باید عددی **اول** و حداقل به اندازهٔ $m = ۲۰$ باشد، و $|H| = n(n-1)$. داریم $20 \cdot 19 = 380$ (۲۰ اول نیست)، $21 \cdot 20 = 420$ (۲۱ اول نیست)، $23 \cdot 22 = 506$ (**۲۳ اول است** ✓) و $26 \cdot 25 = 650$ (۲۶ اول نیست).

۲ گزینهٔ ۳

مرتب‌سازی و دو اشاره‌گر: $O(n \log n)$. درهم‌سازی فقط **امید** $O(n)$ می‌دهد و بدترین حالتش $O(n^2)$ است، پس جواب تضمین‌شده $O(n \log n)$ است.

۳ گزینهٔ ۲

$m = 2$ ، پس برای هر دو کلید باید $\Pr[\text{برخورد}] \geq 1/2$ باشد؛ یعنی دو کلید **نباید** در هر دو تابع برخورد کنند. امضای هر کلید زوج $(h_1(u), h_2(u))$ است که ۴ حالت دارد و امضاها باید متمایز باشند. پس $n \leq 4$.

۴ **گزینهٔ ۱**

$h(4) = 17 \bmod 10 = 7$. تابع قطعی است، پس احتمال **۱** است. گزینهٔ ۴ ادعایی کلی است که برای یک تابع ثابت معنی ندارد.

---

۵ **گزینهٔ ۳**

باقی‌ماندهٔ مربع‌ها:

- بر ۷: فقط ۴ مقدار ($0, 1, 2, 4$)؛ بلندترین زنجیره ۲۹.
- بر ۹: ۴ مقدار؛ بلندترین زنجیره ۳۳.
- بر ۱۲: ۴ مقدار؛ بلندترین زنجیره ۳۴.
- بر ۱۱: **۶ مقدار** ($0, 1, 3, 4, 5, 9$)؛ بلندترین زنجیره **۱۹**.

پس $m = 11$ بهترین است.

---

۶ **گزینهٔ ۲**

مقادیر h: $0\to0$ ، $1\to1$ ، $2\to4$ ، $3\to2$ ، $4\to2$ ، $5\to4$ ، $6\to1$. از جدول این قیدها به دست می‌آیند:

- ۶ در خانهٔ ۱ (خانهٔ خودش) است، پس **قبل از ۱** آمده (وگرنه ۱ آنجا می‌نشست).
- ۴ در خانهٔ ۲ است، پس **قبل از ۳** آمده.
- ۳ در خانهٔ ۳ است، پس بعد از پر شدن خانهٔ ۲ آمده: $4 < 3$.
- ۱ در خانهٔ ۴ است، پس خانه‌های ۱، ۲ و ۳ قبلاً پر بوده‌اند: $6, 4, 3 < 1$.
- ۵ در خانهٔ ۵ است ($h = 4$)، پس $1 < 5$.
- ۲ در خانهٔ ۶ است ($h = 4$)، پس $1, 5 < 2$.

ترتیب ۶ کلید $\{4, 3, 6, 1, 5, 2\}$ به این شکل است: قبل از ۱، سه کلید $\{4, 3, 6\}$ با شرط $4 < 3$ (**۳ حالت**)؛ بعد از ۱، ترتیب $5, 2$ ثابت است. کلید ۰ قیدی ندارد و در یکی از ۷ جایگاه می‌نشیند: $3 \times 7 = 21$. (با شمارش رایانه‌ای همهٔ $7!$ ترتیب هم تأیید شد.)

فصل ۴

درخت‌ها (BST و AVL) و هیپ

۴

۵ درس | ۷ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۷ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴ (۴ سؤال درخت و ۳ سؤال هیپ). تیپ‌ها: «چه پرسمانی بدون اطلاعات اضافه در $O(\log n)$ جواب دارد؟»، کران ارتفاع، تعداد چرخش، و جای اعداد در هیپ.

⏱️ **زمان مطالعه** دو بلوک ۹۰ دقیقه‌ای (درس ۱ تا ۳، بعد درس ۴ و ۵).

نقشهٔ فصل: ۱ درخت دودویی و پیمایش‌ها | ۲ درخت جستجوی دودویی (BST) | ۳ AVL و چرخش | ۴ هیپ (Heap) | ۵ انتخاب ساختار ترکیبی | 📝 آزمون فصل

درس ۱ درخت دودویی و پیمایش‌ها

از صفر: درخت خانوادگی وارونه

درخت مثل شجره‌نامه است که **ریشه** بالاست و شاخه‌ها پایین می‌آیند. واژه‌ها را با این درخت یاد بگیر: ریشهٔ ۱، فرزندان ۲ و ۳؛ فرزندان ۲ هم ۴ و ۵.

- **ریشه:** گرهٔ بالایی (۱). **والد و فرزند:** ۲ والد ۴ و ۵ است.
- **برگ:** گرهٔ بی‌فرزند (۴، ۵، ۳).
- **عمق یک گره:** تعداد یال‌ها از ریشه تا آن (عمق ۴ برابر ۲ است). **ارتفاع درخت:** بیشترین عمق (اینجا ۲).
- **درخت دودویی:** هر گره حداکثر ۲ فرزند دارد: فرزند **چپ** و فرزند **راست**.

روش دیدنی برای سه پیمایش

پیمایش یعنی دیدن همهٔ گره‌ها با یک ترتیب ثابت. سه ترتیب فقط در **جای ریشه** فرق دارند: پیش‌ترتیب (ریشه اول)، میان‌ترتیب (ریشه وسط)، پس‌ترتیب (ریشه آخر).

💡 روش (دور زدن درخت)

با مداد از سمت چپ ریشه شروع کن و دور تا دور درخت را بچرخ، طوری که همیشه به گره‌ها چسبیده باشی.

- **پیش‌ترتیب:** هر گره را وقتی بنویس که از **سمت چپش** رد می‌شوی.
- **میان‌ترتیب:** وقتی از **زیرش** رد می‌شوی.
- **پس‌ترتیب:** وقتی از **سمت راستش** رد می‌شوی.
- **ارتفاع** = تعداد یال‌های بلندترین مسیر از ریشه تا برگ. درخت با ارتفاع h حداکثر $2^{(h+1)} - 1$ گره دارد، پس $h \geq \lfloor \log_2 n \rfloor$.
- در هر درخت دودویی: **تعداد برگ‌ها = تعداد گره‌های دو فرزندی + ۱**.
- تعداد درخت‌های دودویی متمایز با n گره، عدد کاتالان C_n است.
- **درخت کامل (complete):** همهٔ سطح‌ها پر، جز احتمالاً سطح آخر که از چپ پر می‌شود. شکل هیپ همین است.

✏️ مثال برای هر مورد

درخت با ریشهٔ ۱، فرزندان ۲ و ۳، و گرهٔ ۴ فرزند چپ ۲.

- **ارتفاع** ۲ است (مسیر $1 \to 2 \to 4$). درختی با ارتفاع ۲ حداکثر $2^3 - 1 = 7$ گره دارد.
- **برگ‌ها** ۴ و ۳ هستند (۲ برگ). گرهٔ دوفرزندی فقط ۱ است، و $1 + 1 = 2$ ✓.
- **کاتالان:** با ۳ گره دقیقاً $C_3 = 5$ شکل درخت دودویی داریم (دو زنجیر چپ‌چپ و راست‌راست، دو زنجیر زیگزاگ، و یک ریشه با دو فرزند).
- **کامل:** این درخت کامل است (سطح آخر از چپ پر شده). اگر ۴ فرزند **راست** ۲ بود، کامل نبود.

پیمایش	ترتیب
پیش‌ترتیب	ریشه، چپ، راست
میان‌ترتیب	چپ، ریشه، راست (در BST خروجی **مرتب** است)
پس‌ترتیب	چپ، راست، ریشه

✏️ مثال

درخت با ریشهٔ ۱، فرزند چپ ۲ (با فرزندان ۴ و ۵) و فرزند راست ۳:

- پیش‌ترتیب: 1 2 4 5 3
- میان‌ترتیب: 4 2 5 1 3
- پس‌ترتیب: 4 5 2 3 1

✏️ مثال (پیش‌ترتیب قدم‌به‌قدم، به روش بازگشتی)

قانون پیش‌ترتیب برای هر گره: «اول خودت را بنویس، بعد کل زیردرخت چپ، بعد کل زیردرخت راست». روی همان درخت:

قدم	گرهٔ فعلی	کار	خروجی تا اینجا
۱	۱	بنویس، برو چپ	1
۲	۲	بنویس، برو چپ	1 2
۳	۴	بنویس (برگ است، برگرد)	1 2 4
۴	۲	چپ تمام شد، برو راست	
۵	۵	بنویس (برگ)	1 2 4 5
۶	۱	زیردرخت چپ تمام شد، برو راست	
۷	۳	بنویس (برگ)	1 2 4 5 3

میان‌ترتیب و پس‌ترتیب همین مسیر را می‌روند؛ فقط لحظهٔ نوشتن فرق می‌کند (بعد از برگشتن از چپ، یا بعد از برگشتن از هر دو طرف).

بازسازی: «پیش‌ترتیب + میان‌ترتیب» یا «پس‌ترتیب + میان‌ترتیب» درخت را **یکتا** تعیین می‌کنند. «پیش + پس» یکتا نیست (مگر درخت پر باشد).

💡 روش بازسازی

اولین عضو پیش‌ترتیب ریشه است. جای آن در میان‌ترتیب، زیردرخت چپ و راست را جدا می‌کند. همین کار را بازگشتی تکرار کن.

✏️ مثال (بازسازی)

پیش‌ترتیب 1 2 4 5 3 و میان‌ترتیب 4 2 5 1 3 .

- ریشه ۱ است. در میان‌ترتیب، 4 2 5 سمت چپ ۱ و 3 سمت راست است.
- زیردرخت چپ: پیش‌ترتیبش 2 4 5 است، پس ریشه‌اش ۲؛ میان‌ترتیب 4 2 5 می‌گوید ۴ چپ و ۵ راست است.
- همان درخت مثال قبل به دست می‌آید.

✏️ مثال (چرا «پیش + پس» یکتا نیست)

درخت «A با فرزند **چپ** B» و درخت «A با فرزند **راست** B» هر دو پیش‌ترتیب A B و پس‌ترتیب B A دارند. میان‌ترتیبشان فرق می‌کند (B A در برابر A B).

درس ۲ درخت جستجوی دودویی (BST)

از صفر: بازی «بزرگ‌تر یا کوچک‌تر؟»

یک عدد بین ۱ تا ۱۰۰ را حدس بزن؛ فقط جواب «بزرگ‌تر» یا «کوچک‌تر» می‌گیری. بهترین راه این است که هر بار وسط را بپرسی. BST همین بازی را در قالب درخت ذخیره می‌کند: در هر گره، **همهٔ کوچک‌ترها سمت چپ** و **همهٔ بزرگ‌ترها سمت راست** هستند. پس جستجو از ریشه شروع می‌شود و در هر گره فقط یک طرف را ادامه می‌دهی.

✏️ مثال (جستجوی ۴۰)

در BST با ریشهٔ ۵۰، فرزندان ۳۰ و ۷۰، و ۲۰ و ۴۰ زیر ۳۰: $40 < 50$ ⇐ چپ؛ $40 > 30$ ⇐ راست؛ پیدا شد. فقط ۳ گره دیده شد. هزینهٔ جستجو = **عمق** گره، پس هر چه درخت کوتاه‌تر باشد بهتر است.

- قاعده: همهٔ کلیدهای زیردرخت چپ < ریشه < همهٔ کلیدهای زیردرخت راست.
- جستجو، درج، حذف، min، max، پیش‌رو و پس‌رو همه $O(h)$ هستند.
- **حذف گرهٔ دوفرزندی:** کلیدش را با **پس‌رو** (کوچک‌ترین کلید زیردرخت راست) یا پیش‌رو عوض کن.
- **h بین $\log n$ و n است.** درج کلیدها به‌ترتیب مرتب، درخت را یک **زنجیر** با ارتفاع $n-1$ می‌کند.

✏️ مثال (درج قدم‌به‌قدم)

کلیدهای $50, 30, 70, 20, 40, 60, 80$ را به همین ترتیب درج کن. هر کلید از ریشه شروع می‌کند و تا جای خالی پایین می‌رود:

کلید	مسیر	جای نهایی
۵۰	–	ریشه
۳۰	$30 < 50$	چپ ۵۰
۷۰	$70 > 50$	راست ۵۰
۲۰	$20 < 50$ ⇐ $20 < 30$	چپ ۳۰
۴۰	$40 < 50$ ⇐ $40 > 30$	راست ۳۰
۶۰	$60 > 50$ ⇐ $60 < 70$	چپ ۷۰
۸۰	$80 > 50$ ⇐ $80 > 70$	راست ۷۰

درخت حاصل کاملاً متوازن است (ارتفاع ۲). میان‌ترتیبش $20\ 30\ 40\ 50\ 60\ 70\ 80$ است، یعنی مرتب.

حذف در BST: سه حالت

حالت	کار	مثال روی درخت بالا
گره **برگ** است	فقط حذفش کن	حذف ۲۰ ⇐ ۳۰ فقط فرزند راست ۴۰ را دارد
گره **یک فرزند** دارد	فرزند را جای گره بگذار (پدربزرگ به نوه وصل شود)	بعد از حذف ۲۰، حذف ۳۰ ⇐ ۴۰ مستقیم فرزند چپ ۵۰ می‌شود
گره **دو فرزند** دارد	کلیدش را با **پس‌رو** (کوچک‌ترین کلید زیردرخت راست) عوض کن، بعد پس‌رو را حذف کن	حذف ۵۰ ⇐ پس‌رو ۶۰ است (از ۷۰ یک بار چپ) ⇐ ۶۰ جای ۵۰ می‌نشیند و برگ ۶۰ حذف می‌شود

چرا پس‌رو؟ چون از همهٔ زیردرخت چپ بزرگ‌تر و از بقیهٔ زیردرخت راست کوچک‌تر است، پس ترتیب BST حفظ می‌شود. پس‌رو هیچ‌وقت فرزند چپ ندارد، پس حذف خودش از حالت ۱ یا ۲ است.

✏️ مثال

کلیدهای $50, 30, 70, 20, 40, 60, 80$ را به‌ترتیب درج کن: ریشه ۵۰، سطح دوم ۳۰ و ۷۰، سطح سوم ۲۰، ۴۰، ۶۰، ۸۰ (ارتفاع ۲).

- **جستجوی ۶۰:** $50 \to 70 \to 60$ (۳ مقایسه).
- **حذف ۵۰ (دوفرزندی):** پس‌رو، یعنی کوچک‌ترین کلید زیردرخت راست، ۶۰ است. ۶۰ جای ۵۰ می‌نشیند و برگ ۶۰ حذف می‌شود.
- **درج مرتب:** $1, 2, 3, 4$ زنجیری با ارتفاع ۳ می‌سازد و جستجوی ۴ چهار مقایسه می‌خواهد.

پرسمان‌هایی که اطلاعات اضافه می‌خواهند

در BST معمولی (هر گره فقط کلید و دو اشاره‌گر):

پرسمان	$O(h)$ ؟
کوچک‌ترین یا بزرگ‌ترین	✅ (همیشه چپ یا همیشه راست)
آیا x هست؟	✅
رتبهٔ x (چندمین کوچک‌ترین است)	❌ به **اندازهٔ زیردرخت** در هر گره نیاز دارد
میانه یا kامین کوچک‌ترین	❌ همان دلیل

✏️ مثال (رتبه با اندازهٔ زیردرخت)

در BST مثال قبل، رتبهٔ ۴۰ را می‌خواهیم.

- در ۵۰: $40 < 50$ ، برو چپ.
- در ۳۰: $40 > 30$ ، پس ۳۰ و زیردرخت چپش (۱ گره) کوچک‌ترند: رتبه $+= 1 + 1 = 2$. برو راست.
- در ۴۰: زیردرخت چپش خالی است: رتبه $+= 0 + 1$ ⇐ **۳** (۲۰، ۳۰، ۴۰).
- بدون فیلد اندازه، باید کل زیردرخت چپ را بشماری، که $O(n)$ است.

کران ارتفاع از روی جمع هزینه‌ها

اگر یک مسیر به طول h وجود داشته باشد، گره‌های روی آن هزینه‌های $1, 2, \dots, h$ دارند و جمعشان حدود $h^2/2$ است. پس اگر جمع کل هزینه‌ها S باشد، $h = O(\sqrt{S})$.

✏️ مثال

اگر $S = 50$: $h(h+1)/2 \le 50$ ⇐ $h \le 9$ (چون $9 \cdot 10/2 = 45$ ولی $10 \cdot 11/2 = 55$).

درخت وزن‌متوازن

اگر در هر گره، اندازهٔ هر زیردرخت حداکثر کسر c از اندازهٔ گره باشد ($c < 1$)، ارتفاع حداکثر $\log_{(1/c)} n$ است. مثلاً با $c = 2/3$ ارتفاع $\log_{(3/2)} n \approx 1.71 \log_2 n$ است.

✏️ مثال

با $n = 1000$ و $c = 2/3$: ارتفاع حداکثر $\log_{(1.5)} 1000 \approx 17$ ؛ درخت کاملاً متوازن حدود $\log_2 1000 \approx 10$ دارد.

درس ۳ AVL و چرخش

از صفر: چرا درخت را متوازن نگه داریم؟

درس ۲ دیدی که درج عددهای مرتب ($1, 2, 3, 4$) در BST یک زنجیر می‌سازد و جستجو $O(n)$ می‌شود. **AVL** بعد از هر درج و حذف بررسی می‌کند که درخت **کج** نشده باشد و اگر شده، با **چرخش** صافش می‌کند تا ارتفاع همیشه حدود $\log n$ بماند.

ضریب توازن هر گره = ارتفاع زیردرخت چپ منهای ارتفاع زیردرخت راست. در AVL باید همیشه -1 ، 0 یا $+1$ باشد. اگر جایی ۲ یا -2 شد، آنجا چرخش لازم است.

✏️ مثال (حساب ضریب توازن)

زنجیر $30 \to 20 \to 10$ (هر کدام فرزند راست قبلی). ارتفاع زیردرخت خالی را -1 بگیر:

گره	ارتفاع چپ	ارتفاع راست	ضریب توازن
۳۰	-۱	-۱	۰
۲۰	-۱	۰	-۱
۱۰	-۱	۱	**-۲** ⇐ نامتوازن

اولین گرهٔ نامتوازن از پایین ۱۰ است؛ چرخش همان‌جا انجام می‌شود.

چهار حالت چرخش

اسم حالت از مسیر **گرهٔ نامتوازن تا کلید تازه** می‌آید: دو قدم اول این مسیر چپ (L) یا راست (R) است.

حالت	شکل	درمان
LL	کلید تازه در چپِ چپ	یک چرخش **راست** روی گرهٔ نامتوازن
RR	کلید تازه در راستِ راست	یک چرخش **چپ** روی گرهٔ نامتوازن
LR	کلید تازه در راستِ چپ (زیگزاگ)	اول چرخش چپ روی فرزند چپ، بعد چرخش راست روی گرهٔ نامتوازن
RL	کلید تازه در چپِ راست (زیگزاگ)	اول چرخش راست روی فرزند راست، بعد چرخش چپ روی گرهٔ نامتوازن

✏️ مثال (RL)

درج $10, 30, 20$: ۳۰ راست ۱۰ و ۲۰ چپ ۳۰ است (راست، بعد چپ). اول چرخش راست روی ۳۰ (زنجیر $10 \to 20 \to 30$ می‌شود، یعنی RR)، بعد چرخش چپ روی ۱۰ ⇐ ریشه ۲۰ با فرزندان ۱۰ و ۳۰.

✏️ مثال (پنج درج پشت‌سرهم)

درج $10, 20, 30, 40, 50$ در AVL:

درج	چه شد	درخت بعد از آن
۱۰، ۲۰	متوازن	۱۰ با فرزند راست ۲۰
۳۰	۱۰ ضریب -۲ (RR) ⇐ چرخش چپ روی ۱۰	ریشه ۲۰؛ فرزندان ۱۰ و ۳۰
۴۰	متوازن (۲۰ ضریب -۱)	۴۰ فرزند راست ۳۰
۵۰	۳۰ ضریب -۲ (RR) ⇐ چرخش چپ روی ۳۰	ریشه ۲۰؛ چپ ۱۰؛ راست ۴۰ با فرزندان ۳۰ و ۵۰

در BST معمولی همین ورودی زنجیری با ارتفاع ۴ می‌ساخت؛ AVL ارتفاع را ۲ نگه داشت.

تشبیه چرخش: یک آویز (موبایل) را که از یک نخ آویزان است در نظر بگیر. اگر یک طرفش سنگین شد، آویز را از گرهٔ **وسطی** آویزان می‌کنی؛ گرهٔ وسط بالا می‌رود و دو گرهٔ دیگر دو طرفش می‌نشینند. ترتیب چپ به راست (یعنی خاصیت BST) عوض نمی‌شود.

- **AVL:** در هر گره، اختلاف ارتفاع زیردرخت چپ و راست حداکثر ۱ است. ارتفاع $\approx 1.44 \log_2 n$.
- **حداقل گره در AVL با ارتفاع h:** $N(h) = N(h-1) + N(h-2) + 1$ ، با مقادیر $N(0)=1$، $N(1)=2$، $N(2)=4$، $N(3)=7$، $N(4)=12$، $N(5)=20$.
- **چرخش‌ها:** حالت‌های LL و RR با یک چرخش ساده درست می‌شوند؛ LR و RL با چرخش دوگانه. **درج** در AVL حداکثر یک چرخش (ساده یا دوگانه) لازم دارد. **حذف** ممکن است تا $O(\log n)$ چرخش بخواهد.

✏️ **مثال برای هر مورد**

- **حداقل گره:** $N(2) = N(1) + N(0) + 1 = 2 + 1 + 1 = 4$: ریشه با یک زیردرخت چپ به ارتفاع ۱ (دو گره) و یک برگ راست.
- **RR (چرخش چپ ساده):** درج $10, 20, 30$: ۱۰ نامتوازن می‌شود (زیردرخت راستش ارتفاع ۱ و چپش خالی). با یک چرخش چپ، ریشه ۲۰ با فرزندان ۱۰ و ۳۰ می‌شود.
- **LR (دوگانه):** درج $30, 10, 20$: ۱۰ فرزند چپ ۳۰ است و ۲۰ فرزند راست ۱۰ (شکل زیگزاگ). اول چرخش چپ روی ۱۰ (حالا LL شده)، بعد چرخش راست روی ۳۰ ⇐ ریشه ۲۰ با فرزندان ۱۰ و ۳۰.

ترفند «ستون راست» برای شمردن حداقل چرخش

ستون راست یعنی مسیری که از ریشه فقط به فرزند راست می‌رود. چرخش چپ روی یک گرهٔ این ستون، طولش را **دقیقاً یکی** کم می‌کند و چرخش‌های دیگر حداکثر یکی تغییرش می‌دهند. پس: **حداقل چرخش ≥ (طول ستون راست اول) − (طول ستون راست هدف).**

✏️ **مثال**

زنجیر راست $1 \to 2 \to 3$ ستون راست ۳ گره‌ای دارد. هدف «ریشه ۲ با فرزندان ۱ و ۳» ستون راست $2 \to 3$ با ۲ گره دارد. کران $3 - 2 = 1$ است، و واقعاً یک چرخش چپ روی ۱ کافی است.

درس ۴ هیپ (Heap)

از صفر: اورژانس بیمارستان

در اورژانس بیماران به ترتیب ورود ویزیت نمی‌شوند؛ هر بار **بدحال‌ترین** نفر اول می‌رود. ساختاری که همیشه «بزرگ‌ترین» را سریع بدهد و درج سریع هم داشته باشد، **صف اولویت** است و هیپ بهترین پیاده‌سازی آن است.

هیپ بیشینه دو قانون دارد:

۱. **شکل:** درخت کامل است (سطح‌ها از بالا پر و سطح آخر از چپ).

۲. **ترتیب:** هر والد از فرزندانش بزرگ‌تر یا مساوی است. پس بزرگ‌ترین همیشه ریشه است. (برادرها با هم ترتیبی ندارند، برخلاف BST.)

چرا در آرایه؟ چون درخت کامل است، گره‌ها را سطر به سطر در آرایه می‌نویسیم و اشاره‌گری لازم نیست: گرهٔ i فرزندانش را در $2i$ و $2i+1$ دارد.

درج (بالا رفتن): عضو تازه را ته آرایه بگذار؛ تا وقتی از والدش بزرگ‌تر است، جایشان را عوض کن، مثل حبابی که بالا می‌آید.
حذف بیشینه (پایین رفتن): ریشه را بردار، آخرین عضو را جایش بگذار و تا وقتی از فرزند بزرگ‌ترش کوچک‌تر است، با او عوض کن. هر دو حداکثر به اندازهٔ ارتفاع، یعنی $O(\log n)$ ، قدم دارند.

- درخت **کامل** که در هر گره، کلید والد ≥ کلید فرزندان است (هیپ بیشینه). در آرایه با اندیس از ۱: والد $\lfloor i/2 \rfloor$ و فرزندان $2i$ و $2i+1$.

عمل	زمان
یافتن بیشینه	$O(1)$
درج (بالا بردن)	$O(\log n)$
حذف بیشینه (آخرین عضو را در ریشه بگذار و پایین ببر)	$O(\log n)$
افزایش کلید	$O(\log n)$
ساخت هیپ از n عدد (build-heap)	**$O(n)$** ، نه $O(n \log n)$
جستجوی یک کلید دلخواه	$O(n)$ (هیپ برای جستجو نیست)
کمینه در هیپ بیشینه	بین برگ‌ها، یعنی $\lceil n/2 \rceil$ مقایسه

✏️ مثال برای هر سطر جدول

هیپ بیشینهٔ $[9, 7, 8, 3, 5, 6]$ (اندیس از ۱).

- **اندیس‌ها:** والد اندیس ۵ (کلید ۵) اندیس $\lfloor 5/2 \rfloor = 2$ (کلید ۷) است. فرزندان اندیس ۳ (کلید ۸) اندیس‌های ۶ و ۷ هستند.
- **بیشینه:** $A[1] = 9$.
- **درج ۱۰:** در اندیس ۷ می‌نشیند. از والدش ۸ بزرگ‌تر است، پس جابه‌جا می‌شوند؛ بعد از ۹ هم بزرگ‌تر است، پس به ریشه می‌رسد: $[10, 7, 9, 3, 5, 6, 8]$.
- **build-heap** روی $[3, 5, 9, 6, 8, 20, 10]$: از اندیس $\lfloor 7/2 \rfloor = 3$ به عقب، هر گره را پایین ببر.
  - اندیس ۳: ۹ با ۲۰ جابه‌جا ⇐ $[3, 5, 20, 6, 8, 9, 10]$
  - اندیس ۲: ۵ با ۸ ⇐ $[3, 8, 20, 6, 5, 9, 10]$
  - اندیس ۱: ۳ با ۲۰، بعد با ۱۰ ⇐ $[20, 8, 10, 6, 5, 9, 3]$
  - نصف گره‌ها برگ‌اند و کاری ندارند؛ یک‌چهارم یک سطح پایین می‌روند و الی آخر. جمع $O(n)$ است.
- **کمینه:** در $[9, 7, 8, 3, 5, 6]$ برگ‌ها اندیس ۴ تا ۶ هستند (3, 5, 6) و کمینه، ۳، بینشان است.

✏️ مثال (آرایه و درخت کنار هم)

هیپ $[9, 7, 8, 3, 5, 6]$:

اندیس	۱	۲	۳	۴	۵	۶
کلید	۹	۷	۸	۳	۵	۶
والد ($\lfloor i/2 \rfloor$)	—	۱	۱	۲	۲	۳

یعنی ۹ ریشه است، ۷ و ۸ فرزندانش، ۳ و ۵ فرزندان ۷، و ۶ فرزند چپ ۸. هر والد از فرزندانش بزرگ‌تر است ✓.

✏️ مثال (حذف بیشینه، قدم‌به‌قدم)

از همان هیپ.

۱. ۹ را بردار. آخرین عضو (۶) را در ریشه بگذار ⇐ $[6, 7, 8, 3, 5]$.

۲. ۶ با فرزندانش (۷ و ۸) مقایسه می‌شود. بزرگ‌ترین فرزند ۸ است و $6 < 8$ ⇐ جابه‌جا ⇐ $[8, 7, 6, 3, 5]$.

۳. ۶ حالا در اندیس ۳ است و فرزندی ندارد (اندیس ۶ خالی است) ⇐ تمام.

نتیجه $[8, 7, 6, 3, 5]$ و بیشینهٔ جدید ۸ است. ⚠️ همیشه با **بزرگ‌ترِ** دو فرزند عوض کن؛ اگر با ۷ عوض می‌کردی، ۷ والد ۸ می‌شد و قانون هیپ می‌شکست.

نکته‌های تستی

- kامین بزرگ‌ترین عضو در عمق حداکثر $k-1$ است. مثلاً دومین بزرگ‌ترین حتماً فرزند ریشه است.
- مجموعهٔ گره‌هایی که کلیدشان از یک مقدار بزرگ‌تر است، **یک زیردرخت همبند شامل ریشه** است (چون والد هر کدام هم بزرگ‌تر است).
- برای اینکه t برگ در عمق d درون چنین زیردرختی باشند، حداقل $\Sigma \lceil t/2^{(d-i)} \rceil$ (روی همهٔ سطح‌های i) گره لازم است.
- **کجا هیپ لازم است:** پریم، دایکسترا، هافمن (صف اولویت). **کراسکال** به مرتب‌سازی یال‌ها و Union-Find نیاز دارد، نه هیپ.
- **هیپ dتایی:** ارتفاع $\log_d n$. درج سریع‌تر است، ولی حذف بیشینه $O(d \cdot \log_d n)$ هزینه دارد.

> ✏️ **مثال برای هر نکته**
>
> - در $[9, 7, 8, 3, 5, 6]$ ، دومین بزرگ‌ترین (۸) و سومین (۷) هر دو در عمق ۱ هستند.
> - کلیدهای بزرگ‌تر از ۵: $\{9, 7, 8, 6\}$ در اندیس‌های ۱، ۲، ۳، ۶؛ یک زیردرخت همبند شامل ریشه.
> - **فرمول برگ‌ها:** برای $t = 2$ برگ در عمق $d = 2$: $\lceil 2/4 \rceil + \lceil 2/2 \rceil + 2 = 1 + 1 + 2 = 4$ گره (ریشه، یک فرزند، و دو نوهٔ زیر آن فرزند).
> - **dتایی:** با $n = 1024$ ، هیپ ۴تایی ارتفاع ۵ دارد و دودویی ارتفاع ۱۰. حذف بیشینه در ۴تایی در هر سطح ۴ فرزند را مقایسه می‌کند، پس حدود $4 \times 5 = 20$ مقایسه؛ همان‌قدر که دودویی (2×10).

درس ۵ انتخاب ساختار ترکیبی

اگر چند عمل با هم خواسته شده، برای هر عمل ببین کدام ساختار جوابش را می‌دهد و ترکیبشان کن:

- **درخت متوازن (AVL یا قرمز-سیاه)** با اندازهٔ زیردرخت: درج، حذف، min و max، میانه، رتبه، همه $O(\log n)$.
- **جدول درهم‌سازی:** پیدا کردن عنصر با کلید در امید $O(1)$.
- **هیپ:** فقط بیشینه یا کمینه سریع است، میانه نه.

این درس به چه سؤالی جواب می‌دهد؟

تست‌ها می‌گویند «ساختاری می‌خواهیم که این ۳ یا ۴ عمل را سریع انجام دهد». هیچ ساختار تکی همه را ندارد؛ پس برای هر عمل بهترین ساختار را پیدا کن و اگر لازم بود دو ساختار را **با هم** نگه دار (هر درج در هر دو انجام شود).

✏️ مثال (ترکیب)

- **«درج، حذف، میانه و جستجو با کلید»:** AVL با اندازهٔ زیردرخت (برای میانه) + جدول درهم‌سازی (برای جستجوی سریع).
- **میانهٔ پویا با دو هیپ:** نیمهٔ کوچک‌تر در یک هیپ **بیشینه**، نیمهٔ بزرگ‌تر در یک هیپ **کمینه**. بعد از درج 5, 2, 8, 1, 9: پایینی $\{1, 2, 5\}$ (بالایش ۵) و بالایی $\{8, 9\}$. میانه بالای هیپ پایینی است: **۵** در $O(1)$، و هر درج $O(\log n)$.

✍️ تمرین فصل ۴

اول خودت حل کن، بعد پاسخ‌نامه

۱. پیش‌ترتیب `A B D E C F` و میان‌ترتیب `D B E A F C` است. پس‌ترتیب چیست؟

۲. حداکثر گره در درخت دودویی با ارتفاع ۴؟ حداقل گره در AVL با ارتفاع ۴؟

۳. کلیدهای ۳۰، ۲۰ و ۱۰ را به‌ترتیب در AVL درج کن. ریشهٔ نهایی؟

۴. هیپ بیشینهٔ `[50, 30, 40, 10, 20, 35]`. بعد از حذف بیشینه آرایه چه می‌شود؟

۵. در هیپ بیشینه با کلیدهای متمایز، سومین بزرگ‌ترین عضو در چه عمق‌هایی ممکن است باشد؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۷۰**

📝 آزمون فصل ۴

سؤال‌های واقعی کنکور ارشد کامپیوتر

۷ سؤال | ⏱️ زمان: ۲۰ دقیقه برای ۷ سؤال | 🔑 کلید و راه‌حل: **صفحهٔ ۷۰**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۱ · سؤال ۶۷ — غلط یا حدسی؟ ☐

فرض کنید یک درخت دودویی جستجو بر روی n عدد حقیقی متمایز با ارتفاع $O(\log n)$ در اختیار داریم. چه تعداد از پرسمان‌های زیر را بدون پیش‌پردازش و اطلاعات اضافی می‌توان در $O(\log n)$ پاسخ داد؟ (در هر گره صرفاً یک کلید و دو اشاره‌گر به فرزندان نگه‌داشته شده است.)

- محاسبهٔ کوچک‌ترین عدد
- محاسبهٔ میانه
- تعیین آنکه آیا عدد داده‌شدهٔ x در درخت وجود دارد
- محاسبهٔ مرتبهٔ عدد داده‌شدهٔ x در بین n عدد ذخیره‌شده در درخت

۱) صفر ۲) ۳ ۳) ۲ ۴) ۱

۲ ارشد ۱۴۰۰ · سؤال ۶۲ — غلط یا حدسی؟ ☐

اعداد $1, 2, \ldots, 10$ را به‌ترتیب از چپ به راست در یک درخت دودویی جستجو که در اول کار تهی است درج می‌کنیم. بعد از درج همهٔ عناصر می‌خواهیم درخت حاصل را به درخت دودویی جستجو با ارتفاع ۳ تبدیل کنیم. برای این کار تنها مجاز به استفاده از عمل چرخش (به چپ یا راست) هستیم. با حداقل چندبار چرخش می‌توان این کار را انجام داد؟ (منظور از چرخش همان عملیات متعارفی است که برای متوازن‌سازی درخت‌های دودویی جستجو استفاده می‌شود.)

۱) ۶ ۲) ۷ ۳) ۸ ۴) ۹

۳ ارشد ۱۳۹۹ · سؤال ۸۸ — غلط یا حدسی؟ ☐

اعداد صحیح $x_1, \ldots, x_n$ را در یک درخت دودویی جستجو با ارتفاع h ذخیره کرده‌ایم. فرض کنید هزینهٔ جستجوی x_i (تعداد مقایسه‌های لازم در درخت برای پیدا کردن x_i) برابر c_i باشد. می‌دانیم $\Sigma\, c_i = O(n \log n)$ است. کدام گزینه درست است؟

۱) $h = \Omega(\sqrt{n})$ ۲) $h = O(\log n)$
۳) $h = O(\sqrt{(n \log n)})$ ۴) می‌توان مثالی زد که $h = \Omega(n)$ باشد.

۴ ارشد ۱۳۹۸ · سؤال ۵۲ — غلط یا حدسی؟

فرض کنید یک درخت دودویی جستجو با n گره داریم. به ازای گرهٔ u از این درخت، وزن آن را تعداد گره‌ها در زیردرخت به ریشهٔ u (شامل u) در نظر بگیرید. می‌دانیم در درخت فوق به ازای هر گرهٔ داخلی u، نسبت وزن فرزند چپ و فرزند راست حداقل ۰٫۵ و حداکثر ۲ است. بهترین کران بالا برای زمان جستجو در این درخت در بدترین حالت، در بین گزینه‌ها کدام است؟ (مبنای لگاریتم‌ها ۲ است.)

۱) $1.5 \log n$ ۲) $2.5 \log n$ ۳) $2 \log n$ ۴) $3 \log n$

۵ ارشد ۱۴۰۴ · سؤال ۶۵ — غلط یا حدسی؟

فرض کنید یک ساختار دادهٔ جدید به نام لیست اولویت‌دار پویا (Dynamic Priority List) دارید. این ساختار داده از لیستی از گره‌ها تشکیل شده است که هر گره دارای یک مقدار کلید و یک اولویت است. عمل‌های زیر بر روی این ساختار داده تعریف شده‌اند:

- `insert(x, p)` : افزودن یک عنصر با کلید x و اولویت p به لیست
- `DeleteMax()` : حذف گره‌ای که دارای بیشترین اولویت است. اگر چند گره دارای اولویت یکسان باشند، اولین گره از سمت چپ حذف می‌شود.
- `FindMedian()` : یافتن مقدار میانه از بین تمامی کلیدهای موجود در لیست
- `IncreasePriority(x, k)` : افزایش اولویت عنصر با کلید x به اندازهٔ k

هرکدام از این عملیات باید در کمترین پیچیدگی زمانی ممکن انجام شوند. حال، کدام‌یک از روش‌های زیر مناسب‌ترین ساختار دادهٔ ترکیبی برای پیاده‌سازی لیست اولویت‌دار پویا است، به گونه‌ای که همهٔ عملیات با پیچیدگی بهینه انجام شوند؟

۱) استفاده از یک پشته و یک صف به‌صورت هم‌زمان برای مدیریت ترتیب ورود و خروج عناصر و اولویت آن‌ها

۲) ترکیب یک درخت فیبوناچی (Fibonacci Heap) و یک لیست پیوندی ساده برای مدیریت عناصر و اولویت‌ها

۳) ترکیب یک هرم دوتایی (Double-ended Heap) برای مدیریت اولویت‌ها و یک ساختار دادهٔ مرتب برای دسترسی سریع به میانه

۴) ترکیب یک درخت جستجوی دودویی متوازن (Balanced Binary Search Tree) برای ذخیرهٔ اولویت‌ها و یک جدول درهم‌سازی (Hash Table) برای دسترسی سریع به عناصر

۶ ارشد ۱۴۰۲ · سؤال ۶۱ — غلط یا حدسی؟

برای پیاده‌سازی مؤثر کدام‌یک از الگوریتم‌های زیر، ساختمان دادهٔ heap لازم **نیست**؟

۱) پریم ۲) دایکسترا ۳) هافمن ۴) کراسکال

۷ ارشد ۱۴۰۱ · سؤال ۶۴ — غلط یا حدسی؟ ☐

اعداد یک تا ۱۲۷ در یک هرم بیشینه که به‌صورت یک درخت دودویی کامل با ارتفاع ۶ پیاده‌سازی شده، قرار گرفته‌اند. حداکثر تعداد برگ با مقدار بیشتر از ۱۰۰ در این درخت چقدر می‌تواند باشد؟

۱) ۲۰ ۲) ۱۲ ۳) ۱۱ ۴) ۲

🗂️ کارت خلاصهٔ فصل ۴

برای مرور سریع، شب قبل از آزمون

- ✔ برگ‌ها = گره‌های دوفرزندی + ۱. حداکثر گره $2^{(h+1)} - 1$. پیش یا پس‌ترتیب + میان‌ترتیب یعنی بازسازی یکتا.
- ✔ BST: همهٔ عمل‌ها $O(h)$. درج مرتب یعنی زنجیر. میانه و رتبه به اندازهٔ زیردرخت نیاز دارند.
- ✔ Σ هزینه $= S \Leftarrow h = O(\sqrt{S})$. وزن‌متوازن با کسر $c \Leftarrow h \le \log_{(1/c)} n$.
- ✔ AVL: $N(h) = N(h-1) + N(h-2) + 1$ ($1, 2, 4, 7, 12, 20$). درج حداکثر یک چرخش دارد. حداقل چرخش ≤ اختلاف طول ستون راست.
- ✔ هیپ: ساخت $O(n)$؛ درج و حذف $O(\log n)$؛ جستجو $O(n)$؛ کمینه بین برگ‌ها؛ ام بزرگ در عمق k ≤ $k-1$؛ گره‌های «بزرگ‌تر از x» زیردرخت همبند شامل ریشه‌اند.
- ✔ کراسکال هیپ نمی‌خواهد. میانه + اولویت + دسترسی با کلید: BST متوازن + جدول درهم‌سازی.

پاسخ‌نامهٔ فصل ۴

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۴

برگشت ← صفحهٔ ۶۶

۱. $D\,E\,B\,F\,C\,A$

۲. $2^5 - 1 = 31$ و $N(4) = 12$

۳. حالت LL است و یک چرخش راست لازم دارد. ریشهٔ نهایی **۲۰** است، با فرزندان ۱۰ و ۳۰.

۴. ۳۵ به ریشه می‌رود: $[35, 30, 40, 10, 20]$. بعد با ۴۰ جابه‌جا می‌شود: $[40, 30, 35, 10, 20]$.

۵. عمق ۱ یا ۲.

آزمون فصل ۴ – کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۶۷

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷
۳	۱	۳	۳	۴	۴	۲

۱ — **گزینهٔ ۳ (دو تا)**

کوچک‌ترین عدد و جستجو در $O(h)$ جواب دارند. میانه و رتبه به اندازهٔ زیردرخت در گره‌ها نیاز دارند.

۲ — **گزینهٔ ۱ (۶)**

- **کران پایین:** درخت اول زنجیر راست است و ستون راستش ۱۰ گره دارد. درخت با ارتفاع ۳ حداکثر ۴ گره روی ستون راست دارد. هر چرخش طول ستون راست را حداکثر یکی تغییر می‌دهد، پس حداقل $10 - 4 = 6$ چرخش لازم است.
- **ساخت با ۶ چرخش:** چرخش چپ روی ۱، بعد ۳، بعد ۲ زیردرخت $2(1,3)$ را زیر ۴ می‌سازد. چرخش چپ روی ۵، بعد ۷، بعد ۶ زیردرخت $6(5,7)$ را زیر ۸ می‌سازد. درخت نهایی: ریشه ۴، چپ $2(1,3)$ ، راست 8 با چپ $6(5,7)$ و راست $9 \to 10$. ارتفاعش ۳ است.

۳ — **گزینهٔ ۳**

مسیر به طول h هزینه‌ای حدود $h^2/2$ دارد که باید $\le \Sigma c_i = O(n \log n)$ باشد. پس $h = O(\sqrt{(n \log n)})$.

- گزینهٔ ۱ غلط است: درخت متوازن $h = \log n$ دارد.
- گزینهٔ ۲ غلط است: یک مسیر به طول $\sqrt{(n \log n)}$ کنار یک درخت متوازن، جمع را $O(n \log n)$ نگه می‌دارد.
- گزینهٔ ۴ غلط است: با $h = \Omega(n)$ جمع $\Omega(n^2)$ می‌شد.

۴ **گزینهٔ ۳**

با نسبت ۰٫۵ تا ۲ بین دو فرزند، هر فرزند حداکثر $2/3$ وزن والد را دارد. پس عمق حداکثر $\log_{(3/2)} n \approx 1.71 \log n$ است. $1.5 \log n$ کران درستی نیست، و بین بقیه کوچک‌ترین کران درست $2 \log n$ است.

۵ **گزینهٔ ۴**

درخت متوازن (با اندازهٔ زیردرخت) درج، حذف بیشینه، میانه و تغییر اولویت را در $O(\log n)$ انجام می‌دهد، و جدول درهم‌سازی عنصر x را در امید $O(1)$ پیدا می‌کند. هیپ میانه را نمی‌دهد و پشته و صف اولویت را مدیریت نمی‌کنند.

۶ **گزینهٔ ۴**

کراسکال یال‌ها را مرتب می‌کند و Union-Find می‌خواهد. پریم و دایکسترا و هافمن صف اولویت (هیپ) لازم دارند.

۷ **گزینهٔ ۲ (۱۲)**

اعداد بزرگ‌تر از ۱۰۰ (یعنی ۲۷ عدد، از ۱۰۱ تا ۱۲۷) یک زیردرخت همبند شامل ریشه می‌سازند. برای t برگ در عمق ۶، در هر عمق d حداقل $\lceil t/2^{(6-d)} \rceil$ گره لازم است:

- برای $t = 12$: $12 + 6 + 3 + 2 + 1 + 1 + 1 = 26 \le 27$ ✓
- برای $t = 13$: $13 + 7 + 4 + 2 + 1 + 1 + 1 = 29 > 27$ ✗

پس حداکثر **۱۲** است.

فصل ۵

مرتب‌سازی ۵

۴ درس | ۱۲ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** **۱۲ سؤال** در ارشد ۱۳۹۸ تا ۱۴۰۴، فقط در ۱۴۰۴ چهار سؤال. تیپ‌ها: «بعد از بخش‌بندی، کدام عدد می‌تواند محور باشد؟»، اثر تغییر یک الگوریتم روی زمانش، کران پایین، و مرتب‌سازی ورودی‌های خاص (دوآهنگی، kمرتب، چند آرایهٔ مرتب).

⏱️ **زمان مطالعه** دو بلوک ۹۰ دقیقه‌ای + ۳۰ دقیقه آزمون فصل.

نقشهٔ فصل: ۱ جدول اصلی | ۲ مرتب‌سازی سریع (QuickSort) | ۳ کران پایین و مرتب‌سازی غیرمقایسه‌ای | ۴ ورودی‌های خاص | 📝 آزمون فصل

درس ۱ جدول اصلی

الگوریتم	بهترین	میانگین	بدترین	حافظهٔ اضافه	پایدار؟	نکته
درجی	n	n^2	n^2	$O(1)$	✅	روی ورودی تقریباً مرتب عالی است؛ زمان = n + تعداد نابه‌جایی‌ها
انتخابی	n^2	n^2	n^2	$O(1)$	❌	فقط $n-1$ جابه‌جایی
حبابی	n (با پرچم)	n^2	n^2	$O(1)$	✅	
ادغامی	$n \log n$	$n \log n$	$n \log n$	$O(n)$	✅	
سریع	$n \log n$	$n \log n$ (حدود $1.39\, n \log n$ مقایسه)	n^2	$O(\log n)$ پشته	❌	جابه‌جایی کم، در عمل سریع
هیپ	$n \log n$	$n \log n$	$n \log n$	$O(1)$	❌	
شمارشی	$n + k$			$O(n + k)$	✅	**مقایسه‌ای نیست**
مبنایی	$d(n + b)$				✅	

✏️ مثال برای هر سطر جدول (ردگیری روی آرایهٔ کوچک)

- **درجی** روی [3, 1, 2] : نابه‌جایی‌ها (3,1) و (3,2) هستند، پس ۲ جابه‌جایی. ۱ را درج کن ⇐ [1, 3, 2] ؛ ۲ را درج کن ⇐ [1, 2, 3] .
- **انتخابی** روی [3, 1, 2] : کمینه (۱) با خانهٔ اول عوض ⇐ [1, 3, 2] ؛ کمینهٔ بقیه (۲) با خانهٔ دوم عوض ⇐ [1, 2, 3]. یعنی $n - 1 = 2$ جابه‌جایی. **ناپایداری:** در $[2^a, 2^b, 1]$ ، ۱ با 2^a عوض می‌شود ⇐ $[1, 2^b, 2^a]$ و ترتیب دو ۲ به هم خورد.
- **حبابی** روی [3, 1, 2] : دور اول ⇐ [1, 3, 2] ⇐ [1, 2, 3] ؛ دور دوم هیچ جابه‌جایی ندارد و پرچم متوقفش می‌کند.
- **ادغامی** روی [3, 1, 2, 4] : تقسیم به [3, 1] و [2, 4] ، مرتب‌شده [1, 3] و [2, 4] ، ادغام ⇐ 1, 2, 3, 4 .
- **هیپ:** build-heap و بعد n بار «بیشینه را به آخر آرایه ببر و هیپ را یکی کوچک کن».
- **شمارشی** روی [3, 1, 3, 0] با مقادیر ۰ تا ۳: شمارنده‌ها [1, 1, 0, 2] ⇐ خروجی 0, 1, 3, 3 . هیچ دو عددی با هم مقایسه نشدند.
- **مبنایی** روی [170, 45, 75, 90] (مبنای ۱۰، رقم به رقم از راست، هر بار پایدار):
  - یکان: 170, 90, 45, 75
  - دهگان: 45, 170, 75, 90
  - صدگان: 45, 75, 90, 170 ✓ ($d = 3$ دور)

⚠️ درجی با جستجوی دودویی

تعداد **مقایسه‌ها** $O(n \log n)$ می‌شود، ولی **جابه‌جایی‌ها** همچنان $O(n^2)$ هستند. پس زمان کل $O(n^2)$ می‌ماند.

✏️ مثال

درج ۵ در [1, 2, 3, 4, 6, 7, 8] : جستجوی دودویی با ۳ مقایسه جایش را پیدا می‌کند، ولی ۶، ۷ و ۸ باید یکی‌یکی جابه‌جا شوند. در بدترین حالت (درج کوچک‌ترین)، همهٔ i عضو قبلی جابه‌جا می‌شوند.

درس ۲ مرتب‌سازی سریع (QuickSort)

- **بخش‌بندی (partition):** بعد از آن، محور سر جای نهایی‌اش است: همه چپ‌ها کوچک‌ترند و همه راست‌ها بزرگ‌تر.
- **تست «کدام عدد می‌تواند محور باشد؟»:** عددی که **همهٔ** عددهای سمت چپش کوچک‌تر و **همهٔ** عددهای سمت راستش بزرگ‌تر باشند.
- **رابطهٔ میانگین:** $T(n) = O(n) + (1/n) \cdot \Sigma_i (T(i) + T(n-1-i))$ ، با جواب $\Theta(n \log n)$.
- **بدترین حالت** $T(n) = T(n-1) + O(n) = \Theta(n^2)$ است (مثلاً آرایهٔ مرتب با محور اول).
- **محور = میانه با الگوریتم انتخاب خطی** ⇐ بدترین حالت $O(n \log n)$.

- **محور = میانهٔ k عضو** ⇐ تضمین می‌شود حداقل $(k-1)/2$ عضو در هر طرف باشند. پس بدترین حالت $T(n) = T(n - k/2) + O(n)$ است. اگر k کوچک باشد (مثلاً $2 \log n + 1$)، جواب هنوز تقریباً n^2 است.
- **سریع در برابر ادغامی:** سریع در میانگین **مقایسهٔ بیشتر** ولی **جابه‌جایی کمتری** دارد (درجا است).

✏️ **مثال برای هر مورد**

- **بخش‌بندی (لوموتو، محور = آخرین):** $[4, 7, 2, 6, 1, 5]$ با محور ۵ ⇐ $[4, 2, 1, 5, 7, 6]$. ۵ سر جای نهایی‌اش (خانهٔ چهارم) است.
- **کدام می‌تواند محور باشد؟** در $[1, 3, 2, 4, 6, 5]$: عدد ۱ (چپش خالی، راستش همه بزرگ‌تر) و عدد ۴ (چپش $1, 3, 2$ کوچک‌تر و راستش $6, 5$ بزرگ‌تر). ۳ نمی‌تواند باشد، چون ۲ بعدش آمده.
- **بدترین حالت:** $[1, 2, 3, 4]$ با محور اول: بخش‌ها هر بار اندازهٔ ۰ و $n - 1$ دارند؛ مقایسه‌ها $= 6 = 3 + 2 + 1$ $n(n-1)/2$.
- **میانهٔ سه‌تایی (k = ۳):** فقط تضمین می‌کند حداقل ۱ عضو در هر طرف باشد: $T(n) = T(n-2) + O(n)$ که هنوز $\Theta(n^2)$ است.

درس ۳ کران پایین و مرتب‌سازی غیرمقایسه‌ای

- **درخت تصمیم:** هر الگوریتم مقایسه‌ای n! برگ دارد، پس عمق (بدترین تعداد مقایسه) $\geq \lceil \log_2(n!) \rceil = \Omega(n \log n)$.
- درخت دودویی با m برگ، عمقی $\geq \lceil \log_2 m \rceil$ دارد.
- شمارشی و مبنایی مقایسه‌ای **نیستند**، پس کران $\Omega(n \log n)$ شامل آن‌ها نمی‌شود.

✏️ **مثال**

- $n = 3$: $3! = 6$ برگ ⇐ عمق $\geq \lceil \log_2 6 \rceil = 3$. یعنی هیچ الگوریتمی ۳ عدد را در بدترین حالت با ۲ مقایسه مرتب نمی‌کند (۲ مقایسه حداکثر ۴ نتیجه دارد، ولی ۶ ترتیب داریم).
- $n = 4$: $\lceil \log_2 24 \rceil = 5$ مقایسه.

درس ۴ ورودی‌های خاص

ورودی	بهترین زمان	روش
kمرتب (هر عضو حداکثر k خانه با جای نهایی‌اش فاصله دارد)	$O(n \log k)$	هیپ کمینه با $k+1$ عضو
دوآهنگی (صعودی و بعد نزولی، حتی چرخشی)	$O(n)$	بیشینه را پیدا کن و دو بخش مرتب را ادغام کن
چند آرایهٔ مرتب (حتی یکی نزولی)	$O(n)$	آرایهٔ نزولی را وارون کن و با ادغام، اجتماع یا اشتراک بگیر

ورودی	بهترین زمان	روش
ادغام m‌راهه با m ثابت	$O(n \log n)$	$T(n) = m \cdot T(n/m) + O(n)$
شیرسورت روی ماتریس $n \times n$ (سطرها مارپیچی، بعد ستون‌ها)	$\lceil \log_2 n \rceil + 1$ فاز	

✏️ مثال برای هر سطر جدول

- **kمرتب (k = ۱)** روی $[2, 1, 4, 3, 6, 5]$: هیپ ۲ عضوی $\{2, 1\}$ ⇐ ۱ بیرون؛ ۴ وارد ⇐ ۲ بیرون؛ ۳ وارد ⇐ ۳ بیرون؛ ... ⇐ $1, 2, 3, 4, 5, 6$.
- **دوآهنگی** $[1, 4, 7, 5, 2]$: بیشینه ۷. بخش صعودی $[1, 4, 7]$ و بخش نزولی $[5, 2]$ که وارون می‌شود $[2, 5]$. ادغام ⇐ $1, 2, 4, 5, 7$.
- **دو آرایه، یکی نزولی:** $[1, 3, 5]$ و $[6, 4, 2]$. دومی را وارون کن ⇐ $[2, 4, 6]$ ، ادغام ⇐ $1, \ldots, 6$ در $O(n)$.
- **ادغام ۳راهه:** $T(n) = 3T(n/3) + O(n)$ با $c = \log_3 3 = 1$ ⇐ حالت ۲: $\Theta(n \log n)$ ؛ همان مرتبهٔ ادغامی دوراهه.
- **شیرسورت ۴×۴:** $\lceil \log_2 4 \rceil + 1 = 3$ فاز.

✍️ تمرین فصل ۵

اول خودت حل کن، بعد پاسخ‌نامه

۱. بعد از بخش‌بندی، آرایه $2, 1, 3, 6, 5, 4, 7$ شده. کدام اعداد می‌توانند محور باشند؟

۲. چرا درجی روی آرایهٔ مرتب $O(n)$ است؟

۳. کمترین تعداد مقایسه در بدترین حالت برای مرتب‌سازی ۵ عدد، حداقل چند است؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۸۰**

📝 آزمون فصل ۵

سؤال‌های واقعی کنکور ارشد کامپیوتر

۱۲ سؤال | ⏱️ زمان: ۳۰ دقیقه برای ۱۲ سؤال | 🔑 کلید و راه‌حل: **صفحهٔ ۸۰**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۴ · سؤال ۵۷ — غلط یا حدسی؟ ☐

دنبالهٔ $A[1..n]$ یک دنبالهٔ دوآهنگی (Bitonic) است، بدین معنی که هرگاه عناصر دنباله را به‌ترتیب دور یک دایره بنشانیم، یک عنصر یافت می‌شود که دنبالهٔ عناصر بعدی که از آن شروع می‌شود ابتدا صعودی و سپس نزولی هستند. بهترین زمان برای مرتب‌سازی یک دنبالهٔ دوآهنگی، کدام مورد است؟

۱) $O(n \log n)$ ۲) $O(n\sqrt{n})$ ۳) $O(n^2)$ ۴) $O(n)$

۲ ارشد ۱۴۰۴ · سؤال ۶۰ — غلط یا حدسی؟ ☐

در الگوریتم مرتب‌سازی Insertion Sort، اگر برای جستجوی مکان مناسب عنصر جاری به جای جستجوی خطی از جستجوی دودویی استفاده کنیم، کدام مورد زیر درخصوص پیچیدگی زمانی آن در بدترین حالت درست است؟

۱) به $O(n \log n)$ کاهش می‌یابد.
۲) همچنان $O(n^2)$ باقی می‌ماند.
۳) به $O(\log n)$ کاهش می‌یابد.
۴) به $O(n)$ کاهش می‌یابد.

۳ ارشد ۱۴۰۴ · سؤال ۶۳ — غلط یا حدسی؟ ☐

چند گزاره از گزاره‌های زیر درست است؟

- هر الگوریتم قطعی که n کلید متمایز را فقط با مقایسهٔ کلیدها مرتب‌سازی می‌کند، باید در بدترین حالت حداقل $\lceil \log(n!) \rceil$ مقایسهٔ کلیدها را انجام دهد.
- اگر m تعداد برگ‌ها در یک درخت دودویی و d عمق آن باشد آنگاه $d \geq \lceil \log m \rceil$.
- الگوریتم مرتب‌سازی شمارشی یک الگوریتم مبتنی‌بر مقایسه است که در بدترین وضعیت از مرتبهٔ $O(n)$ است.

۱) صفر ۲) ۱ ۳) ۲ ۴) ۳

۴ ارشد ۱۴۰۴ · سؤال ۶۶ — غلط یا حدسی؟

در دو الگوریتم مرتب‌سازی سریع (Quick Sort) و مرتب‌سازی ادغامی (Merge Sort)، تعداد مقایسه‌ها و جابه‌جایی‌ها متفاوت است. کدام گزارهٔ زیر، دربارهٔ این تفاوت‌ها درست است؟

۱) همواره، تعداد مقایسه‌ها در هر دو الگوریتم برابر است، اما مرتب‌سازی سریع، تعداد جابه‌جایی‌های بیشتری دارد.

۲) همواره، مرتب‌سازی ادغامی، تعداد مقایسه‌های کمتری نسبت به مرتب‌سازی سریع دارد و تعداد جابه‌جایی‌های آن نیز کمتر است.

۳) در حالت میانگین، مرتب‌سازی سریع، تعداد مقایسه‌های بیشتری نسبت به مرتب‌سازی ادغامی دارد و تعداد جابه‌جایی‌های آن کمتر است.

۴) در حالت میانگین، مرتب‌سازی سریع، تعداد مقایسه‌های برابری با مرتب‌سازی ادغامی دارد، اما مقدار جابه‌جایی‌های آن بیشتر است.

۵ ارشد ۱۴۰۳ · سؤال ۵۷ — غلط یا حدسی؟

آرایهٔ A به طول n را kمرتب می‌کنیم هرگاه برای هر i که $k \le i \le n-k$ داشته باشیم $A[i-k] \le A[i] \le A[i+k]$، یعنی آرایهٔ A به k لیست مرتب که هر کدام تقریباً n/k عنصر دارند افراز می‌شود. فرض کنید A یک آرایهٔ kمرتب به طول n باشد. سریع‌ترین الگوریتم برای تبدیل این آرایه به یک آرایهٔ ۱مرتب، از چه مرتبهٔ زمانی است؟

۱) $O(n)$ ۲) $O(nk)$ ۳) $O(k \log n)$ ۴) $O(n \log k)$

۶ ارشد ۱۴۰۳ · سؤال ۶۵ — غلط یا حدسی؟

فرض کنید که در الگوریتم مرتب‌سازی سریع برای انتخاب محور از میان n عنصر آرایه $2\lfloor \log n \rfloor + 1$ عنصر اولیه را انتخاب کنیم و با الگوریتم مرتب‌سازی درجی آن‌ها را مرتب کنیم. عنصر میانهٔ این تعداد عنصر مرتب را به‌عنوان محور انتخاب می‌کنیم. بقیهٔ الگوریتم همانند الگوریتم مرتب‌سازی عمل می‌کند. بهترین گزینه برای بدترین زمان اجرای این الگوریتم کدام است؟

۱) $T(n) = T(n - \lfloor \log n \rfloor) + O(n)$

۲) $T(n) = T(n - \lfloor \log n \rfloor) + O(\log^2 n)$

۳) $T(n) = T(n - \lfloor \log n \rfloor) + T(\lfloor \log n \rfloor) + O(n)$

۴) $T(n) = T(n - \lfloor \log n \rfloor) + T(\lfloor \log n \rfloor) + O(\log^2 n)$

۷ ارشد ۱۴۰۲ · سؤال ۶۶ — غلط یا حدسی؟

الگوریتم مرتب‌سازی ادغام (Mergesort) را درنظر بگیرید. به‌جای اینکه آرایهٔ ورودی را به دو قسمت تقریباً مساوی تقسیم کنیم، آن را به m بخش تقسیم می‌کنیم که لزوماً مساوی نیستند (m یک ثابت است). سپس به‌صورت بازگشتی الگوریتم ادغام اصلاح‌شده را روی هر قسمت اجرا می‌کنیم و m آرایهٔ مرتب‌شده را ادغام می‌کنیم. زمان اجرای الگوریتم ادغام اصلاح‌شده در حالتی که $m > 2$ کدام است؟

۱) $O(mn)$ ۲) $O(m \log n)$ ۳) $O(\log \log n)$ ۴) $O(n \log n)$

۸ ارشد ۱۴۰۱ · سؤال ۶۵ — غلط یا حدسی؟

فرض کنید سه آرایهٔ A و B و C هر کدام شامل n عدد، داده شده است. عناصر داخل آرایه‌ها متمایز هستند. آرایهٔ A و C به‌صورت صعودی و آرایهٔ B به‌صورت نزولی مرتب است. اگر بخواهیم آرایهٔ D را بسازیم که شامل عناصر $(A \cup B) \cap C$ باشد و به‌صورت صعودی مرتب شده باشد و عضو تکراری نیز نداشته باشد، بهترین پیچیدگی زمانی ممکن برای این کار کدام مورد است؟ (توجه: ممکن است عناصری، در دو یا سه آرایه باشند.)

۱) $O(n^2)$ ۲) $O(n \log n)$ ۳) $O(\log n)$ ۴) $O(n)$

۹ ارشد ۱۴۰۰ · سؤال ۶۴ — غلط یا حدسی؟

یک ماتریس ۶۴ در ۶۴ داریم که درایه‌های آن همه ۰ یا ۱ هستند. می‌خواهیم این ماتریس را به‌صورت مارپیچی مرتب کنیم، یعنی اگر در انتها، سطر اول را از چپ به راست به سطر دوم از راست به چپ و ... بچسبانیم یک آرایهٔ ۴۰۹۶ بیتی مرتب از ۰ و ۱ خواهیم داشت. ادعا می‌کنیم که الگوریتم زیر این کار را انجام می‌دهد: ۱. k بار تکرار کن. a. همهٔ سطرها را مستقلاً و در جهت خود مرتب کن. یعنی سطرهای فرد را از چپ به راست، سطرهای زوج را از راست به چپ مرتب کن. b. همهٔ ستون‌ها را از بالا به پایین مرتب کن.

کمترین مقدار k در بدترین حالت چند است؟

۱) ۷
۲) ۳۲
۳) ۶۴
۴) ممکن است به‌ازای هیچ مقدار k ماتریس لزوماً مرتب نشود.

۱۰ ارشد ۱۳۹۹ · سؤال ۸۹ — غلط یا حدسی؟

فرض کنید $T(n)$ متوسط زمان اجرای الگوریتم مرتب‌سازی سریع به ازای همهٔ جایگشت‌های ممکن ورودی از n عدد متمایز باشد. کدام رابطهٔ بازگشتی زیر درست است؟

۱) $T(n) = O(n) + \Sigma_{i=0}^{n-1} (T(i) + T(n-i))$

۲) $T(n) = O(n) + (1/n) \Sigma_{i=0}^{n-1} (T(i) + T(n-i))$

۳) $T(n) = (1/n)(O(n) + \Sigma_{i=0}^{n-1} (T(i) + T(n-i)))$

۴) $T(n) = O(n) + (1/n) \Sigma_{i=0}^{n-1} \min(T(i), T(n-i))$

۱۱ ارشد ۱۳۹۹ · سؤال ۹۶ — غلط یا حدسی؟

فرض کنید در الگوریتم مرتب‌سازی سریع پس از عمل بخش‌بندی (Partition) آرایهٔ $\langle 3, 1, 2, 4, 5, 8, 7, 6, 9 \rangle$ به‌دست آمده است. چند عدد از بین ۹ عدد در آرایه ممکن است محور این بخش‌بندی قرار گرفته باشند؟

۱) ۲ ۲) ۳ ۳) ۴ ۴) ۵

۱۲ ارشد ۱۳۹۸ · سؤال ۵۳ — غلط یا حدسی؟

در الگوریتم مرتب‌سازی سریع در هر مرحله یک عنصر دلخواه به‌عنوان محور (pivot) انتخاب و بقیهٔ اعداد با آن مقایسه می‌شود. اگر در هر مرحله میانهٔ اعداد به‌عنوان محور انتخاب شود و از الگوریتم با تعداد خطی مقایسه برای محاسبهٔ میانه استفاده شود، در بدترین حالت تعداد مقایسه‌ها در این نسخهٔ تغییریافتهٔ الگوریتم مرتب‌سازی سریع کدام است؟

۱) $O(n \log^2 n)$ ۲) $O(n \log n)$ ۳) $O(n^2)$ ۴) $O(n)$

کارت خلاصهٔ فصل ۵

برای مرور سریع، شب قبل از آزمون

- ✔ درجی: n + نابه‌جایی‌ها ؛ با جستجوی دودویی هم n^2 می‌ماند. ادغامی پایدار است و $O(n)$ حافظه می‌خواهد. سریع در بدترین حالت n^2 است، با میانهٔ خطی $n \log n$.
- ✔ محور ممکن: همهٔ چپ‌ها کوچک‌تر و همهٔ راست‌ها بزرگ‌تر.
- ✔ سریع در برابر ادغامی: مقایسهٔ بیشتر، جابه‌جایی کمتر.
- ✔ کران $\lceil \log_2 n! \rceil$ فقط برای مرتب‌سازی مقایسه‌ای است. شمارشی و مبنایی مقایسه‌ای نیستند.
- ✔ kمرتب: $n \log k$. دوآهنگی و چند آرایهٔ مرتب: $O(n)$. ادغام mراههٔ ثابت: $n \log n$. شیرسورت: $\log n + 1$ فاز.

پاسخ‌نامهٔ فصل ۵

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۵

برگشت ← صفحهٔ ۷۵

۱. **۳ و ۷.** برای ۳ همهٔ چپ‌ها کوچک‌تر و همهٔ راست‌ها بزرگ‌ترند. ۷ آخرین است و همه قبلش کوچک‌ترند. ۶ نمی‌تواند باشد چون ۵ و ۴ بعدش‌اند.

۲. هر عضو فقط یک بار مقایسه می‌شود و جابه‌جایی ندارد (تعداد نابه‌جایی‌ها صفر است).

۳. $\lceil \log_2 120 \rceil = 7$

آزمون فصل ۵ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۷۶

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰	سؤال ۱۱	سؤال ۱۲
۴	۲	۳	۳	۴	۱	۴	۴	۱	۲	۲	۲

۱ — **گزینهٔ ۴**

نقطهٔ چرخش و بیشینه را در $O(n)$ پیدا کن. یک بخش صعودی و یک بخش نزولی داریم؛ دومی را وارون کن و ادغام کن: $O(n)$.

۲ — **گزینهٔ ۲**

جستجوی دودویی فقط مقایسه‌ها را کم می‌کند. جابه‌جا کردن عضوها برای باز کردن جا همچنان $O(n^2)$ است.

۳ — **گزینهٔ ۳ (دو تا)**

گزارهٔ ۱ (کران درخت تصمیم) و گزارهٔ ۲ درست‌اند. گزارهٔ ۳ غلط است، چون شمارشی مقایسه‌ای **نیست** (و زمانش $O(n+k)$ است).

۴ — **گزینهٔ ۳**

سریع در میانگین حدود $1.39\, n \log n$ مقایسه دارد، در برابر حدود $n \log n$ برای ادغامی. ولی درجا کار می‌کند و جابه‌جایی کمتری دارد؛ ادغامی در هر سطح همه را کپی می‌کند.

۵ — **گزینهٔ ۴**

هیپ کمینه‌ای که از هر لیست یکی نگه می‌دارد (ادغام kراهه)، یا پنجرهٔ $k+1$ عضوی: $O(n \log k)$.

۶ **گزینهٔ ۱**

میانهٔ $2\lfloor \log n \rfloor + 1$ عضو حداقل $\lfloor \log n \rfloor$ عضو در هر طرف دارد. پس بدترین تقسیم $\lfloor \log n \rfloor$ و $n - \lfloor \log n \rfloor - 1$ است. هزینهٔ انتخاب محور $O(\log^2 n)$ و بخش‌بندی $O(n)$ است. جملهٔ $T(\lfloor \log n \rfloor) = O(\log^2 n)$ در $O(n)$ جذب می‌شود، پس گزینهٔ ۳ از نظر مجانبی همان گزینهٔ ۱ است؛ کلید رسمی گزینهٔ ۱ را که ساده‌تر است زده. (جواب این رابطه $\Theta(n^2/\log n)$ است.)

۷ **گزینهٔ ۴**

$T(n) = m \cdot T(n/m) + O(n)$ (با m ثابت، ادغام mراهه $O(n)$ است): $O(n \log n)$.

۸ **گزینهٔ ۴**

B را وارون کن ($O(n)$)، A و B را با ادغام یکی کن و تکراری‌ها را حذف کن، بعد با C اشتراک بگیر (ادغام دوم). همه خطی است.

۹ **گزینهٔ ۱**

شیرسورت با $\lceil \log_2 64 \rceil + 1 = 7$ فاز مرتب می‌شود.

۱۰ **گزینهٔ ۲**

محور با احتمال $1/n$ رتبهٔ iام است؛ هزینهٔ بخش‌بندی $O(n)$ است و میانگین روی همهٔ i گرفته می‌شود.

۱۱ **گزینهٔ ۲**

فقط ۴، ۵ و ۹ شرط محور را دارند: همهٔ چپ‌ها کوچک‌تر و همهٔ راست‌ها بزرگ‌تر.

۱۲ **گزینهٔ ۲**

محور میانه است، پس $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

فصل ۶

جستجو و انتخاب (kامین عضو)

۶

۴ درس | ۹ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۹ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴. تیپ‌ها: جستجوی دودویی روی ساختارهای غیرعادی (آرایهٔ چرخشی، نامتناهی، ماتریس مرتب)، حداقل مقایسه برای بیشینه و دومین بیشینه، و امید تعداد مقایسه.

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل:

۱ جستجوی دودویی و تعمیم‌هایش
۲ حداقل مقایسه برای بیشینه، کمینه و دومین بیشینه
۳ انتخاب kامین عضو
۴ امید تعداد مقایسه
📝 آزمون فصل

درس ۱ جستجوی دودویی و تعمیم‌هایش

💡 **ایدهٔ کلی**

جستجوی دودویی روی هر ویژگی **یکنوا** کار می‌کند، نه فقط روی خود اعداد.

مسئله	زمان	ترفند
آرایهٔ مرتب	$O(\log n)$	
آرایهٔ مرتب چرخشی (مثلاً 40 50 … 90 10 20 30)	$O(\log n)$	اول نقطهٔ چرخش (کوچک‌ترین) را با دودویی پیدا کن، بعد kامین عضو مستقیم با اندیس پیدا می‌شود
آرایهٔ نامتناهی (n نامعلوم)	$O(\log n)$	جستجوی نمایی: اندیس‌های ۱، ۲، ۴، ۸، … تا از x رد شوی، بعد دودویی
$A[i] = i$ در آرایهٔ مرتب از اعداد صحیح **متمایز**	$O(\log n)$	$A[i] - i$ نانزولی است
$A[i] = 3i + 2$	**نه لزوماً**	$A[i] - 3i$ یکنوا نیست ($A[i+1] \geq A[i] + 1$ ولی $3i$ سه‌تا سه‌تا زیاد می‌شود)
ماتریس n×n که سطر و ستونش مرتب است	$O(n)$	از گوشهٔ بالا-راست شروع کن: اگر بزرگ‌تر بود چپ برو، اگر کوچک‌تر بود پایین برو
عضوی که بیش از n/c بار در آرایهٔ مرتب تکرار شده	$O(c \cdot \log n)$	فقط عضوهای مکان‌های n/c، $2n/c$، … کاندیدند؛ برای هر کدام با دودویی بازه‌اش را پیدا کن

مثال برای هر سطر جدول

- **مرتب:** جستجوی ۱۱ در $[1, 3, 5, 7, 9, 11, 13]$: وسط ۷ ⇐ راست $[9, 11, 13]$ ؛ وسط ۱۱ ✓ (۲ مقایسه).
- **چرخشی:** در $40\ 50\ 60\ 70\ 80\ 90\ 10\ 20\ 30$ کوچک‌ترین در اندیس ۶ (از صفر) است. چهارمین کوچک‌ترین در اندیس $(6 + 3) \bmod 9 = 0$ است، یعنی ۴۰ ✓ (ترتیب: ۱۰، ۲۰، ۳۰، ۴۰).
- **نامتناهی:** $A[i] = 2i - 1$ (یعنی $1, 3, 5, \ldots$) و $x = 13$. اندیس‌های ۱، ۲، ۴، ۸ را ببین: $A[8] = 15 \geq 13$. حالا دودویی در اندیس‌های ۵ تا ۸: $A[6] = 11 < 13$ و بعد $A[7] = 13$ ✓.
- **A[i] = i :** $A = [-2, 0, 2, 4, 6]$ (اندیس از ۰). $A[i] - i = [-2, -1, 0, 1, 2]$ نانزولی است؛ دودویی برای ۰ ⇐ $i = 2$.
- **A[i] = 3i + 2 :** $A = [1, 2, 8, 9]$ و $A[i] - 3i = [1, -1, 2, 0]$ بالا و پایین می‌رود. جواب $i = 2$ هست (8 $= 3 \cdot 2 + 2$)، ولی از وسط آرایه نمی‌شود فهمید کدام نیمه را نگه داشت.
- **ماتریس:** در $[[1, 4, 7], [2, 5, 8], [3, 6, 9]]$ دنبال ۵ بگرد: از ۷ (بالا-راست) شروع کن؛ $7 > 5$ ⇐ چپ به ۴؛ $4 < 5$ ⇐ پایین به ۵ ✓. هر قدم یک سطر یا یک ستون را حذف می‌کند.
- **تکرار بیش از n/c:** $n = 8$ و $c = 4$ در $[1, 2, 3, 3, 3, 5, 6, 7]$. کاندیدها مکان‌های ۲، ۴، ۶، ۸ هستند (2, 3, 5, 7). عدد ۳ سه بار (بیش از ۲ بار) آمده و حتماً مکان ۴ را پوشانده؛ هر بازهٔ به طول بیش از n/c یکی از این مکان‌ها را می‌پوشاند.

جستجو روی درخت (تیپ «هر پرسش می‌گوید کدام طرف»)

در درخت، رأس **مرکزوار** (centroid) را بپرس. هر مؤلفهٔ باقی‌مانده حداکثر نصف رأس‌ها را دارد. پس با حدود $\log_2 n$ پرسش هدف پیدا می‌شود.

- **کران پایین روی مسیر:** با q پرسش حداکثر $2^{(q+1)} - 1$ رأس را می‌شود تشخیص داد.

مثال

مسیر ۷ رأسی $1 - 2 - 3 - 4 - 5 - 6 - 7$. رأس ۴ را بپرس؛ جواب «همین است»، «چپ» ($\{1, 2, 3\}$) یا «راست» ($\{5, 6, 7\}$) است. اگر بقیه ماند، وسط آن را بپرس (۲ یا ۶). با $q = 2$ پرسش، ۷ رأس تشخیص داده می‌شود: $2^3 - 1 = 7$ ✓.

درس ۲ حداقل مقایسه برای بیشینه، کمینه و دومین بیشینه

مسئله	حداقل مقایسه در بدترین حالت
بیشینه	$n - 1$
بیشینه و کمینه با هم	$\lceil 3n/2 \rceil - 2$ (جفت‌جفت مقایسه کن)
دومین بیشینه	$n + \lceil \log_2 n \rceil - 2$ (تورنمنت: دومین فقط بین کسانی است که به برنده باخته‌اند، و آن‌ها $\lceil \log n \rceil$ نفرند)

✏️ **مثال برای هر سطر جدول ($n = 8$)**

- **بیشینه:** تورنمنت یا پیمایش خطی، ۷ مقایسه.
- **بیشینه و کمینه:** ۴ مقایسهٔ جفتی؛ بیشینه بین ۴ برنده (۳ مقایسه)؛ کمینه بین ۴ بازنده (۳ مقایسه). جمع $3 \cdot 8/2 - 2 = 10$ ✓. روش ساده (جدا جدا) $7 + 7 = 14$ مقایسه می‌خواهد.
- **دومین بیشینه:** تورنمنت ۷ مقایسه دارد و برنده در $\log_2 8 = 3$ بازی شرکت کرده. دومین بین همین ۳ بازنده است ⇐ ۲ مقایسهٔ دیگر. جمع $8 + 3 - 2 = 9$ ✓.

درس ۳ انتخاب kامین عضو

- **انتخاب سریع (QuickSelect):** میانگین $O(n)$، بدترین حالت $O(n^2)$.
- **میانهٔ میانه‌ها:** گروه‌های ۵تایی، میانهٔ هر گروه، و بعد میانهٔ آن میانه‌ها به‌عنوان محور. رابطه $T(n) \le T(n/5) + T(7n/10) + O(n)$ است که چون $1/5 + 7/10 < 1$، جوابش $O(n)$ **در بدترین حالت** است.
- با گروه‌های ۳تایی، $1/3 + 2/3 = 1$ و جواب $O(n \log n)$ می‌شود (کار نمی‌کند).
- **kامین کوچک‌ترین با هیپ:** $O(n + k \log n)$.

✏️ **مثال برای هر مورد**

- **QuickSelect:** سومین کوچک‌ترین در $[7, 2, 9, 4, 1]$. محور ۷ ⇐ کوچک‌ترها $[2, 4, 1]$ (۳ عضو)؛ پس جواب همین‌جاست. محور ۲ ⇐ کوچک‌تر $[1]$؛ ۲ دومین است، پس سومی در $[4]$ است ⇐ **۴** ✓ (مرتب: $1, 2, 4, 7, 9$).
- **چرا $7n/10$ ؟** نصف گروه‌ها ($n/10$ گروه) میانه‌ای ≤ محور دارند و هر کدام ۳ عضو ≤ محور دارند (میانه و دو بزرگ‌تر از آن). پس حداقل $3n/10$ عضو ≤ محورند، و بخش کوچک‌تر حداکثر $7n/10$ عضو دارد.
- **گروه ۳تایی:** فقط $2 \times n/6 = n/3$ عضو تضمینی است ⇐ $T(n/3) + T(2n/3) + O(n) = \Theta(n \log n)$.
- **هیپ:** با $n = 1000$ و $k = 3$: build-heap حدود ۱۰۰۰ قدم، و ۳ بار حذف کمینه هر کدام حدود ۱۰ قدم.

درس ۴ امید تعداد مقایسه

در جستجوی خطی از ابتدا، اگر x در مکان i باشد، i مقایسه لازم است. پس امید = $\sum_{i} i \cdot \Pr[x \text{ در } i]$. اگر احتمال‌ها در بخش‌های مختلف فرق دارند، برای هر بخش «احتمال بخش × میانگین مکان در آن بخش» را حساب کن و جمع بزن.

✏️ مثال

- **یکنواخت:** با ۴ مکان و احتمال برابر: $(1 + 2 + 3 + 4)/4 = 2.5$.
- **بخش‌بندی:** ۱۰۰ مکان. x با احتمال 0.8 در یکی از مکان‌های ۱ تا ۵۰ است و با احتمال 0.2 در ۵۱ تا ۱۰۰ (در هر بخش یکنواخت). میانگین مکان در بخش اول 25.5 و در بخش دوم 75.5 است: $0.8 \times 25.5 + 0.2 \times 75.5 = 20.4 + 15.1 = 35.5$.

✍️ تمرین فصل ۶

اول خودت حل کن، بعد پاسخ‌نامه

۱. در آرایهٔ چرخشی $30\ 40\ 50\ 60\ 70\ 80\ 90\ 10\ 20$ ، نقطهٔ چرخش با جستجوی دودویی چطور پیدا می‌شود؟

۲. حداقل مقایسه برای یافتن بیشینه و کمینهٔ ۱۰۰ عدد؟

۳. در آرایهٔ مرتب n تایی، چطور در $O(\log n)$ تعداد تکرار x را پیدا می‌کنی؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۸۹**

📝 آزمون فصل ۶

سؤال‌های واقعی کنکور ارشد کامپیوتر

۹ سؤال | ⏱️ زمان: ۲۲ دقیقه برای ۹ سؤال | 🔑 کلید و راه‌حل: صفحهٔ ۸۹

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۴ · سؤال ۶۴ — غلط یا حدسی؟ ☐

هر الگوریتم قطعی که بتواند بزرگ‌ترین کلید دوم (دومین بزرگ‌ترین کلید) را در هر ورودی ممکن، تنها با مقایسهٔ کلیدها بیابد، باید در بدترین حالت، حداقل چند مقایسه انجام دهد؟ (تعداد عناصر برابر n است.)

۱) $n + \lceil \log n \rceil - 1$ ۲) $n + \lceil \log n \rceil - 2$ ۳) $n + \lceil \log n \rceil + 2$ ۴) $2n + \lceil \log n \rceil$

۲ ارشد ۱۴۰۳ · سؤال ۶۲ — غلط یا حدسی؟ ☐

فرض کنید آرایه‌ای به طول n داریم که به شکل حلقوی مرتب صعودی است. برای مثال آرایهٔ زیر: 40 | 50 | 60 | 70 | 80 | 90 | 10 | 20 | 30 می‌خواهیم الگوریتمی بنویسیم که $\lfloor \sqrt{n} \rfloor$ امین کوچک‌ترین عنصر این آرایه را بیابیم. مرتبهٔ زمانی این الگوریتم چیست؟

۱) $O(n)$ ۲) $O(\sqrt{n})$ ۳) $O(\log^2 n)$ ۴) $O(\log n)$

۳ ارشد ۱۴۰۲ · سؤال ۶۲ — غلط یا حدسی؟ ☐

یک آرایهٔ دوبعدی $n \times n$ از اعداد داده‌شده است که هر سطر به‌صورت صعودی از چپ به راست و هر ستون به‌صورت صعودی از بالا به پایین مرتب شده‌است. کمترین پیچیدگی زمانی برای پیداکردن یک عدد داده‌شده در این آرایهٔ دوبعدی کدام است؟

۱) $O(n \log n)$ ۲) $O(\log n)$ ۳) $O(n^2)$ ۴) $O(n)$

۴ ارشد ۱۴۰۱ · سؤال ۶۹ — غلط یا حدسی؟ ☐

مسئلهٔ جستجوی عنصر x در آرایهٔ A شامل n عنصر را در نظر بگیرید. فرض کنید اطلاع داریم که توزیع ورودی به این‌صورت است که احتمال حضور عنصر x در نیمهٔ دوم آرایه سه برابر احتمال حضور آن در نیمهٔ اول است. همچنین برای هر نیمه، احتمال حضور در هر خانه یکسان است. تعداد مقایسه‌های الگوریتم جستجوی خطی برای یافتن عنصر x در آرایه به‌طور متوسط چقدر است؟ (فرض کنید طول آرایهٔ A زوج است و عدد x در آرایه وجود دارد. ضمناً جستجوی خطی از ابتدای آرایه شروع می‌شود.)

۱) n ۲) $5n/8$ ۳) $3n/4$ ۴) $n/2$

۵ ارشد ۱۴۰۱ · سؤال ۷۱ — غلط یا حدسی؟

آرایهٔ نامتناهی A را در نظر بگیرید. فرض کنید در n خانهٔ اول این آرایه n عدد صحیح متناهی به‌صورت مرتب‌شده (صعودی) قرار گرفته‌اند و بقیهٔ خانه‌های آرایه با ∞ پر شده است. به‌ازای عدد دادهٔ x می‌خواهیم بررسی کنیم آیا عدد x در آرایه وجود دارد یا خیر. با چه مرتبهٔ زمانی می‌توان به این پرسش پاسخ داد؟ (با فرض آنکه مقدار n را از قبل نمی‌دانیم.)

۱) $O(n)$ ۲) $O(\log^2 n)$ ۳) $O(\log n)$ ۴) $O(1)$

۶ ارشد ۱۴۰۰ · سؤال ۶۸ — غلط یا حدسی؟

یک درخت ۱۰ رأسی داریم که یکی از رأس‌های آن به‌عنوان هدف درنظر گرفته شده است، اما ما از آن اطلاع نداریم. در هر پرسمان می‌توانیم یک رأس را انتخاب کنیم و متوجه شویم آیا این رأس هدف است یا نه و اگر نیست کدام یال آن به هدف نزدیک‌تر است. در بدترین حالت با حداقل چند پرسمان می‌توانیم رأس هدف را پیدا کنیم؟

۱) ۹ ۲) ۵ ۳) ۴ ۴) ۳

۷ ارشد ۱۴۰۰ · سؤال ۷۰ — غلط یا حدسی؟

سالن مربعی شکل در اختیار داریم که مختصات گوشهٔ چپ‌پایین آن $(0,0)$ و مختصات گوشهٔ راست‌بالای آن $(10,10)$ است. چهار نفر در این سالن در مکان‌های $(9,4)$ ، $(5,8)$ ، $(4,3)$ و $(1,7)$ قرار گرفته‌اند. می‌خواهیم از گوشهٔ چپ‌پایین به گوشهٔ راست‌بالا برویم. به هر شکل می‌توانیم حرکت کنیم، تنها نباید از سالن خارج شویم. حداکثر فاصلهٔ اجتماعی که می‌توانیم رعایت کنیم چند است؟

۱) $\sqrt{17}/2$ ۲) 2.5 ۳) $2\sqrt{2}$ ۴) $\sqrt{26}/2$

۸ ارشد ۱۳۹۹ · سؤال ۸۷ — غلط یا حدسی؟

فرض کنید یک آرایهٔ مرتب از n عدد صحیح در اختیار داریم. با چه تعداد مقایسه می‌توانیم بفهمیم عددی بیش از $n/5$ بار در آرایه تکرار شده است یا خیر؟ (بهترین گزینه را انتخاب کنید.)

۱) $O(n)$ ۲) $O(\log n)$ ۳) $O(\log^* n)$ ۴) $O(\log \log n)$

۹ ارشد ۱۳۹۹ · سؤال ۹۱ — غلط یا حدسی؟

فرض کنید آرایهٔ $A[1..n]$ شامل n عدد صحیح متمایز است که به صورت صعودی مرتب شده‌اند. چند تا از مسائل زیر را می‌توان با مرتبهٔ زمانی $O(\log n)$ حل کرد؟

- پیدا کردن یک اندیس i طوری‌که $A[i] = i$ شود.
- پیدا کردن یک اندیس i طوری‌که $A[i] = 3i + 2$ شود.
- پیدا کردن یک اندیس i طوری‌که $A[i] = 4i^2 + 3i + 5$ شود.

۱) ۰ ۲) ۱ ۳) ۲ ۴) ۳

🗂️ کارت خلاصهٔ فصل ۶

برای مرور سریع، شب قبل از آزمون

- ✔ دودویی روی هر ویژگی یکنوا کار می‌کند. چرخشی: اول نقطهٔ چرخش. نامتناهی: نمایی و بعد دودویی. ماتریس مرتب: پلکانی $O(n)$.
- ✔ $A[i] = i$ با اعداد صحیح متمایز: $O(\log n)$. ضریب بزرگ‌تر از ۱ یکنوایی را می‌شکند.
- ✔ بیشینه $n - 1$ ؛ بیشینه و کمینه $3n/2 - 2$ ؛ دومین بیشینه $n + \lceil \log n \rceil - 2$.
- ✔ میانهٔ میانه‌ها با گروه ۵تایی $O(n)$ بدترین حالت؛ با ۳تایی کار نمی‌کند.
- ✔ درخت: پرسیدن مرکزوار، حدود $\log_2 n$ پرسش.

پاسخ‌نامهٔ فصل ۶

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۶

برگشت ← صفحهٔ ۸۵

۱. عضو وسط را با عضو آخر مقایسه کن: اگر بزرگ‌تر بود، کوچک‌ترین در نیمهٔ راست است؛ وگرنه در نیمهٔ چپ (با خود وسط).

۲. $3 \cdot 100/2 - 2 = 148$

۳. با دو جستجوی دودویی، اولین و آخرین مکان x را پیدا کن. تعداد = اختلاف + ۱.

آزمون فصل ۶ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۸۶

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹
۲	۴	۴	۲	۳	۴	۴	۲	۲

۱ گزینهٔ ۲

تورنمنت: $n - 1$ مقایسه برای برنده. دومین بین $\lceil \log n \rceil$ نفری است که به برنده باخته‌اند، که $\lceil \log n \rceil - 1$ مقایسهٔ دیگر می‌خواهد.

۲ گزینهٔ ۴

نقطهٔ چرخش با دودویی در $O(\log n)$ پیدا می‌شود. kامین کوچک‌ترین در اندیس $(\text{چرخش} + k - 1) \bmod n$ است.

۳ گزینهٔ ۴

روش پلکانی از گوشهٔ بالا-راست: هر مقایسه یک سطر یا یک ستون را حذف می‌کند، پس حداکثر $2n$ گام.

۴ گزینهٔ ۲

احتمال نیمهٔ اول ¼ و نیمهٔ دوم ¾ است. میانگین مکان در نیمهٔ اول حدود $n/4$ و در نیمهٔ دوم حدود $3n/4$ است. امید: $(1/4)(n/4) + (3/4)(3n/4) = n/16 + 9n/16 = 5n/8$.

۵ گزینهٔ ۳

جستجوی نمایی: اندیس‌های $1, 2, 4, \dots$ را چک کن تا به عدد بزرگ‌تر از x یا ∞ برسی ($O(\log n)$ گام)، بعد دودویی در آن بازه.

۶ **گزینهٔ ۴ (۳)**

پرسیدن رأس مرکزوار: ۱۰ ⇐ حداکثر ۵ ⇐ حداکثر ۲ ⇐ حداکثر ۱. پس ۳ پرسش کافی است. روی مسیر ۱۰ رأسی، با ۲ پرسش حداکثر ۷ رأس تشخیص داده می‌شود، پس ۳ لازم هم هست.

۷ **گزینهٔ ۴**

دور هر نفر دایره‌ای به شعاع d بکش. راه بسته می‌شود وقتی زنجیره‌ای از دایره‌های متقاطع، دیوارهای «پایین یا راست» را به دیوارهای «چپ یا بالا» وصل کند. نفر $(9,4)$ در $d \geq 1$ به دیوار راست می‌رسد. فاصله‌اش تا $(4,3)$ برابر $\sqrt{26}$ است، پس در $d = \sqrt{26}/2$ به آن وصل می‌شود. نفر $(4,3)$ با $(1,7)$ (فاصلهٔ ۵، یعنی $d = 2.5$) به دیوار چپ وصل می‌شود. پس زنجیره در $d = \sqrt{26}/2 \approx 2.55$ بسته می‌شود. هیچ زنجیرهٔ زودتری نیست (با رایانه چک شد).

۸ **گزینهٔ ۲**

فقط عضوهای مکان‌های $n/5$، $2n/5$، … کاندیدند (حداکثر ۵ تا). برای هر کدام اولین و آخرین مکان را با دودویی پیدا کن: $O(\log n)$.

۹ **گزینهٔ ۲ (فقط اولی)**

برای اعداد صحیح متمایز مرتب، $A[i] - i$ نانزولی است، پس دودویی کار می‌کند. $A[i] - 3i$ و $A[i] - 4i^2 - \dots$ یکنوا نیستند.

فصل ۷

تقسیم و حل، و الگوریتم‌های حریصانه

۷

۵ درس | ۱۱ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۱۱ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴ (۴ تقسیم و حل و ۷ حریصانه). تیپ‌ها: درستی یک الگوریتم تقسیم و حل، زمانبندی کارها با جریمه، هافمن، و «حریصانه کجا شکست می‌خورد؟».

⏱️ **زمان مطالعه** دو بلوک ۹۰ دقیقه‌ای (درس ۱ و ۲، بعد درس ۳ تا ۵).

نقشهٔ فصل:
۱ الگوهای تقسیم و حل
۲ جریان بیشینه (در حد تست‌ها)
۳ حریصانه، کی درست است؟
۴ کدگذاری هافمن
۵ مثال نقض ساختن برای حریصانه
📝 آزمون فصل

درس ۱ الگوهای تقسیم و حل

مسئله	رابطه	زمان
زیرآرایه با بیشترین مجموع	$2T(n/2) + O(n)$	$O(n \log n)$ (کادان $O(n)$ است)
ضرب اعداد یا چندجمله‌ای (کاراتسوبا)	$3T(n/2) + O(n)$	$O(n^{1.585})$
ضرب ماتریس (استراسن)	$7T(n/2) + O(n^2)$	$O(n^{2.81})$
نزدیک‌ترین جفت نقاط	$2T(n/2) + O(n)$	$O(n \log n)$
شمردن جفت‌های نابه‌جا	ادغامی	$O(n \log n)$

✏️ مثال برای هر سطر جدول

- **بیشترین مجموع:** در $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ جواب زیرآرایهٔ $[4, -1, 2, 1]$ با مجموع **۶** است. تقسیم و حل: بهترین در نیمهٔ چپ، بهترین در نیمهٔ راست، و بهترینِ رد شونده از وسط. کادان در یک پیمایش: $\text{cur} = \max(x, \text{cur} + x)$.
- **کاراتسوبا:** 12×34 با $a = 1, b = 2, c = 3, d = 4$: $ac = 3$ ، $bd = 8$ ، $(a+b)(c+d) = 3 \times 7 = 21$ ، پس $ad + bc = 21 - 3 - 8 = 10$. جواب: $3 \cdot 100 + 10 \cdot 10 + 8 = 408$ ✓. سه ضرب به‌جای چهار.
- **استراسن:** ماتریس را به چهار بلوک $n/2 \times n/2$ تقسیم می‌کند و به‌جای ۸ ضرب بلوکی، ۷ ضرب (و چند جمع) انجام می‌دهد.
- **نزدیک‌ترین جفت:** نقاط را با یک خط عمودی نصف کن و δ را از دو نیمه بگیر. فقط نوار به عرض 2δ دور خط را بررسی کن؛ هر نقطه در نوار فقط با حداکثر ۷ نقطهٔ بعدی (مرتب بر حسب y) مقایسه می‌شود.
- **نابه‌جایی‌ها** در $[2, 4, 1, 3]$: جفت‌های $(2,1)$ ، $(4,1)$ ، $(4,3)$ ، یعنی ۳ تا. در ادغام $[2, 4]$ و $[1, 3]$: وقتی ۱ از راست برداشته می‌شود، ۲ عضو چپ باقی است ($+2$)؛ وقتی ۳ برداشته می‌شود، ۱ عضو ($+1$). جمع ۳ ✓.

نکته‌ها

- **تعداد فراخوانی در کاراتسوبا:** با ۳ زیرمسئلهٔ نصف، اگر از درجهٔ D شروع کنی و درجهٔ $d \geq$ «کوچک» باشد، سطح‌ها $1, 3, 9, \ldots$ فراخوانی دارند. مثلاً برای ۱۶ ⇐ ۸ ⇐ ۴: $1 + 3 + 9 = 13$.
- **ضرب و وارون ماتریس هم‌ارزند:** هر الگوریتم ضرب را می‌شود به وارون با همان مرتبه تبدیل کرد و برعکس.
- **درستی تقسیم و حل:** باید ثابت کنی جواب کل از جواب زیرمسئله‌ها **قطعاً** به دست می‌آید. یک نمونهٔ رایج از الگوریتم نادرست: «عضو پرتکرار (حداقل $n/3$) را در هر یک‌سوم پیدا کن و یکی را برگردان». هر یک‌سوم ممکن است چند عضو پرتکرار داشته باشد و برگرداندن فقط یکی، جواب را گم می‌کند.
- **ترفند تبدیل:** شرط $A[i] \cdot A[j] > i \cdot j$ با تعریف $B[i] = A[i]/i$ می‌شود $B[i] \cdot B[j] > 1$. بعد با مرتب‌سازی و دو اشاره‌گر: $O(n \log n)$.

✏️ مثال

$A = [2, 1, 6]$ (اندیس از ۱، اعداد مثبت) ⇐ $B = [2/1, 1/2, 6/3] = [2, 0.5, 2]$.

- $(1, 3)$: $B_1 \cdot B_3 = 4 > 1$ ✓؛ چک: $2 \times 6 = 12 > 1 \times 3$.
- $(1, 2)$ و $(2, 3)$: $B \cdot B = 1$ ، که بزرگ‌تر از ۱ نیست؛ چک: $2 \times 1 = 2 = 1 \times 2$ و $1 \times 6 = 6 = 2 \times 3$.

درس ۲ جریان بیشینه (در حد تست‌ها)

- **جریان بیشینه = برش کمینه.** برش کمینه را پیدا کن؛ جمع ظرفیت یال‌هایی که از سمت s به سمت t می‌روند.
- افزایش ظرفیت یک یال فقط وقتی جریان را زیاد می‌کند که آن یال در **همهٔ** برش‌های کمینه باشد. سقف افزایش را **برش بعدی** تعیین می‌کند.

✏️ مثال

یال‌ها: $s\to a$ (3) ، $s\to b$ (2) ، $a\to t$ (2) ، $b\to t$ (3) ، $a\to b$ (1) .

- **جریان:** $s\to a\to t$ با ۲، $s\to b\to t$ با ۲، $s\to a\to b\to t$ با ۱ ⇐ جمع **۵**.
- **برش‌ها:** $\{s\}$ با $3+2=5$ ؛ $\{s,a\}$ با $2+2+1=5$ ؛ $\{s,b\}$ با $3+3=6$ ؛ $\{s,a,b\}$ با $2+3=5$. کمینه ۵ است = جریان بیشینه ✓.
- هیچ یالی در **هر سه** برش کمینه نیست، پس افزایش ظرفیت یک یال تنها جریان را زیاد نمی‌کند.

✏️ مثال (برش بعدی)

فقط $s\to a$ (2) و $a\to t$ (5) . جریان ۲ است. اگر ظرفیت $s\to a$ را به ۴ برسانی جریان ۴ می‌شود، ولی با ۱۰ فقط تا **۵** بالا می‌رود (برش بعدی $a\to t$).

درس ۳ حریصانه، کی درست است؟

💡 قاعده

حریصانه درست است اگر (۱) **خاصیت انتخاب حریصانه** داشته باشد (یک جواب بهینه با انتخاب حریصانه شروع می‌شود) و (۲) **زیرساختار بهینه** داشته باشد. اثباتش معمولاً با **استدلال تعویض** است.

مسئله	حریصانهٔ درست	حریصانهٔ غلط
انتخاب فعالیت (بیشترین بازهٔ ناهم‌پوشان)	**زودترین زمان پایان**	کوتاه‌ترین بازه؛ کمترین هم‌پوشانی (مثال نقض ۹ بازه‌ای)
کوله‌پشتی کسری	بیشترین نسبت ارزش به وزن	—
کوله‌پشتی ۰ و ۱	❌ حریصانه کار نمی‌کند (برنامه‌نویسی پویا لازم است)	
سکه	برای $\{1,2,5\}$ ، $\{1,5,10\}$ و $\{1,4,7\}$ درست است	$\{1,7,10\}$ برای ۱۴: حریصانه $10+1+1+1+1$ (۵ سکه)، ولی بهینه $7+7$ (۲ سکه)
پوشش حداکثر (Max Coverage)	تقریبی است، نه بهینه	انتخاب محلی بهینه، جواب کلی را تضمین نمی‌کند

✏️ **مثال برای هر سطر جدول**

- **فعالیت:** [1,3] ، [2,5] ، [4,6] : زودترین پایان [1,3] است، بعد [4,6] ⇐ ۲ فعالیت.
  - **نقض «کوتاه‌ترین»:** [1,5] ، [4,7] ، [6,10] . کوتاه‌ترین [4,7] است که با هر دو تای دیگر هم‌پوشانی دارد ⇐ فقط ۱ فعالیت؛ بهینه [1,5] و [6,10] است (۲ تا).
- **کسری:** ظرفیت ۵۰ و کالاهای (ارزش، وزن) (60, 10) ، (100, 20) ، (120, 30) با نسبت‌های ۶، ۵، ۴. دو تای اول کامل (۱۶۰) و 20/30 از سومی (۸۰) ⇐ **۲۴۰**.
- **۰ و ۱:** همان کالاها. حریصانه با نسبت، اولی و دومی را برمی‌دارد (۱۶۰، وزن ۳۰) و سومی دیگر جا نمی‌شود. ولی دومی و سومی با هم $100 + 120 = 220$ می‌دهند.
- **سکه با $\{1, 5, 10\}$:** برای ۲۸: $10 + 10 + 5 + 1 + 1 + 1$ (۶ سکه)، که بهینه است.
- **پوشش:** $S_1 = \{1,2,3,4\}$ ، $S_2 = \{1,2,5\}$ ، $S_3 = \{3,4,6\}$ و ۲ مجموعه مجاز است. حریصانه اول S_1 (۴ عضو تازه) و بعد یکی دیگر (فقط ۱ عضو تازه) ⇐ ۵ عضو. ولی $S_2 \cup S_3$ هر ۶ عضو را می‌پوشاند.

زمانبندی کارهای واحد با مهلت و جریمه

کارها را به‌ترتیب **جریمهٔ نزولی** بررسی کن. هر کار را بپذیر اگر مجموعهٔ پذیرفته‌ها هنوز **امکان‌پذیر** باشد؛ یعنی برای هر t، تعداد کارهای پذیرفته با مهلت $\le t$ حداکثر t باشد. جریمهٔ کل = جمع جریمهٔ کارهای ردشده.

✏️ **مثال**

کارهای واحد (مهلت، جریمه): a (1, 30) ، b (1, 20) ، c (2, 10) ، d (2, 40) .

- ترتیب بررسی: d، a، b، c.
- d را بپذیر ✓. a را بپذیر: یک کار با مهلت ≥ ۱ و دو کار با مهلت ≥ ۲ ✓.
- b: دو کار با مهلت ≥ ۱ می‌شد ✗. c: سه کار با مهلت ≥ ۲ می‌شد ✗.
- جریمهٔ کل $30 = 10 + 20$. زمانبندی: a در لحظهٔ ۱ و d در لحظهٔ ۲.

درس ۴ کدگذاری هافمن

- دو کمترین فراوانی را ادغام کن و تکرار کن. درخت **همیشه پر** است (هر گرهٔ داخلی ۲ فرزند دارد). پس طول‌های کد در **برابری کرافت** صدق می‌کنند: $\Sigma\ 2^{(-l_i)} = 1$.
- برای ۴ حرف، طول‌های ممکن فقط (1,2,3,3) یا (2,2,2,2) هستند.
  - اگر طول کل متن **فرد** باشد، (2,2,2,2) ممکن نیست (طول کل $2N$ زوج می‌شد).
- **بیشترین ارتفاع** ($n - 1$) با فراوانی‌های **فیبوناچی** ساخته می‌شود.
- اگر همهٔ کدها هم‌طول شوند (مثلاً ۳۲ حرف با کد ۵ بیتی)، فراوانی‌ها **لزوماً برابر نیستند**. کافی است هیچ دو فراوانی کوچک‌تری با هم کمتر از بزرگ‌ترین نشوند، و اختلاف فراوانی‌ها می‌تواند دلخواه بزرگ باشد.

✏️ مثال برای هر مورد

- **ساختن:** فراوانی‌های a:5, b:2, c:1, d:1 . ادغام $c + d = 2$ ، بعد $b + (cd) = 4$ ، بعد $a + 4 = 9$. طول‌ها a:1, b:2, c:3, d:3، یعنی (1,2,3,3) . کرافت: $1/2 + 1/4 + 1/8 + 1/8 = 1$ ✓.
- **فرد بودن:** طول کل متن کدشده $5{\cdot}1 + 2{\cdot}2 + 1{\cdot}3 + 1{\cdot}3 = 15$ است که فرد است؛ با (2,2,2,2) زوج می‌شد.
- **فیبوناچی:** 1, 1, 2, 3, 5 ⇐ ادغام‌ها 2، 4، 7، 12 . درخت یک زنجیر با ارتفاع $n - 1 = 4$ است و طول کدها 4, 3, 2, 1.
- **فراوانی نابرابر، کد هم‌طول:** 10, 11, 12, 13 ⇐ $10 + 11 = 21$ ، $12 + 13 = 25$ ، $21 + 25 = 46$. هر چهار کد ۲ بیتی‌اند، چون $13 \geq 21$.

درس ۵ مثال نقض ساختن برای حریصانه

۱. حالتی بساز که انتخاب حریصانه **دو** عضو جواب بهینه را از بین ببرد.

۲. با اضافه کردن عضوهای «فریبنده» کاری کن که معیار حریصانه آن انتخاب بد را ترجیح دهد.

۳. **تساوی** هم کافی است، اگر سؤال بگوید «در صورت تساوی، یکی دلخواه انتخاب می‌شود».

✏️ مثال

حریصانهٔ «زودترین **شروع**» برای انتخاب فعالیت. بازهٔ بلند [1, 10] (فریبنده) با [2, 3] و [4, 5] هم‌پوشانی دارد. حریصانه [1, 10] را برمی‌دارد ⇐ ۱ فعالیت، ولی بهینه ۲ است. یک انتخاب بد، دو عضو بهینه را از بین برد (قدم ۱ و ۲).

✍️ تمرین فصل ۷ — اول خودت حل کن، بعد پاسخ‌نامه

۱. با سکه‌های {1, 3, 4} ، حریصانه برای ۶ چند سکه می‌دهد؟ بهینه چند است؟

۲. فراوانی‌ها 5, 9, 12, 13, 16, 45 هستند. طول کد هافمن حرف با فراوانی ۴۵ چند است؟

۳. فعالیت‌های [1,4]، [3,5]، [0,6]، [5,7]، [3,9]، [5,9]، [6,10]، [8,11] با حریصانهٔ «زودترین پایان» (بازه‌هایی که فقط در نقطهٔ انتها به هم می‌رسند هم‌پوشان حساب نمی‌شوند): چند فعالیت انتخاب می‌شود؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۱۰۰

📝 آزمون فصل ۷

سؤال‌های واقعی کنکور ارشد کامپیوتر

۱۱ سؤال | ⏱️ زمان: ۳۰ دقیقه برای ۱۱ سؤال | 🔑 کلید و راه‌حل: **صفحهٔ ۱۰۰**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۳ · سؤال ۶۷ — غلط یا حدسی؟ ☐

چند مورد از گزاره‌های زیر درست است؟

- هر الگوریتم که ضرب دو ماتریس را محاسبه کند، می‌تواند در همان مرتبه وارون یک ماتریس را محاسبه کند و بالعکس.
- برای محاسبهٔ ضرب دو چندجمله‌ای از درجهٔ ۱۶ تعداد فراخوانی‌های لازم با استفاده از الگوریتم تقسیم و حل، وقتی که چندجمله‌ای حداکثر از درجهٔ ۴، چندجمله‌ای کوچک تلقی می‌شود برابر است با ۱۳.
- در شبکهٔ جریان داده‌شدهٔ زیر اگر فقط مجاز به افزایش ظرفیت یک یال باشیم، حداکثر می‌توان ۷ واحد به ظرفیت یک یال آن اضافه کرد تا شبکه حداکثر جریان عبوری را داشته باشیم. (یال‌ها و ظرفیت‌ها: $s \to 2:\ 5$ ، $s \to 3:\ 9$ ، $2 \to 3:\ 2$ ، $2 \to t:\ 14$ ، $3 \to t:\ 3$)

۱) صفر ۲) ۱ ۳) ۲ ۴) ۳

۲ ارشد ۱۴۰۱ · سؤال ۶۳ — غلط یا حدسی؟ ☐

آرایهٔ A شامل n عدد داده شده است. هدف پیدا کردن تعداد جفت اندیس‌های i و j است، به‌گونه‌ای که $A[i] \times A[j] > i \times j$. این کار در چه زمانی قابل انجام است؟ (بهترین گزینه را انتخاب کنید.)

۱) $O(n^2 \log n)$ ۲) $O(n^2)$ ۳) $O(n \log n)$ ۴) $O(n \log^2 n)$

۳ ارشد ۱۴۰۱ · سؤال ۷۰ — غلط یا حدسی؟

آرایهٔ A شامل n عنصر داده شده است. عنصری از آرایه که حداقل $n/3$ بار تکرار شده باشد را یک عضو پرتکرار می‌نامیم. می‌خواهیم در آرایهٔ A یک عضو پرتکرار را در صورت وجود پیدا کنیم. برای این کار از یک روش تقسیم و غلبه به این صورت استفاده می‌کنیم: ابتدا آرایه را به سه قسمت با اندازهٔ برابر تقسیم می‌کنیم و در کدام از قسمت‌ها به‌طور بازگشتی در صورت وجود یک عضو پرتکرار را پیدا می‌کنیم. سپس میزان تکرار هر کدام از این سه عضو پرتکرار را در آرایهٔ اصلی جستجو می‌کنیم و در صورتی‌که میزان تکرار هر عضو حداقل $n/3$ بود آن عضو را به‌عنوان عضو پرتکرار برمی‌گردانیم. کدام گزاره درخصوص این الگوریتم درست است؟ (فرض کنید n توانی از ۳ است.)

۱) زمان اجرا $O(n \log n)$ است، اما الگوریتم لزوماً درست کار نمی‌کند.

۲) زمان اجرا $O(n \log n)$ است و الگوریتم درست کار می‌کند.

۳) زمان اجرا $O(n)$ است، اما الگوریتم لزوماً درست کار نمی‌کند.

۴) زمان اجرا $O(n)$ است و الگوریتم درست کار می‌کند.

۴ ارشد ۱۴۰۰ · سؤال ۷۱ — غلط یا حدسی؟

آرایه‌ای شامل n عدد را در نظر بگیرید. در هر پرسمان، می‌توانیم دو اندیس i و j که اندیس شروع و پایان یک بازه از آرایه هستند را بدهیم و به ما مجموع اعداد بازه داده می‌شود. هدف پیداکردن بازه‌ای است که مجموع اعداد بازه بیشینه شود. چه تعداد پرسمان برای این کار نیاز است؟

۱) $O(n)$ ۲) $O(n^2)$ ۳) $O(\log n)$ ۴) $O(n \log n)$

۵ ارشد ۱۴۰۴ · سؤال ۶۲ — غلط یا حدسی؟

فرض کنید برای حل مسئلهٔ پوشش حداکثر (maximum Coverage)، از الگوریتم حریصانه استفاده می‌کنید که مجموعه‌ای از عناصر را با کمترین هزینهٔ ممکن پوشش دهد. چرا این الگوریتم **نمی‌تواند** جواب بهینه را تضمین کند؟

۱) الگوریتم حریصانه همیشه انتخاب محلی بهینه دارد، اما ممکن است در سطح کلان بهترین نباشد.

۲) مسئلهٔ پوشش حداکثر یک مسئلهٔ NPکامل است و نیاز به حل دقیق دارد.

۳) تمام راه‌حل‌های ممکن را بررسی نمی‌کند.

۴) پیچیدگی زمانی آن بالا است.

۶ ارشد ۱۴۰۳ · سؤال ۶۴ — غلط یا حدسی؟

هریک از کارهای زیر در یک واحد زمان قابل اجرا است. هر یک از این کارها دارای یک زمان خاتمه است و درصورتی‌که بعد از زمان خاتمه انجام شود مشمول یک جریمه خواهد شد. اگر این کارها را برای اجرا به کمترین جریمه زمان‌بندی کنیم، مقدار جریمه چقدر است؟

کار	w_1	w_2	w_3	w_4	w_5	w_6	w_7
مهلت (Deadline)	۱۰	۲	۳	۳	۲	۵	۱
جریمه (Penalty)	۱۰	۴۵	۵۵	۶۵	۷۰	۳۳	۱۸

۱) ۲۸ ۲) ۴۳ ۳) ۵۱ ۴) ۶۳

۷ ارشد ۱۴۰۰ · سؤال ۶۷ — غلط یا حدسی؟

متنی که هر حرف آن یکی از چهار نویسهٔ $\{a, b, c, d\}$ است را با الگوریتم هافمن کدگذاری کرده‌ایم. طول کد هافمن این متن ۲۰۲۱ بیت شده است. طول کد چهار نویسهٔ فوق در کدگذاری هافمن کدام است؟

۱) $1, 2, 3, 3$ ۲) $1, 2, 2, 3$ ۳) $1, 2, 3, 4$ ۴) $2, 2, 2, 2$

۸ ارشد ۱۳۹۹ · سؤال ۹۴ — غلط یا حدسی؟

فرض کنید در کدگذاری هافمن، طول کد همهٔ کاراکترها یکسان شده است. با فرض آنکه تعداد کاراکترها ۳۲ می‌باشد، چند تا از گزاره‌های زیر همیشه درست است؟

- تعداد تکرار همهٔ کاراکترها یکسان است.
- اختلاف تکرار هر دو کاراکتر حداکثر یک است.
- اختلاف تکرار هر دو کاراکتر حداکثر دو است.
- به ازای هر عدد ثابت c می‌توان مثالی زد که دو کاراکتر وجود داشته باشند که اختلاف تکرارشان حداقل c باشد.

۱) ۰ ۲) ۱ ۳) ۲ ۴) ۳

۹ ارشد ۱۳۹۹ · سؤال ۹۵ — غلط یا حدسی؟

الگوریتم خرد کردن پول با روش حریصانهٔ «استفاده از پرارزش‌ترین سکه، تا حد امکان» روی کدام مجموعهٔ سکه‌ها، لزوماً جواب بهینه (با کمترین تعداد سکه) را تولید **نمی‌کند**؟ (فرض کنید از سکه‌های هر مجموعه به تعداد نامتناهی داریم.)

۱) $\{1, 2, 5\}$ ۲) $\{1, 4, 7\}$ ۳) $\{1, 5, 10\}$ ۴) $\{1, 7, 10\}$

۱۰ ارشد ۱۳۹۹ · سؤال ۹۸ — غلط یا حدسی؟

کدام‌یک از دنباله‌های زیر (به ازای nهای بزرگ) بیش‌ترین ارتفاع ممکن را برای درخت هافمن ایجاد می‌کند؟ (اعضای دنباله‌ها نشان‌دهندهٔ تعداد تکرار کاراکترها در متن ورودی است نه خود کاراکترها.)

۱) دنباله‌ای از n عدد برابر

۲) دنباله‌ای از n عدد فیبوناچی پشت سر هم

۳) دنبالهٔ $(1, 2, 3, 4, 5, \ldots, n)$

۴) دنبالهٔ $(1^2, 2^2, 3^2, 4^2, 5^2, \ldots, n^2)$

۱۱ ارشد ۱۳۹۸ · سؤال ۵۹ — غلط یا حدسی؟

فرض کنید n بازه روی محور اعداد حقیقی داریم. می‌خواهیم بیشترین تعداد بازه را پیدا کنیم که با یکدیگر هم‌پوشانی نداشته باشند. برای بازهٔ I فرض کنید $c(I)$ برابر تعداد بازه‌هایی است که با I هم‌پوشانی دارند. برای حل این مسئله از الگوریتم حریصانهٔ زیر استفاده می‌کنیم: در هر مرحله از بین بازه‌های باقی‌مانده بازهٔ I را که $c(I)$ آن از همه کمتر است انتخاب و تمام بازه‌هایی که با آن هم‌پوشانی دارند حذف می‌کنیم. اگر چند بازه وجود داشت که تعداد هم‌پوشانی‌شان کمینه بود یکی را به دلخواه انتخاب می‌کنیم. کمترین n که الگوریتم حریصانهٔ فوق لزوماً جواب بهینه را تولید **نمی‌کند**، در بین گزینه‌ها کدام است؟

۱) ۹ ۲) ۵ ۳) ۳ ۴) ۲

کارت خلاصهٔ فصل ۷

برای مرور سریع، شب قبل از آزمون

- ✔ کاراتسوبا $3T(n/2)$ ؛ استراسن $7T(n/2)$ ؛ فراخوانی‌ها $1 + 3 + 9 + \ldots$. ضرب و وارون ماتریس هم‌مرتبه‌اند.
- ✔ درستی تقسیم و حل: «فقط یکی برگرداندن» در مسائل چندجوابی اشتباه است.
- ✔ مجموع بازه با پرسمان: n پرسش پیشوندی کافی است.
- ✔ حریصانه: زودترین پایان ✓؛ کوله‌پشتی کسری ✓؛ ۰ و ۱ ✗؛ سکهٔ $\{1, 7, 10\}$ ✗.
- ✔ مهلت و جریمه: جریمهٔ نزولی + چک امکان‌پذیری (تعداد کارهای با مهلت $\leq t$ حداکثر t).
- ✔ هافمن: درخت پر (کرافت = ۱)؛ ۴ حرف ⇐ $(1,2,3,3)$ یا $(2,2,2,2)$ ؛ فیبوناچی ⇐ بیشترین ارتفاع.

پاسخ‌نامهٔ فصل ۷

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۷

برگشت ← صفحهٔ ۹۵

۱. حریصانه $1 + 1 + 4$ (۳ سکه) می‌دهد؛ بهینه $3 + 3$ (۲ سکه) است.

۲. **۱** (ادغام‌ها: $5+9=14$ ، $12+13=25$ ، $14+16=30$ ، $25+30=55$ ، $45+55=100$ ؛ پس ۴۵ مستقیم فرزند ریشه است).

۳. **۳**: $[1,4]$ ، بعد $[5,7]$ ، بعد $[8,11]$.

آزمون فصل ۷ – کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۹۶

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰	سؤال ۱۱
۴	۳	۱	۱	۱	۴	۱	۲	۴	۲	۱

۱ **گزینهٔ ۴ (هر سه درست)**

- گزارهٔ ۱: هم‌ارزی ضرب و وارون ماتریس (نتیجهٔ شناخته‌شده).
- گزارهٔ ۲: کاراتسوبا با ۳ زیرمسئلهٔ نصف: درجهٔ ۱۶ (۱ فراخوانی)، درجهٔ ۸ (۳ فراخوانی)، درجهٔ ۴ (۹ فراخوانی). جمع **۱۳**.
- گزارهٔ ۳: جریان فعلی ۱۰ است (برش $\{s\to2,\ 3\to2,\ 3\to t\} = 5 + 2 + 3$). اگر $s\to2$ را ۷ واحد زیاد کنیم (۱۲ شود)، جریان می‌شود $12 + 3 + 2 = 17$ ، و بیشتر از این به خاطر برش $\{2\to t,\ 3\to t\} = 14 + 3 = 17$ ممکن نیست. پس افزایش بیش از ۷ واحد فایده ندارد.

۲ **گزینهٔ ۳**

$B[i] = A[i]/i$ و شمردن جفت‌های با $B[i]\cdot B[j] > 1$ (با رعایت علامت) با مرتب‌سازی و دو اشاره‌گر: $O(n \log n)$.

۳ **گزینهٔ ۱**

$T(n) = 3T(n/3) + O(n) = O(n \log n)$. ولی هر یک‌سوم ممکن است چند عضو پرتکرار داشته باشد و الگوریتم فقط یکی را برمی‌گرداند، پس عضو پرتکرار کل ممکن است گم شود.

۴ **گزینهٔ ۱**

با n پرسش، مجموع‌های پیشوندی $S(1..k)$ را بگیر. مجموع هر بازه $S(j) - S(i-1)$ است و بیشینه بدون پرسش اضافه حساب می‌شود.

۵ **گزینهٔ ۱**

حریصانه بهینهٔ محلی را انتخاب می‌کند که لزوماً بهینهٔ کلی نیست.

۶ **گزینهٔ ۴ (۶۳)**

به‌ترتیب جریمهٔ نزولی:

- w_5 (۷۰، مهلت ۲) ✓
- w_4 (۶۵، مهلت ۳) ✓
- w_3 (۵۵، مهلت ۳) ✓ (سه کار با مهلت ≥ ۳)
- w_2 (۴۵، مهلت ۲) ✗ (چهار کار با مهلت ≥ ۳ می‌شد)
- w_6 (۳۳، مهلت ۵) ✓
- w_7 (۱۸، مهلت ۱) ✗ (چهار کار با مهلت ≥ ۳ می‌شد)
- w_1 (۱۰، مهلت ۱۰) ✓

ردشده‌ها: $45 + 18 = 63$. (با شمارش رایانه‌ای همهٔ زیرمجموعه‌ها هم تأیید شد.)

۷ **گزینهٔ ۱**

درخت هافمن پر است، پس فقط $(1,2,3,3)$ یا $(2,2,2,2)$ ممکن است (گزینهٔ ۲ کرافت را نقض می‌کند و گزینهٔ ۳ درخت پر نمی‌سازد). با $(2,2,2,2)$ طول کل $2N$ زوج است، ولی ۲۰۲۱ فرد است.

۸ **گزینهٔ ۲ (فقط آخری)**

کدها هم‌طول (۵ بیتی) می‌شوند اگر هیچ دو فراوانی کوچک‌تری با هم کمتر از بزرگ‌ترین نشوند، مثلاً همه بین f و $2f$ باشند. با f بزرگ، اختلاف دلخواه بزرگ می‌شود. پس سه گزارهٔ اول همیشه درست نیستند.

۹ **گزینهٔ ۴**

برای ۱۴: حریصانه $10 + 1 + 1 + 1$ (۵ سکه) و بهینه $7 + 7$ (۲ سکه). برای سه مجموعهٔ دیگر حریصانه همیشه بهینه است (با رایانه تا ۱۰۰ چک شد).

۱۰ **گزینهٔ ۲**

با فراوانی‌های فیبوناچی، هر ادغام با عضو بعدی انجام می‌شود و درخت زنجیری با ارتفاع $n - 1$ می‌سازد.

۱۱ **گزینهٔ ۱ (۹)**

مثال نقض ۹ بازه‌ای:

- جواب بهینه ۴ بازه است: $A_1=[0,2]$ ، $A_2=[3,5]$ ، $A_3=[6,8]$ ، $A_4=[9,11]$.
- بازهٔ $M = [4.5, 6.5]$ فقط با A_2 و A_3 هم‌پوشانی دارد، پس $c(M) = 2$.
- دو کپی از $[1.5, 3.5]$ و دو کپی از $[7.5, 9.5]$ اضافه کن. حالا $c(A_1) = c(A_4) = 2$ است.
- در تساوی، حریصانه می‌تواند M را بردارد و A_2 و A_3 را حذف کند. در نهایت فقط ۳ بازه به دست می‌آید.

با تعداد کمتر از ۹ بازه (بین گزینه‌ها) چنین مثالی ساخته نمی‌شود.

فصل ۸

الگوریتم‌های گراف

۸

★ پرتکرار در کنکور | ۵ درس | ۱۳ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** **۱۳ سؤال** در ارشد ۱۳۹۸ تا ۱۴۰۴، پرتکرارترین مبحث ساختمان داده. تیپ‌ها: دایکسترا با وزن منفی، ویژگی‌های درخت پوشای کمینه، «کدام مسئله خطی است؟»، و مدل‌سازی (نرخ ارز، فاصله تا یک مجموعه).

🧩 **پیش‌نیاز** گسسته فصل ۷ (مفاهیم گراف).

⏱️ **زمان مطالعه** دو بلوک ۹۰ دقیقه‌ای + ۳۰ دقیقه آزمون فصل.

نقشهٔ فصل: ۱ BFS و DFS | ۲ درخت پوشای کمینه (MST) | ۳ کوتاه‌ترین مسیر | ۴ جریان بیشینه | ۵ گراف و ضرب ماتریس | 📝 آزمون فصل

درس ۱ BFS و DFS

	BFS	DFS
زمان	$O(V+E)$	$O(V+E)$
کاربرد	کوتاه‌ترین مسیر **بدون وزن**؛ تشخیص دوبخشی؛ **شمردن کوتاه‌ترین مسیرها**	تشخیص دور؛ مرتب‌سازی توپولوژیک؛ مؤلفه‌های قویاً همبند؛ رأس برشی و پل

✏️ **مثال**

گراف بی‌جهت با یال‌های 1-2، 1-3، 2-4، 3-4، 4-5 .

- **BFS از ۱:** لایه‌ها $\{1\}$ ، $\{2,3\}$ ، $\{4\}$ ، $\{5\}$ ⇐ فاصلهٔ ۵ برابر ۳ است.
- **دوبخشی:** رأس‌های لایه‌های زوج یک رنگ و فرد رنگ دیگر. هیچ یالی دو رأس هم‌لایه را وصل نمی‌کند ⇐ دوبخشی است.
- **DFS از ۱** (همسایهٔ کوچک‌تر اول): $1 \to 2 \to 4 \to 3$. ۳ همسایهٔ دیده‌شدهٔ ۱ را دارد ⇐ یال عقب 3–1 ، پس دور 1-2-4-3-1 هست. بعد از ۴ به ۵ می‌رود.

یال‌های DFS

- **بی‌جهت:** فقط یال درختی و یال **عقب** (به جد). یال متقاطع وجود ندارد.
- **جهت‌دار:** درختی، عقب، جلو، متقاطع. **دور** ⇔ **یال عقب.**
- **مرتب‌سازی توپولوژیک:** ترتیب نزولی زمان پایان DFS (فقط برای DAG).

- **برگ درخت DFS** در گراف بی‌جهت: همهٔ همسایه‌هایش اجدادش هستند (یال عقب). دو برگ لزوماً همسایهٔ مشترک ندارند و لزوماً رأس برشی جدایشان نمی‌کند.

✏️ مثال (جهت‌دار)

یال‌ها $a\to b$، $b\to c$، $c\to a$، $a\to c$، $b\to d$. DFS از a (اول b بعد c)، بعد از d:

- $a\to b$ ، $b\to c$: درختی. $c\to a$: **عقب** (a جد c است) ⇐ دور دارد.
- $a\to c$: وقتی a سراغ c می‌رود، c نوادهٔ تمام‌شدهٔ a است ⇐ **جلو**.
- $d\to b$: b در درخت DFS قبلی تمام شده ⇐ **متقاطع**.

✏️ مثال (توپولوژیک)

DAG با یال‌های $a\to b$، $a\to c$، $b\to d$، $c\to d$. DFS از a: d اول تمام می‌شود، بعد b، بعد c، بعد a. ترتیب نزولی زمان پایان: a, c, b, d ✓.

✏️ مثال (برگ)

در گراف بی‌جهت مثال قبل (DFS از ۱)، برگ‌ها ۳ و ۵ هستند. همسایه‌های ۳ (یعنی ۱ و ۴) هر دو جد آن‌اند.

شمردن کوتاه‌ترین مسیرها (بدون وزن)

در BFS: $cnt[v] = \Sigma\, cnt[u]$، روی u هایی که در لایهٔ قبلی و همسایهٔ v هستند. زمان $O(V+E)$. اگر فقط «حداقل k» مهم است، شمارنده را روی k متوقف کن تا عددها بزرگ نشوند.

✏️ مثال

در گراف مثال اول: $cnt[1]=1$ ، $cnt[2]=cnt[3]=1$ ، $cnt[4]=cnt[2]+cnt[3]=2$ (مسیرهای 1-2-4 و 1-3-4)، $cnt[5]=cnt[4]=2$.

درس ۲ درخت پوشای کمینه (MST)

- **خاصیت برش:** سبک‌ترین یالِ هر برش در **یک** MST هست.
- **خاصیت دور:** سنگین‌ترین یالِ هر دور (اگر یکتا باشد) در **هیچ** MST نیست.
- **وزن‌های متمایز ⇐ MST یکتا** است، پس پریم و کراسکال جواب یکسان می‌دهند.
- با وزن‌های تکراری، **هر** MST با یک ترتیب مناسب شکستن تساوی، هم از پریم هم از کراسکال به دست می‌آید.
- **همهٔ وزن‌ها برابر** ⇐ هر درخت پوشا MST است، پس BFS یا DFS در $O(V+E)$ کافی است.

✏️ مثال

یال‌ها (4) a-b، (1) a-c، (2) b-c، (5) b-d، (8) c-d .

- **کراسکال:** (1) a-c ✓، (2) b-c ✓، (4) a-b ✗ (دور a-b-c می‌سازد)، (5) b-d ✓، (8) c-d ✗. وزن MST: 1 + 2 + 5 = 8.
- **پریم از a:** سبک‌ترین یال خروجی از {a} یال a-c است؛ از {a, c} یال b-c ؛ از {a, b, c} یال b-d . همان درخت.
- **خاصیت برش:** برش {d} در برابر بقیه: یال‌های (5) b-d و (8) c-d ؛ سبک‌ترین (b-d) در MST هست.
- **خاصیت دور:** در دور a-b-c سنگین‌ترین یال (4) a-b است و در MST نیست؛ در دور b-c-d هم (8) c-d .
- **وزن‌های برابر:** مثلث با سه یال وزن ۱ سه MST دارد (هر دو یال دلخواه).

الگوریتم	زمان
کراسکال	$O(E \log E)$ (مرتب‌سازی) + Union-Find. اگر یال‌ها **از قبل مرتب** باشند: $O(E \cdot \alpha)$ ، که برای گراف کامل $O(n^2)$ است
پریم با هیپ دودویی	$O(E \log V)$
پریم با هیپ فیبوناچی	$O(E + V \log V)$ ، که هیچ‌وقت بدتر از کراسکال نیست
گراف مسطح	$\Theta(n)$ (بروفکا با فشرده‌سازی؛ چون $E = O(V)$ و ادغام رأس‌ها مسطح بودن را حفظ می‌کند)

✏️ مثال (گراف متراکم، n = ۱۰۰۰ و E ≈ ۵×۱۰⁵)

کراسکال حدود $E \log E \approx 9.5 \times 10^6$ ، پریم با هیپ دودویی حدود $E \log V \approx 5 \times 10^6$ ، و پریم با هیپ فیبوناچی حدود $E + V \log V \approx 5.1 \times 10^5$ قدم.

⚠️ تلهٔ تستی

برای گراف وزن‌دار **کلی**، الگوریتم قطعی MST در زمان خطی شناخته شده نیست. پس «MST در زمان خطی» در تست‌ها معمولاً **نشدنی** حساب می‌شود.

درس ۳ کوتاه‌ترین مسیر

الگوریتم	شرط	زمان
BFS	بدون وزن	$O(V + E)$
DAG (ترتیب توپولوژیک)	بدون دور، وزن منفی مجاز	$O(V + E)$
دایکسترا	**وزن نامنفی**	$O((V + E) \log V)$

الگوریتم	شرط	زمان
بلمن-فورد	وزن منفی مجاز؛ دور منفی را **تشخیص می‌دهد**	$O(VE)$
فلوید-وارشال	همهٔ جفت‌ها	$O(V^3)$

مثال برای هر سطر جدول

- **DAG با وزن منفی:** $a\to b\ (2)$ ، $a\to c\ (-1)$ ، $c\to b\ (1)$ ، $b\to d\ (3)$. با ترتیب توپولوژیک $d(c) = : a, c, b, d$ ، -1 . $d(b) = \min(2, -1 + 1) = 0$ ، $d(d) = 3$.
- **بلمن-فورد:** دور $a\to b\ (1)$ ، $b\to c\ (-2)$ ، $c\to a\ (-1)$ جمع -2 دارد. بعد از $V - 1 = 2$ دور آزادسازی، دور سوم هنوز فاصله‌ای را کم می‌کند ⇐ دور منفی تشخیص داده می‌شود.
- **فلوید-وارشال:** $1\to 2\ (4)$ ، $2\to 3\ (1)$ ، $1\to 3\ (7)$. با $k = 2$ (اجازهٔ عبور از ۲): $d[1][3] = \min(7, 4 + 1) = 5$.

ترفندهای مدل‌سازی

- **فاصله تا یک مجموعه** ($d(u, S) = \min \delta(u, v)$): یال‌ها را وارونه کن، یک منبع مجازی با یال‌های وزن ۰ به همهٔ S وصل کن، و **یک بار** دایکسترا بزن: $O((n + m) \log n)$.
- **نرخ ارز (آربیتراژ):** وزن یال $-\log(\text{نرخ})$ است. **دور منفی** یعنی دوری که حاصل‌ضرب نرخ‌هایش بیشتر از ۱ است، پس سود **بی‌نهایت** با تکرار آن دور.
- **دایکسترا با وزن منفی:**
  - اگر فقط یال‌های **متصل به s** منفی باشند، کار می‌کند: هر مسیر ساده دقیقاً یکی از آن‌ها را دارد، پس با افزودن یک ثابت به همه، مسئله همان می‌ماند.
  - اگر فقط **پل‌ها** منفی باشند هم کار می‌کند: همهٔ مسیرهای ساده به یک رأس از همان پل‌ها می‌گذرند.
  - اگر یال منفی روی یک **دور** باشد، ممکن است اشتباه کند (مثلاً مثلث با یک یال منفی).

مثال

- **یال‌های متصل به s:** $s\to a\ (-3)$ ، $s\to b\ (-1)$ ، $b\to a\ (1)$. به یال‌های s عدد ۳ اضافه کن: $s\to a\ (0)$ ، $s\to b\ (2)$. هر مسیر از s دقیقاً یک یال s دارد، پس طول همهٔ مسیرها دقیقاً ۳ زیاد شده و ترتیبشان عوض نمی‌شود.
- **شکست:** $s\to a\ (2)$ ، $s\to b\ (3)$ ، $b\to a\ (-2)$. دایکسترا a را با فاصلهٔ ۲ قطعی می‌کند، ولی مسیر $s\to b\to a$ طول ۱ دارد ✗.

مثال

دلار به یورو ۰٫۹، یورو به پوند ۰٫۸، پوند به دلار ۱٫۵. حاصل‌ضرب $0.9 \times 0.8 \times 1.5 = 1.08 > 1$ ⇐ هر دور ۸٪ سود. جمع وزن‌ها $-\log 1.08 < 0$ ⇐ دور منفی.

درس ۴ جریان بیشینه

- **جریان بیشینه = برش کمینه.**
- **فورد-فالکرسون** با **گراف باقی‌مانده** (یال‌های برگشتی برای پس گرفتن جریان) درست است. ادموندز-کارپ $O(VE^2)$ است.
- **بدون یال برگشتی** (فقط مسیرهایی که ظرفیت خالی دارند، تا وقتی دیگر مسیری نماند) ممکن است **دلخواه بد** شود. مثال: یک مسیر طولانی که از وسط k مسیر جدا از هم می‌گذرد و همه را مسدود می‌کند؛ جریان ۱ در برابر بیشینهٔ k.

✏️ مثال (k = ۲)

s→a ،s→b ،a→b ،a→t ،b→t ، همه با ظرفیت ۱.

- اگر اول مسیر s→a→b→t انتخاب شود، s→a و b→t پر می‌شوند و بدون یال برگشتی مسیر دیگری نمی‌ماند ⇐ جریان ۱.
- با گراف باقی‌مانده: مسیر s→b→a→t از یال برگشتی b→a می‌گذرد (جریان روی a→b را پس می‌گیرد) ⇐ جریان ۲ ✓ (بیشینه).

درس ۵ گراف و ضرب ماتریس

- $A^2[i][j]$ تعداد همسایه‌های مشترک i و j است. پس «جفت‌هایی با همسایهٔ مشترک» در زمان ضرب ماتریس $O(n^\omega)$ با $\omega < 2.38$ حل می‌شود.
- **کوتاه‌ترین مسیر همهٔ جفت‌ها با وزن** در زمان $O(n^{(3-\varepsilon)})$ شناخته شده نیست.

✏️ مثال

مسیر 3 – 2 – 1 : $A^2[1][3] = 1$ (همسایهٔ مشترک ۲) و $A^2[2][2] = 2$ (درجهٔ ۲؛ روی قطر، درجه می‌آید).

✍️ تمرین فصل ۸ — اول خودت حل کن، بعد پاسخ‌نامه

۱. گراف جهت‌دار: s→a (4) ، s→b (1) ، b→a (2) ، a→c (1) ، b→c (5) . فاصله‌های دایکسترا از s؟

۲. کراسکال روی a-b (1) ، b-c (2) ، a-c (3) ، c-d (4) ، b-d (5) . وزن MST؟

۳. DAG با یال‌های a→b، a→c، b→d، c→d چند ترتیب توپولوژیک دارد؟

۴. اگر BFS از **هر** رأس ارتفاع حداکثر ۲ داشته باشد، تعداد یال‌ها چه بازه‌ای می‌تواند داشته باشد؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۱۳**

📝 آزمون فصل ۸

سؤال‌های واقعی کنکور ارشد کامپیوتر

۱۳ سؤال | ⏱️ زمان: ۳۵ دقیقه برای ۱۳ سؤال | 🔑 کلید و راه‌حل: صفحهٔ ۱۱۳

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۳ · سؤال ۵۸ — غلط یا حدسی؟ ☐

فرض کنید $G = (V, E)$ یک گراف همبند وزن‌دار باشد. چند مورد از گزاره‌های زیر درست است؟

- اگر وزن تمام یال‌های گراف با هم برابر باشد، می‌توان درخت فراگیر کمینهٔ آن را با الگوریتمی از مرتبهٔ $O(|E|)$ به‌دست آورد.
- اگر G گراف جهت‌دار باشد، یافتن دور در این گراف را می‌توان در مرتبهٔ $O(|V| + |E|)$ محاسبه کرد.
- چنانچه وزن یال‌های گراف دوبه‌دو متمایز باشند، الگوریتم پریم و کروسکال دارای جواب یکسانی هستند.
- الگوریتم پریم را می‌توان به‌نحوی پیاده‌سازی کرد که همواره مرتبهٔ آن بدتر از الگوریتم کروسکال نباشد.

۱) ۱ ۲) ۲ ۳) ۳ ۴) ۴

۲ ارشد ۱۴۰۲ · سؤال ۶۰ — غلط یا حدسی؟ ☐

کدام‌یک از مسائل زیر در زمان خطی برحسب تعداد رئوس و یال‌های گراف ورودی قابل‌حل **نیست**؟

۱) تشخیص همبندی گراف ساده
۲) تشخیص دوبخشی‌بودن گراف ساده
۳) پیداکردن درخت پوشای کمینهٔ گراف وزن‌دار همبند
۴) پیداکردن ترتیب توپولوژیکی رئوس گراف غیرحلقوی جهت‌دار (DAG)

۳ ارشد ۱۴۰۱ · سؤال ۶۱ — غلط یا حدسی؟ ☐

گراف جهت‌دار و وزن‌دار $G = (E, V)$ با n رأس و $O(n)$ یال و زیرمجموعهٔ $S \subset V$ از رأس‌های گراف با اندازهٔ حداقل $n/2$ داده شده است. فرض کنید وزن تمام یال‌های گراف مثبت است. به‌ازای هر رأس u از گراف، فاصلهٔ رأس u از مجموعهٔ S که آن را با $d(u, S)$ نمایش می‌دهیم عبارت است از $d(u, S) = \min_{v \in S} \delta(u, v)$ ، که در آن $\delta(u, v)$ برابر با طول کوتاه‌ترین مسیر جهت‌دار از u به v در گراف G است. در چه مرتبهٔ زمانی می‌توان مقادیر $d(u, S)$ را به‌ازای تمام رأس‌های u از گراف محاسبه کرد؟ دقت کنید که خروجی شامل n مقدار $d(u, S)$ به‌ازای تمام رأس‌های u از گراف است. (بهترین گزینه را انتخاب کنید.)

۱) $O(n^3)$ ۲) $O(n^2)$ ۳) $O(n \log^2 n)$ ۴) $O(n \log n)$

۴ ارشد ۱۴۰۱ · سؤال ۶۲ — غلط یا حدسی؟

فرض کنید G یک گراف وزن‌دار و مسطح با n رأس است. درخت پوشای کمینهٔ G را در چه زمانی می‌توان محاسبه کرد؟ توجه کنید لزومی ندارد از الگوریتم‌های معروف برای محاسبهٔ درخت پوشای کمینه استفاده شود. (بهترین گزینه را انتخاب کنید.)

۱) $\Theta(n)$
۲) $\Theta(n \log n)$
۳) $\Theta(n \log \log n)$
۴) $\Theta(n^2)$

۵ ارشد ۱۴۰۱ · سؤال ۷۲ — غلط یا حدسی؟

فرض کنید G یک گراف بدون جهت و بدون وزن با n رأس و m یال باشد. مسئلهٔ زیر را در نظر بگیرید: به‌ازای دو رأس u و v داده‌شده و پارامتر ورودی k، آیا تعداد کوتاه‌ترین مسیرها بین u و v حداقل k است؟ کدام گزینه در مورد این مسئله درست است؟

۱) می‌توان الگوریتمی با زمان اجرای چندجمله‌ای برای این مسئله ارائه داد، اما زمان اجرای این الگوریتم نمی‌تواند برحسب n و m خطی باشد.
۲) می‌توان الگوریتمی با زمان اجرای $O(m + n)$ برای این مسئله ارائه داد.
۳) این یک مسئلهٔ ان‌پی-تمام است.
۴) این یک مسئلهٔ ان‌پی-سخت است.

۶ ارشد ۱۴۰۰ · سؤال ۶۵ — غلط یا حدسی؟

کدام‌یک از مسائل زیر را می‌توان در زمان $O(n^{2.5})$ حل کرد؟

- الف) پیداکردن کوتاه‌ترین مسیر بین هر دو رأس در گراف وزن‌دار با n رأس
- ب) ضرب دو ماتریس $n \times n$
- ج) پیداکردن تعداد جفت رئوسی که همسایهٔ مشترک دارند در یک گراف n رأسی

۱) الف و ب
۲) الف و ج
۳) ب و ج
۴) هیچ‌یک از موارد فوق

۷ ارشد ۱۴۰۰ · سؤال ۶۶ — غلط یا حدسی؟

فرض کنید 100T در اختیار داریم. جدول نرخ تبدیل ارزها در زیر داده شده است. به‌عنوان نمونه طبق جدول زیر هر 1E برابر 30T می‌باشد. می‌خواهیم با چندین بار تبدیل پول و نهایتاً تبدیل آن به T درآمد کسب کنیم. چه میزان درآمد می‌توانیم کسب کنیم؟ (توجه کنید که 100T اولیه درآمد حساب نمی‌شود.)

۱ واحدِ ... برابر است با	T	P	E	D
D	۲۵	1/1,4	1/1,2	۱
E	۳۰	1/1,1	۱	1,2
P	۳۵	۱	1,1	1,4
T	۱	1/35	1/30	1/25

۱) صفر
۲) 102T
۳) 1400T
۴) به هر میزان که بخواهیم

۸ ارشد ۱۴۰۰ · سؤال ۶۹ — غلط یا حدسی؟

گراف بدون جهت و وزن‌دار G و رأس مشخص s از این گراف را درنظر بگیرید. از الگوریتم دایکسترا برای محاسبهٔ کوتاه‌ترین مسیر سادهٔ از s به بقیهٔ رئوس استفاده کرده‌ایم. به‌ازای چند حالت زیر این الگوریتم با وجود وزن‌های منفی همیشه درست کار می‌کند؟

- هر یالی بتواند وزن منفی داشته باشد.
- تنها یال‌های منتهی به s بتوانند وزن منفی داشته باشند.
- تنها یال‌های برشی گراف G بتوانند وزن منفی داشته باشند.
- به‌ازای هر دور از گراف G حداکثر یک یال بتواند وزن منفی داشته باشد.

۱) ۰
۲) ۱
۳) ۲
۴) ۳

۹ ارشد ۱۳۹۹ · سؤال ۹۰ — غلط یا حدسی؟

فرض کنید T درخت جستجوی عمق اول گراف همبند و بدون جهت G است. دو رأس u و v در این درخت برگ و در G دارای درجهٔ حداقل ۲ هستند. کدام‌یک از گزاره‌های زیر صحیح است؟

- الف) باید یک رأس w وجود داشته باشد که با u و v در G همسایه باشد.
- ب) باید یک رأس w وجود داشته باشد که حذف آن u را از v در G جدا می‌کند.

۱) الف نادرست، ب نادرست
۲) الف نادرست، ب درست
۳) الف درست، ب نادرست
۴) الف درست، ب درست

۱۰ ارشد ۱۳۹۹ · سؤال ۹۳ — غلط یا حدسی؟

گراف همبند و وزن‌دار G را در نظر بگیرید. درخت پوشای کمینهٔ G لزوماً یکتا نیست و G می‌تواند چندین درخت پوشای کمینه داشته باشد. فرض کنید T یک درخت پوشای کمینهٔ دلخواه از G باشد. کدام‌یک از گزاره‌های زیر درست هستند؟

- الف) حتماً یک اجرا از الگوریتم پریم وجود دارد که T را تولید کند.
- ب) حتماً یک اجرا از الگوریتم کروسکال وجود دارد که T را تولید کند.

توجه کنید زمانی‌که وزن یال‌ها متمایز نیست، می‌توان اجراهای متفاوتی برای هر دو الگوریتم پریم و کروسکال در نظر گرفت. در واقع وقتی چندین یال دارای وزن یکسان باشند، می‌توان هر ترتیبی از آن‌ها را برای پردازش مورد نظر الگوریتم‌های فوق در نظر گرفت.

۱) الف درست، ب درست
۲) الف درست، ب نادرست
۳) الف نادرست، ب درست
۴) الف نادرست، ب نادرست

۱۱ ارشد ۱۳۹۸ · سؤال ۵۶ — غلط یا حدسی؟

در گراف همبند بدون جهت $G = (V, E)$ شامل n رأس، اگر از هر رأس BFS را اجرا کنیم، ارتفاع درخت BFS حداکثر ۲ می‌شود. کدام گزینه در خصوص تعداد یال‌های این گراف درست است؟

۱) $|E| = \Theta(n)$
۲) $|E| = \Theta(n^2)$
۳) $|E| = \Omega(n \log n)$
۴) برای هر $n - 1 \le i \le n(n-1)/2$ می‌توان گرافی با i یال مثال زد که این ویژگی را داشته باشد.

۱۲ ارشد ۱۳۹۸ · سؤال ۵۸ — غلط یا حدسی؟

فرض کنید در یک شبکهٔ شار برای محاسبهٔ شار از رأس s به رأس t از الگوریتم زیر استفاده کنیم: شار عبوری از همهٔ یال‌ها را صفر قرار می‌دهیم. در هر مرحله یک مسیر از s به t انتخاب می‌کنیم که یال‌های مسیر همچنان ظرفیت خالی داشته باشند. شار همهٔ یال‌های این مسیر را به اندازهٔ یکسان افزایش می‌دهیم طوری که حداقل یک یال مسیر ظرفیتش تکمیل شود. این کار را آن‌قدر ادامه می‌دهیم تا چنین مسیری یافت نشود. در مورد اندازهٔ شار حاصل از این الگوریتم کدام مورد درست است؟

۱) با شار بیشینه برابر است.
۲) حداقل به اندازهٔ نصف شار بیشینه است.
۳) حداقل به اندازهٔ یک چهارم شار بیشینه است.
۴) می‌توان شبکه‌ای مثال زد که برای آن خروجی الگوریتم فوق کمتر از 0.0001 شار بیشینه باشد.

۱۳ ارشد ۱۳۹۸ · سؤال ۶۰ غلط یا حدسی؟

گراف وزن‌دار، همبند و بدون جهت G با n رأس را در نظر بگیرید. می‌خواهیم درخت پوشای کمینه را با استفاده از الگوریتم کروسکال محاسبه کنیم. اگر یال‌ها را به‌ترتیب وزن داشته باشیم و دیگر در الگوریتم کروسکال نیاز به مرتب کردن یال‌ها نباشد، الگوریتم کروسکال در چه زمانی قابل پیاده‌سازی است؟

۱) $O(n^2 \log n)$ ۲) $O(n^2)$ ۳) $O(n \log n)$ ۴) $O(n)$

کارت خلاصهٔ فصل ۸

برای مرور سریع، شب قبل از آزمون

- ✔ BFS و DFS $O(V+E)$. در DFS بی‌جهت فقط یال درختی و عقب داریم. دور جهت‌دار ⇔ یال عقب. شمارش کوتاه‌ترین مسیرها با BFS.
- ✔ MST: برش و دور. وزن متمایز ⇐ یکتا. وزن‌های برابر ⇐ BFS. هر MST از پریم یا کراسکال به دست می‌آید. کراسکال با یال‌های مرتب $O(E \cdot \alpha)$. مسطح $\Theta(n)$. MST وزن‌دار کلی خطیِ قطعی ندارد.
- ✔ دایکسترا فقط با وزن نامنفی (استثناها: منفی فقط روی یال‌های s یا فقط روی پل‌ها). بلمن-فورد دور منفی را تشخیص می‌دهد. فاصله تا مجموعه: گراف وارونه + منبع مجازی.
- ✔ نرخ ارز: حاصل‌ضرب بیشتر از ۱ روی یک دور ⇐ سود نامحدود.
- ✔ جریان: باقی‌مانده لازم است؛ بدون آن، نتیجه دلخواه بد می‌شود. همسایهٔ مشترک = A^2.

پاسخ‌نامهٔ فصل ۸

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۸

برگشت ← صفحهٔ ۱۰۷

۱. $b=1$ ، $a=3$ (از راه b)، $c=4$

۲. $1+2+4=7$

۳. دو تا: $a\ b\ c\ d$ و $a\ c\ b\ d$

۴. هر عددی از $n-1$ (ستاره) تا $n(n-1)/2$ (کامل). اضافه کردن یال به ستاره قطر را بیشتر از ۲ نمی‌کند.

آزمون فصل ۸ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۱۰۸

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷	سؤال ۸	سؤال ۹	سؤال ۱۰	سؤال ۱۱
۴	۳	۴	۱	۲	۳	۴	۳	۱	۱	۴

سؤال ۱۲	سؤال ۱۳
۴	۲

۱ — گزینهٔ ۴ (هر چهار)

وزن‌های برابر یعنی هر درخت پوشا MST است (BFS در $O(E)$ برای گراف همبند). دور جهت‌دار با DFS پیدا می‌شود. وزن متمایز یعنی MST یکتا. پریم با هیپ فیبوناچی $O(E + V \log V)$ هیچ‌وقت بدتر از کراسکال $O(E \log V)$ نیست.

۲ — گزینهٔ ۳

همبندی، دوبخشی بودن و توپولوژیک همه $O(V+E)$ هستند. MST وزن‌دار کلی، الگوریتم قطعی خطی شناخته‌شده ندارد.

۳ — گزینهٔ ۴

گراف را وارونه کن و یک منبع مجازی با یال ۰ به همهٔ S بگذار. یک بار دایکسترا: $O((n+m)\log n) = O(n \log n)$.

۴ — گزینهٔ ۱

در گراف مسطح $E = O(n)$. بروفکا با فشرده‌سازی و حذف یال‌های موازی، در هر مرحله رأس‌ها را حداقل نصف می‌کند و گراف مسطح می‌ماند، پس جمع کار $\Theta(n)$ است.

۵ **گزینهٔ ۲**

BFS با شمارندهٔ کوتاه‌ترین مسیرها (که روی k متوقف می‌شود): $O(m + n)$.

۶ **گزینهٔ ۳**

ضرب ماتریس در $O(n^\omega)$ با $\omega < 2.38 < 2.5$ ممکن است (با الگوریتم‌های پیشرفته، نه استراسن که 2.81 است). شمردن همسایهٔ مشترک با A^2 همان زمان را دارد. کوتاه‌ترین مسیر وزن‌دار همهٔ جفت‌ها در $O(n^2.5)$ شناخته‌شده نیست. (این سؤال «با تأثیر مثبت» است.)

۷ **گزینهٔ ۴**

دور $T \to E \to P \to T$: $1T = 1/30\ E$ ، و $1E = 1/1{,}1\ P$ ، و $1P = 35T$. حاصل‌ضرب $(1/30)(1/1{,}1)(35) = 35/33 \approx 1{,}06 > 1$. هر بار گردش ۶٪ سود می‌دهد، پس سود نامحدود است (یعنی دور منفی با وزن‌های $-\log$).

۸ **گزینهٔ ۳ (دو حالت)**

حالت‌های «فقط یال‌های متصل به s» و «فقط پل‌ها» درست کار می‌کنند (درس ۳).

- حالت اول غلط است.
- حالت چهارم هم غلط است. مثال: مثلث با $s\text{–}a = 1$ ، $s\text{–}b = 1$ و $a\text{–}b = -5$. دایکسترا a را با فاصلهٔ ۱ نهایی می‌کند، ولی مسیر سادهٔ $s\text{–}b\text{–}a$ طول -4 دارد.

۹ **گزینهٔ ۱**

- مثال نقض الف: درخت DFS با مسیرهای $r\text{–}a\text{–}b\text{–}u$ و $a\text{–}c\text{–}v$ ، یال عقب $u\text{–}a$ و یال عقب $v\text{–}r$. u و v همسایهٔ مشترک ندارند.
- مثال نقض ب: $r\text{–}w$ ، و u و v هر دو فرزند w، با یال‌های عقب $u\text{–}r$ و $v\text{–}r$. حذف هیچ رأسی u و v را جدا نمی‌کند.

۱۰ **گزینهٔ ۱**

با شکستن مناسب تساوی‌ها (اول یال‌های T در هر گروه هم‌وزن)، هر دو الگوریتم T را می‌سازند.

۱۱ **گزینهٔ ۴**

ستاره ($n - 1$ یال) قطر ۲ دارد و افزودن هر یال قطر را بیشتر از ۲ نمی‌کند، تا گراف کامل.

۱۲ **گزینهٔ ۴**

بدون یال برگشتی، یک مسیر بد می‌تواند همه را مسدود کند (درس ۴). با k مسیر جدا از هم، نسبت $1/k$ است و برای $k > 10000$ کمتر از 0.0001 .

۱۳ **گزینهٔ ۲**

بدون مرتب‌سازی، کراسکال $O(E \cdot \alpha(n))$ است. در بدترین حالت $E = \Theta(n^2)$، پس $O(n^2)$.

فصل ۹

برنامه‌نویسی پویا

۹

۲ درس | ۵ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۵ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴. تیپ‌ها: «کدام روش مناسب‌تر است؟»، نوشتن رابطهٔ بازگشتی با LCS، و زمان و حافظهٔ مسائل کلاسیک.

🧩 **پیش‌نیاز** برنامه‌سازی فصل ۴ (بازگشت).

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ کی برنامه‌نویسی پویا؟ | ۲ مسائل کلاسیک | 📝 آزمون فصل

درس ۱ کی برنامه‌نویسی پویا؟

دو شرط لازم است:

۱. **زیرمسئله‌های هم‌پوشان:** بازگشت ساده یک زیرمسئله را بارها حل می‌کند (مثل فیبوناچی).

۲. **زیرساختار بهینه:** جواب بهینه از جواب‌های بهینهٔ زیرمسئله‌ها ساخته می‌شود.

✏️ **مثال**

- **هم‌پوشانی:** بازگشت سادهٔ $fib(5)$ روی هم ۱۵ فراخوانی دارد: $fib(3)$ دو بار و $fib(2)$ سه بار از نو حساب می‌شوند. با جدول، هر کدام فقط یک بار.
- **زیرساختار بهینه:** اگر کوتاه‌ترین مسیر تهران ← اصفهان ← شیراز از اصفهان بگذرد، تکهٔ تهران ← اصفهانِ آن هم کوتاه‌ترین است (وگرنه با جایگزینی، کل مسیر کوتاه‌تر می‌شد). ولی «بلندترین مسیر ساده» این خاصیت را ندارد.

روش	ویژگی
حافظه‌گذاری (بالا به پایین)	بازگشت + جدول؛ فقط زیرمسئله‌های لازم را حل می‌کند
جدول‌سازی (پایین به بالا)	حلقه؛ سربار بازگشت ندارد و معمولاً می‌شود حافظه را کم کرد

✏️ مثال (فیبوناچی)

- **حافظه‌گذاری:** fib(5) بازگشتی صدا زده می‌شود، ولی هر fib(k) بار اول در جدول ذخیره می‌شود و دفعهٔ بعد فقط خوانده می‌شود.
- **جدول‌سازی:** $f[0] = 0$ ، $f[1] = 1$ و حلقه تا ۵: $1, 2, 3, 5$. چون هر خانه فقط به دو خانهٔ قبلی نیاز دارد، دو متغیر کافی است ($O(1)$ حافظه).

هر دو مرتبهٔ زمانی یکسان دارند. وقتی سؤال «مناسب‌ترین روش با کمترین پیچیدگی» را می‌پرسد و مسئله ترتیب ساده‌ای دارد، جواب معمولاً **برنامه‌نویسی پویای تکراری** است.

💡 قالب حل

(۱) حالت را تعریف کن، (۲) رابطهٔ انتقال، (۳) حالت پایه، (۴) ترتیب پر کردن جدول، (۵) جواب کجای جدول است.

✏️ مثال (قالب روی یک مسئله)

به چند روش می‌شود از ۴ پله بالا رفت، اگر هر قدم ۱ یا ۲ پله باشد؟

۱. **حالت:** $f(i)$ = تعداد روش‌های رسیدن به پلهٔ i.

۲. **انتقال:** آخرین قدم ۱ یا ۲ پله بوده: $f(i) = f(i-1) + f(i-2)$.

۳. **پایه:** $f(0) = 1$ ، $f(1) = 1$.

۴. **ترتیب:** i صعودی.

۵. **جواب:** $f(4)$: $f(2) = 2$ ، $f(3) = 3$ ، $f(4) = 5$.

درس ۲ مسائل کلاسیک

مسئله	رابطه	زمان و حافظه
بیشترین مجموع بدون دو عضو متوالی	$f(i) = \max(f(i-1), f(i-2) + a[i])$	$O(n)$ و $O(1)$
LCS	اگر برابرند $d[i][j] = d[i-1][j-1] + 1$ ، وگرنه $\max(d[i-1][j], d[i][j-1])$	$O(mn)$ ؛ فقط طول با $O(\min)$ حافظه
فاصلهٔ ویرایش	min(حذف + 1، درج + 1، جایگزینی + [نابرابر])	$O(mn)$
LIS	$O(n^2)$ ساده؛ $O(n \log n)$ با دودویی	
کوله‌پشتی ۰ و ۱	$d[i][w] = \max(d[i-1][w], d[i-1][w-w_i] + v_i)$	$O(nW)$ (**شبه‌چندجمله‌ای**)
ضرب زنجیره‌ای ماتریس	$m[i][j] = \min$ over k $(m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j)$	$O(n^3)$

مسئله	رابطه	زمان و حافظه
بلندترین زیردنبالهٔ پالیندروم (LPS)	LCS(s, وارون s)	$O(n^2)$

✏️ **مثال برای هر سطر جدول**

- **بدون دو متوالی** در [3, 2, 7, 10] : f = 3, 3, 10, 13 ⇐ **۱۳** (3 + 10).
- **LCS** برای ABC و AC (سطرها حروف ABC، ستون‌ها حروف AC):

	A	C
A	۱	۱
B	۱	۱
C	۱	**۲**

در خانهٔ (C, C) حرف‌ها برابرند: قطری ۱ + ۱ = ۲ ⇐ LCS = 2 (AC).

- **فاصلهٔ ویرایش:** cat ⇐ cut با یک جایگزینی (۱). ab ⇐ ba با دو عمل (۲).
- **LIS** در [3, 1, 4, 1, 5] : نسخهٔ n^2 : طول‌ها 1, 1, 2, 1, 3 ⇐ ۳. نسخهٔ دودویی (آرایهٔ «کوچک‌ترین انتهای هر طول»): [3] ، [1] ، [1, 4] ، [1, 4] ، [1, 4, 5] ⇐ ۳.
- **کوله‌پشتی ۰ و ۱:** W = 5 و کالاهای (وزن، ارزش) (2, 3) ، (3, 4) ، (4, 5) . بهترین: اولی و دومی با وزن ۵ و ارزش **۷**. **شبه‌چندجمله‌ای:** اگر $W = 10^9$ باشد، جدول یک میلیارد ستون دارد؛ در حالی که W در ورودی فقط حدود ۳۰ بیت است.
- **ضرب زنجیره‌ای:** برای ۳ ماتریس، k دو انتخاب دارد: (AB)C یا A(BC) (تمرین ۵).
- **LPS** در AGBCBA : وارونش ABCBGA است و LCS آن دو ABCBA با طول **۵** است.

چند تبدیل مهم

- **کمترین درج یا حذف برای پالیندروم کردن:** $n - LPS = n - LCS(A, \hat{A})$ ($\hat{A}$ وارون A است).
- **LCS دو جایگشت از ۱ تا n:** به‌جای هر عدد جایگشت دوم، جایش در جایگشت اول را بنویس. حالا LIS آن دنباله را بگیر: $O(n \log n)$.
- **بلندترین زیردنبالهٔ «دوتایی» ww :** w باید در نیمهٔ اول و نیمهٔ دوم رشته باشد، پس جواب $\max_p 2 \cdot LCS(S[1..p], S[p+1..n])$ است.

✏️ مثال برای هر تبدیل

- **پالیندروم کردن:** AGBCBA با LPS = 5 ⇐ 6 − 5 = 1 عمل (حذف G، یا درج یک G در جای قرینه).
- **LCS دو جایگشت:** $P_1 = [3, 1, 2, 4]$ و $P_2 = [1, 2, 3, 4]$. جای عددها در P_1: 1→3، 2→1، 3→2، 4→4 . پس P_2 می‌شود [2, 3, 1, 4] و LIS آن (2, 3, 4) طول ۳ دارد ⇐ LCS = 3 (یعنی 1, 2, 4).
- **ww :** در abab با p = 2 : LCS(ab, ab) = 2 ⇐ جواب ۴ (w = ab). در aabb بهترین p جواب ۱ می‌دهد ⇐ جواب ۲ (aa یا bb).

✍️ تمرین فصل ۹

اول خودت حل کن، بعد پاسخ‌نامه

۱. بیشترین مجموع بدون دو عضو متوالی در [2, 7, 9, 3, 1] ؟

۲. LCS("ABCBDAB", "BDCABA") ؟

۳. فاصلهٔ ویرایش kitten و sitting ؟

۴. طول LIS در [3, 1, 4, 1, 5, 9, 2, 6] ؟

۵. ضرب زنجیره‌ای ماتریس‌ها با ابعاد 10×30 ، 30×5 و 5×60 : کمترین تعداد ضرب؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: صفحهٔ ۱۲۲

📝 آزمون فصل ۹

سؤال‌های واقعی کنکور ارشد کامپیوتر

۵ سؤال | ⏱️ زمان: ۱۲ دقیقه | 🔑 کلید و راه‌حل: **صفحهٔ ۱۲۲**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۴ · سؤال ۶۱ — غلط یا حدسی؟ ☐

فرض کنید می‌خواهید از میان دنباله‌ای از اعداد صحیح غیرمنفی تعدادی را انتخاب کنید به‌گونه‌ای که به بیشترین مجموع دست یابید، با این شرط که نمی‌توانید دو عنصر متوالی را انتخاب کنید. به‌عبارت دیگر، هر عددی که انتخاب می‌شود، عدد بعد از آن در دنباله نمی‌تواند انتخاب شود. کدام مورد، مناسب‌ترین روش با کمترین پیچیدگی زمانی برای حل این مسئله است؟

۱) استفاده از الگوریتم بازگشتی به همراه حافظه‌گذاری (Memorization) برای ذخیرهٔ نتایج زیرمسئله‌ها و جلوگیری از محاسبهٔ مجدد آن‌ها

۲) استفاده از الگوریتم پویا (Dynamic Programming) که به‌صورت تکراری مقدار بهینه را از سمت چپ به راست محاسبه می‌کند.

۳) استفاده از الگوریتم تقسیم و حل (Divide and Conquer) برای تقسیم مسئله به زیرمسئله‌های کوچک‌تر

۴) استفاده از الگوریتم جستجوی کامل (Brute Force) که تمام ترکیب‌های ممکن را بررسی می‌کند.

۲ ارشد ۱۴۰۳ · سؤال ۵۹ — غلط یا حدسی؟ ☐

رشته‌هایی که از دو طرف یکسان خوانده می‌شوند پالیندروم (Palindrome) نامیده می‌شوند (مانند abcba). برای محاسبهٔ بزرگ‌ترین زیررشتهٔ پالیندروم یک رشته به طول n، یک الگوریتم پویای کارا به‌ترتیب از راست به چپ دارای چه مرتبهٔ زمان و حافظه است؟

۱) $O(n^2)$ ،$O(n^2)$ ۲) $O(n^2)$ ،$O(n)$ ۳) $O(n \log n)$ ،$O(n)$ ۴) $O(n)$ ،$O(n)$

۳ ارشد ۱۴۰۳ · سؤال ۶۰ — غلط یا حدسی؟

فرض کنید s رشته‌ای به طول n باشد. می‌خواهیم بزرگ‌ترین زیررشته به شکل ww را در این آرایه بیابیم که طول آن را با $LDS(S)$ (Longest Double String) نشان می‌دهیم. دراین‌صورت رابطهٔ بازگشتی طول بزرگ‌ترین زیررشته (LDS) چیست؟ (توجه کنید LCS تابعی است که طول بزرگ‌ترین زیررشتهٔ مشترک دو رشتهٔ ورودی را برمی‌گرداند.)

۱) $\max LCS(S[1..p], S[p+1..n])$ روی $1 \le p < n$

۲) $\max LCS(S[1..\lfloor p/2 \rfloor], S[\lceil p/2 \rceil..n])$

۳) $2 \max LCS(S[1..p], S[p+1..n])$

۴) $2 \max LCS(S[1..\lfloor p/2 \rfloor], S[\lceil p/2 \rceil..n])$

۴ ارشد ۱۴۰۰ · سؤال ۷۲ — غلط یا حدسی؟

دو دنباله که هر کدام یک جایگشت از اعداد ۱ تا n هستند، داده شده است. بزرگ‌ترین زیردنبالهٔ مشترک این دو دنباله را در چه زمانی می‌توان به‌دست آورد؟

۱) $O(n^2)$ ۲) $O(\log n)$ ۳) $O(n \log n)$ ۴) $O(n\sqrt{n})$

۵ ارشد ۱۳۹۸ · سؤال ۵۷ — غلط یا حدسی؟

رشتهٔ A شامل n کاراکتر را در نظر بگیرید. می‌خواهیم این رشته را به یک رشتهٔ آینه‌ای تبدیل کنیم. اعمال مجاز، حذف یک کاراکتر یا درج یک کاراکتر در هر جای رشته است. حداقل چند عمل نیاز است تا این کار انجام شود؟ ($\hat{A}$ رشتهٔ آینه‌ای A است و منظور از LCS و ED به‌ترتیب طول بزرگ‌ترین زیردنبالهٔ مشترک و فاصلهٔ ویرایشی (با فرض عمل‌های حذف، درج و جایگزینی) است.)

۱) $n - LCS(A, \hat{A})$ ۲) $n - ED(A, \hat{A})$ ۳) $LCS(A, \hat{A})$ ۴) $ED(A, \hat{A})$

کارت خلاصهٔ فصل ۹

برای مرور سریع، شب قبل از آزمون

- ✔ برنامه‌نویسی پویا = زیرمسئلهٔ هم‌پوشان + زیرساختار بهینه. حافظه‌گذاری و جدول هم‌مرتبه‌اند؛ تکراری سربار ندارد.
- ✔ بدون عضو متوالی $O(n)$. LCS و ویرایش $O(mn)$. LIS $O(n \log n)$. کوله‌پشتی $O(nW)$ شبه‌چندجمله‌ای است. زنجیرهٔ ماتریس $O(n^3)$.
- ✔ پالیندروم: $LPS = LCS(s, s^R)$ و کمترین عمل $n - LPS$. LCS دو جایگشت = LIS. ww یعنی $2 \cdot \max LCS(پیشوند, پسوند)$.

پاسخ‌نامهٔ فصل ۹

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۹

برگشت ← صفحهٔ ۱۱۹

۱. 12 ($2 + 9 + 1$)

۲. 4 (مثلاً $BCBA$)

۳. 3

۴. 4 (مثلاً $1, 4, 5, 9$)

۵. $(AB)C = 10{\cdot}30{\cdot}5 + 10{\cdot}5{\cdot}60 = 4500$. حالت دیگر $A(BC) = 27000$ است.

آزمون فصل ۹ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۱۲۰

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵
۲	۱	۳	۳	۱

۱ گزینهٔ ۲

$f(i) = \max(f(i-1),\, f(i-2) + a[i])$: یک حلقهٔ $O(n)$ با $O(1)$ حافظه. حافظه‌گذاری هم $O(n)$ است، ولی سربار بازگشت و پشته دارد؛ کلید روش تکراری را «مناسب‌ترین» دانسته.

۲ گزینهٔ ۱

جدول $d[i][j]$ برای بازهٔ $s[i..j]$ با رابطهٔ «اگر دو سر برابرند $2 + d[i+1][j-1]$ ، وگرنه $\max(d[i+1][j],\, d[i][j-1])$ » در $O(n^2)$ زمان و $O(n^2)$ حافظه پر می‌شود.

۳ گزینهٔ ۳

نسخهٔ اول w در $S[1..p]$ و نسخهٔ دوم در $S[p+1..n]$ است. پس طول w برابر LCS این دو بخش است و طول ww دو برابر آن. روی همهٔ p بیشینه بگیر.

۴ گزینهٔ ۳

هر عدد در هر جایگشت فقط یک بار می‌آید. پس LCS به LIS تبدیل می‌شود (درس ۲): $O(n \log n)$.

۵ گزینهٔ ۱

کاراکترهایی که در بلندترین زیردنبالهٔ پالیندروم ($LCS(A, \hat{A})$) نیستند، هر کدام یک عمل لازم دارند (حذف خودش یا درج جفتش). پس $n - LCS(A, \hat{A})$.

فصل ۱۰

۱۰ عقبگرد، شاخه و حد، و NPکامل (+ سؤال‌های متفرقه)

۴ درس | ۷ سؤال کنکور در آزمون فصل

🎯 **سهم در کنکور** ۷ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴ (۲ عقبگرد و شاخه و حد، ۳ NPکامل، ۲ متفرقه). سؤال‌ها بیشتر مفهومی‌اند و با دانستن تعریف‌ها در کمتر از یک دقیقه جواب داده می‌شوند.

⏱️ **زمان مطالعه** یک بلوک ۹۰ دقیقه‌ای.

نقشهٔ فصل: ۱ عقبگرد (Backtracking) | ۲ شاخه و حد (Branch and Bound) | ۳ NP، NP-سخت، NPکامل | ۴ متفرقه

📝 آزمون فصل

درس ۱ عقبگرد (Backtracking)

جواب‌ها را روی **درخت فضای حالت** می‌سازیم و هر شاخه‌ای که معلوم شد به جواب نمی‌رسد را **هرس** می‌کنیم.

مسئله	اندازهٔ فضای حالت (بدون هرس)
همهٔ زیرمجموعه‌ها یا زیردنباله‌ها	2^n برگ ($2^{n+1}-1$ گره)، پس $O(2^n)$
همهٔ جایگشت‌ها	$n!$
nوزیر	حداکثر $n!$ (با هرس خیلی کمتر)

هرس فقط زمان عملی را کم می‌کند. **بدترین حالت** همان اندازهٔ فضای حالت است.

✏️ **مثال برای هر سطر جدول**

- **زیرمجموعه‌ها** از $\{1, 2, 3\}$: در هر سطح «بردار یا نه». ۸ برگ و $1 + 2 + 4 + 8 = 15$ گره. **هرس:** برای «جمع ۵ با $\{2, 3, 4\}$ »، وقتی ۲ و ۴ برداشته شد (جمع ۶)، بقیهٔ آن شاخه بررسی نمی‌شود.
- **جایگشت‌ها** از $\{1, 2, 3\}$: سطح اول ۳ انتخاب، دوم ۲، سوم ۱ ⇐ $3! = 6$ برگ.
- **۴وزیر:** وزیر سطر ۱ در ستون ۱ و سطر ۲ در ستون ۳. در سطر ۳ هیچ ستون امنی نیست ⇐ برگرد و وزیر سطر ۲ را جابه‌جا کن. ۴وزیر فقط ۲ جواب دارد.

درس ۲ شاخه و حد (Branch and Bound)

- برای **بیشینه‌سازی**، در هر گره یک **کران بالا** (خوش‌بینانه) حساب کن. اگر کران بالا از بهترین جواب پیداشده تا الان بیشتر نباشد، آن شاخه را حذف کن.
- کران باید **معتبر** باشد (هیچ‌وقت کمتر از جواب واقعی نباشد) و هرچه **تنگ‌تر** باشد، هرس بیشتر است.
- **کران استاندارد برای مسائل شبیه کوله‌پشتی:** نسخهٔ **کسری** را حریصانه حل کن (به‌ترتیب نسبت سود به وزن یا مدت‌زمان). این کران همیشه معتبر و تنگ است.
- کران‌هایی مثل «جمع همهٔ سودهای باقی‌مانده» معتبرند ولی شل‌اند. «میانگین» یا «بیشترین سود یک پروژه» ممکن است معتبر نباشند.

✏️ **مثال**

کوله‌پشتی با ظرفیت ۶ و کالاهای (وزن، ارزش) $(2,3)$ ، $(3,4)$ ، $(4,5)$ با نسبت‌های 1.5 ، 1.33 ، 1.25 .

- **کران ریشه (کسری):** اولی و دومی کامل (وزن ۵، ارزش ۷) و $1/4$ از سومی (1.25) ⇐ 8.25 .
- جواب $\{1,3\}$ با وزن ۶ و ارزش **۸** پیدا شد.
- **شاخهٔ «بدون کالای ۱»:** کران = ۴ (کالای ۲) + $3/4 \times 5 = 3.75$ ⇐ $7.75 < 8$ ⇐ **هرس**.
- کران شل «جمع همهٔ ارزش‌ها» در ریشه ۱۲ می‌داد و تقریباً هیچ شاخه‌ای را حذف نمی‌کرد.

درس ۳ NP، NP سخت، NPکامل

کلاس	تعریف
P	در زمان چندجمله‌ای **حل** می‌شود
NP	جوابِ «بله» در زمان چندجمله‌ای **وارسی** می‌شود. $P \subseteq NP$
NPسخت	هر مسئلهٔ NP در زمان چندجمله‌ای به آن **کاهش** می‌یابد (لازم نیست خودش در NP باشد؛ مثلاً مسئلهٔ توقف)
NPکامل	NPسخت **و** در NP

✏️ مثال برای هر سطر جدول

- **P:** مرتب‌سازی، کوتاه‌ترین مسیر.
- **NP:** «آیا $\{3, 7, 9, 11\}$ زیرمجموعه‌ای با جمع ۲۰ دارد؟» اگر کسی بگوید $\{9, 11\}$ ، با یک جمع ساده وارسی می‌شود؛ پیدا کردنش سخت است، نه وارسی‌اش.
- **NPسخت ولی نه در NP:** مسئلهٔ توقف (حتی تصمیم‌پذیر هم نیست).
- **NPکامل:** SAT، که هم در NP است و هم همه چیز به آن کاهش می‌یابد.
- **هر NPکامل، NPسخت است.** ولی هر NPسخت NPکامل نیست.
- **جهت کاهش:** برای اثبات NPسخت بودن X، یک مسئلهٔ NPسختِ **شناخته‌شده** را به X کاهش بده ($\text{SAT} \leq_p X$).

✏️ مثال (کاهش)

مجموعهٔ مستقل $\geq_p$ خوشه: G مجموعهٔ مستقل به اندازهٔ k دارد $\Leftrightarrow$ مکمل G (یال‌ها برعکس) خوشه به اندازهٔ k دارد. ساختن مکمل چندجمله‌ای است، پس چون مجموعهٔ مستقل NPسخت است، خوشه هم هست.

NPکامل‌های معروف	چندجمله‌ای‌های شبیه
SAT، ۳SAT	**۲SAT**
۳رنگ‌پذیری	**۲رنگ‌پذیری** (دوبخشی)
مسیر یا دور همیلتونی، TSP	دور اویلری
خوشه، مجموعهٔ مستقل، پوشش رأسی	تطابق بیشینه
جمع زیرمجموعه، کوله‌پشتی (تصمیم)	کوله‌پشتی کسری
بلندترین مسیر ساده	کوتاه‌ترین مسیر

✏️ مثال (چرا هر جفت فرق دارد)

- **۲رنگ در برابر ۳رنگ:** با ۲ رنگ، رنگ یک رأس رنگ همسایه‌هایش را **اجبار** می‌کند (یک BFS کافی است). با ۳ رنگ، هر همسایه دو انتخاب دارد و انتخاب‌ها شاخه‌شاخه می‌شوند.
- **اویلری در برابر همیلتونی:** اویلری فقط با چک «همه درجه‌ها زوج و همبند» در $O(V + E)$ معلوم می‌شود. برای همیلتونی چنین شرط ساده‌ای شناخته نشده.
- **کسری در برابر ۰ و ۱:** کسری با حریصانه حل می‌شود (فصل ۷)؛ ۰ و ۱ نه.

⚠️ **تلهٔ تستی**

کوله‌پشتی و جمع زیرمجموعه با برنامه‌نویسی پویا در $O(nW)$ حل می‌شوند، ولی این **شبه‌چندجمله‌ای** است (W عدد است، نه طول ورودی). پس با NPکامل بودنشان تناقضی ندارد.

NPسخت‌ها روی درخت چندجمله‌ای می‌شوند

روی **درخت** (بدون وزن): بلندترین مسیر (قطر با دو BFS)، رنگ‌آمیزی (۲ رنگ، یا ۱ اگر یال نداشته باشد)، و بزرگ‌ترین مجموعهٔ مستقل (برنامه‌نویسی پویا روی درخت) همه **چندجمله‌ای**‌اند.

✏️ **مثال**

درخت با یال‌های 1-2، 2-3، 2-4، 4-5 .

- **قطر با دو BFS:** از ۱، دورترین رأس ۵ است (فاصلهٔ ۳). از ۵، دورترین ۱ یا ۳ است (فاصلهٔ ۳) ⇐ قطر **۳**.
- **مجموعهٔ مستقل:** روی مسیر 1-2-3-4 بزرگ‌ترینش ۲ عضو دارد (مثلاً $\{1, 3\}$). روی ستاره‌ای با ۴ برگ، خود ۴ برگ (۴ عضو).

kمجموع (k عضو با جمع صفر)

k	بهترین زمان رایج
۱	$O(n)$ (هر عضو را نگاه کن؛ $O(1)$ ممکن نیست)
۲	$O(n \log n)$ (مرتب‌سازی + دو اشاره‌گر)؛ $O(n)$ قطعی برای اعداد حقیقی شناخته نیست
۳	$O(n^2)$ (مرتب‌سازی + برای هر عضو، دو اشاره‌گر)

✏️ **مثال (۳مجموع)**

$[-4, -1, -1, 0, 1, 2]$ (مرتب). عضو اول را ثابت کن و در بقیه با دو اشاره‌گر دنبال جمع قرینه بگرد. با -1 (اولی): در $[-1, 0, 1, 2]$ دنبال جمع ۱ ⇐ $-1 + 2$ ✓ ⇐ $(-1, -1, 2)$ ؛ و $0 + 1$ ✓ ⇐ $(-1, 0, 1)$. هر عضو $O(n)$ کار دارد ⇐ $O(n^2)$.

درس ۴ متفرقه

- **پوستهٔ محدب:** $\Theta(n \log n)$ (گراهام). کران پایین از مرتب‌سازی می‌آید: نقاط روی سهمی (x, x^2) قرار بده؛ پوسته‌شان همان ترتیب مرتب است.
- **حریصانه، پویا یا تقسیم و حل؟** اگر هر سه جواب بهینه می‌دهند، **حریصانه** معمولاً سریع‌تر است، چون فقط یک انتخاب در هر قدم دارد و جدول یا بازگشت نمی‌خواهد.

✏️ مثال

- **پوسته:** نقاط $(1, 1)$ ، $(3, 9)$ ، $(2, 4)$ روی سهمی $y = x^2$ هستند. پوسته‌شان به ترتیب $x = 1, 2, 3$ است؛ پس پوسته ساختن، مرتب‌سازی هم هست.
- **سکه با $\{1, 5, 10\}$:** حریصانه و پویا هر دو بهینه‌اند، ولی حریصانه برای ۲۸ فقط ۶ انتخاب می‌کند، در حالی که پویا جدولی به اندازهٔ ۲۸ × تعداد سکه‌ها پر می‌کند.

✍️ تمرین فصل ۱۰

اول خودت حل کن، بعد پاسخ‌نامه

۱. درخت عقبگرد برای همهٔ زیرمجموعه‌های ۵ عضو چند گره دارد؟

۲. اگر $X \in NP$ و $SAT \leq_p X$ ، X چیست؟

۳. آیا ۲SAT در P است؟

۴. آیا برنامه‌نویسی پویای $O(nW)$ برای کوله‌پشتی، الگوریتم چندجمله‌ای است؟

🔑 پاسخ‌ها در پاسخ‌نامهٔ آخر فصل: **صفحهٔ ۱۳۱**

📝 آزمون فصل ۱۰

سؤال‌های واقعی کنکور ارشد کامپیوتر

۷ سؤال | ⏱️ زمان: ۱۲ دقیقه برای ۷ سؤال | 🔑 کلید و راه‌حل: **صفحهٔ ۱۳۱**

✔️ مثل جلسهٔ کنکور: زمان بگیر، روی هیچ سؤالی بیشتر از ۴ دقیقه نمان، و در مربع کنار سؤال‌های غلط یا حدسی علامت بزن تا در دور دوم فقط همان‌ها را حل کنی.

۱ ارشد ۱۴۰۴ · سؤال ۵۸ — غلط یا حدسی؟ ☐

فرض کنید می‌خواهیم تمام زیردنباله‌های یک دنبالهٔ داده‌شده را با استفاده از روش عقبگرد بررسی کنیم. اگر طول دنباله n باشد، پیچیدگی زمانی این روش چیست؟

۱) $O(2^n)$ ۲) $O(n^2)$ ۳) $O(n!)$ ۴) $O(\log n)$

۲ ارشد ۱۴۰۴ · سؤال ۵۹ — غلط یا حدسی؟ ☐

فرض کنید مسئله‌ای به نام انتخاب پروژه‌ها با بیشترین سود (Maximum Profit Project Selection) داریم. در این مسئله، تعدادی پروژه وجود دارد که در آن پروژهٔ i دارای سود P_i و مدت‌زمان d_i است. تنها تعداد مشخصی از پروژه‌ها را می‌توان به‌طور هم‌زمان انجام داد و کل مدت‌زمان انجام پروژه‌ها نباید از T بیشتر شود. هدف این است که بیشترین سود ممکن را از بین پروژه‌های انتخاب‌شده به‌دست آوریم. می‌خواهیم از الگوریتم شاخه و حد (Branch and Bound) برای حل این مسئله استفاده کنیم. کدام‌یک از روش‌های زیر، مناسب‌ترین روش برای تعیین حد بالا (Upper Bound) در هر گره است تا به سرعت تا شاخه‌های غیرممکن یا کم‌بازده حذف شوند؟

۱) استفاده از میانگین سود پروژه‌های باقی‌مانده و افزودن آن به سود فعلی در گرهٔ فعلی به‌عنوان حد بالا

۲) استفاده از بیشترین سود در پروژه‌های باقی‌مانده و افزودن آن به سود فعلی در گرهٔ فعلی به‌عنوان حد بالا

۳) استفاده از مجموع سود تمامی پروژه‌های باقی‌مانده و افزودن آن به سود فعلی در گرهٔ فعلی به‌عنوان حد بالا

۴) انتخاب پروژه‌ها به‌صورت حریصانه، براساس بیشترین نسبت سود به مدت‌زمان و افزودن آن‌ها تا زمانی‌که زمان کل از T فراتر نرود و سپس در نظر گرفتن نتیجه به‌عنوان حد بالا

۳ ارشد ۱۴۰۲ · سؤال ۵۸ — غلط یا حدسی؟ ☐

کدام‌یک از عبارات زیر درست است؟

- الف) هر مسئله‌ای NPکامل یک مسئله‌ای NPسخت است.
- ب) هر مسئله‌ای NPسخت یک مسئله‌ای NPکامل است.
- ج) هر مسئله‌ای P (چندجمله‌ای) یک مسئله‌ای NP هم هست.

۱) الف و ب ۲) الف و ج ۳) ب و ج ۴) الف، ب و ج

۴ ارشد ۱۴۰۱ · سؤال ۶۸ — غلط یا حدسی؟

مسئلهٔ kمجموع بدین شکل تعریف می‌شود: مجموعهٔ A از n عدد حقیقی و عدد k داده شده است. آیا k عضو از مجموعهٔ A وجود دارند که جمع آن‌ها صفر شود. چه تعداد از گزاره‌های زیر درست است؟

- مسئلهٔ ۱مجموع در زمان $O(1)$ قابل حل است.
- مسئلهٔ ۲مجموع در زمان $O(n)$ قابل حل است.
- مسئلهٔ ۳مجموع در زمان $O(n^2)$ قابل حل است.

۱) صفر ۲) ۳ ۳) ۲ ۴) ۱

۵ ارشد ۱۳۹۹ · سؤال ۹۲ — غلط یا حدسی؟

چند مورد از مسائل ان‌پی-سخت زیر هنگامی که ورودی یک درخت بدون وزن است، در زمان چندجمله‌ای قابل حل هستند؟

- پیدا کردن طولانی‌ترین مسیر
- محاسبهٔ کمینه رنگ مورد نیاز برای رنگ‌آمیزی رأسی به‌طوری‌که رئوس مجاور همرنگ نباشند.
- پیدا کردن بزرگ‌ترین مجموعهٔ مستقل (مجموعهٔ رئوسی که بین هر دو رأس یالی وجود نداشته باشد.)

۱) ۰ ۲) ۱ ۳) ۲ ۴) ۳

۶ ارشد ۱۴۰۲ · سؤال ۵۶ — غلط یا حدسی؟

در مسئلهٔ یافتن پوستهٔ محدب، تعدادی نقطه در صفحه داده شده است. هدف پیدا کردن کوچک‌ترین (کمترین محیط) چندضلعی محدب است که شامل همهٔ نقاط باشد. بهترین الگوریتم ممکن برای یافتن این چندضلعی، چه مرتبهٔ زمانی خواهد داشت؟

۱) $\theta(n^2)$ ۲) $\theta(n^3)$ ۳) $\theta(n \log n)$ ۴) $\theta(n^2 \log n)$

۷ ارشد ۱۴۰۲ · سؤال ۵۹ — غلط یا حدسی؟

فرض کنید که یک مسئله را به‌صورت بهینه به توان هم با روش تقسیم و حل، هم با روش برنامه‌ریزی پویا و هم با روش حریصانه حل کرد. در این‌صورت از لحاظ پیچیدگی زمانی کدام‌یک ارجحیت دارد؟

۱) حریصانه ۲) تقسیم و حل
۳) برنامه‌ریزی پویا ۴) به مسئله بستگی دارد.

🗂️ کارت خلاصهٔ فصل ۱۰

برای مرور سریع، شب قبل از آزمون

- ✔ عقبگرد: زیرمجموعه‌ها 2^n و جایگشت‌ها $n!$. هرس بدترین حالت را عوض نمی‌کند.
- ✔ شاخه و حد در بیشینه‌سازی: کران بالای معتبر و تنگ، یعنی **حل کسری حریصانه**.
- ✔ $P \subseteq NP$. NPکامل = NPسخت ∩ NP. NPسخت الزاماً در NP نیست. کاهش: از مسئلهٔ شناخته‌شده به مسئلهٔ جدید.
- ✔ ۲SAT، ۲رنگ و اویلری در P هستند؛ ۳SAT، ۳رنگ و همیلتونی NPکامل. $O(nW)$ شبه‌چندجمله‌ای است.
- ✔ روی درخت: بلندترین مسیر، رنگ‌آمیزی و مجموعهٔ مستقل همه چندجمله‌ای‌اند. ۳مجموع $O(n^2)$. پوستهٔ محدب $\Theta(n \log n)$.

پاسخ‌نامهٔ فصل ۱۰

تا تمرین‌ها و آزمون فصل را خودت حل نکرده‌ای، این صفحه‌ها را نخوان.

تمرین فصل ۱۰

برگشت ← صفحهٔ ۱۲۷

۱. $2^6 - 1 = 63$

۲. NPکامل

۳. بله (با مؤلفه‌های قویاً همبند، در $O(n + m)$).

۴. خیر، شبه‌چندجمله‌ای است.

آزمون فصل ۱۰ — کلید و راه‌حل

کلید رسمی سازمان سنجش

برگشت ← صفحهٔ ۱۲۸

سؤال ۱	سؤال ۲	سؤال ۳	سؤال ۴	سؤال ۵	سؤال ۶	سؤال ۷
۱	۴	۲	۴	۴	۳	۱

۱ **گزینهٔ ۱**

هر عضو یا هست یا نیست: 2^n زیردنباله.

۲ **گزینهٔ ۴**

کران کسری حریصانه (نسبت سود به مدت) معتبر و تنگ است.

- گزینهٔ ۳ معتبر ولی شل است.
- گزینه‌های ۱ و ۲ ممکن است از جواب واقعی کمتر شوند، پس معتبر نیستند.

۳ **گزینهٔ ۲**

الف و ج درست‌اند. ب غلط است؛ مثلاً مسئلهٔ توقف NPسخت است ولی در NP نیست.

۴ **گزینهٔ ۴ (فقط یکی)**

- ۱مجموع باید همهٔ n عدد را نگاه کند، پس $O(n)$ است، نه $O(1)$.
- ۲مجموع برای اعداد حقیقی (در مدل مقایسه‌ای) $\Omega(n \log n)$ است.
- فقط ۳مجموع در $O(n^2)$ درست است.

۵ **گزینهٔ ۴ (هر سه)**

قطر درخت با دو BFS؛ رنگ‌آمیزی با ۲ رنگ (درخت دوبخشی است)؛ مجموعهٔ مستقل بیشینه با برنامه‌نویسی پویا روی درخت.

۶ **گزینهٔ ۳**

گراهام یا تقسیم و حل $O(n \log n)$ ، و کران پایین $\Omega(n \log n)$ از مرتب‌سازی.

۷ **گزینهٔ ۱**

حریصانه در هر قدم فقط یک انتخاب می‌کند و زیرمسئله‌ها را جدول نمی‌کند، پس معمولاً کمترین زمان را دارد.